

Iterative algorithms for solving one-sided partially observable stochastic shortest path games

Petr Tomášek^{*}, Karel Horák, Branislav Božanský

Artificial Intelligence Center, Dept. of Computer Science, Faculty of Electrical Engineering, Czech Technical University in Prague, Czech Republic

ARTICLE INFO

Keywords:

Stochastic shortest path games
Stochastic games
Imperfect information
Partial observability
Indefinite horizon

ABSTRACT

Real-world scenarios often involve dynamic interactions among competing agents, where decisions are made considering actions taken by others. These situations can be modeled as partially observable stochastic games (POSGs), with zero-sum variants capturing strictly competitive interactions (e.g., security scenarios). While such models address a broad range of problems, they commonly focus on infinite-horizon scenarios with discounted-sum objectives. Using the discounted-sum objective, however, can lead to suboptimal solutions in cases where the length of the interaction does not directly affect the gained rewards of the players.

We thus focus on games with undiscounted objective and an indefinite horizon where every realization of the game is guaranteed to terminate after some unspecified number of turns. To manage the computational complexity of solving POSGs in general, we restrict to games with one-sided partial observability where only one player has imperfect information while their opponent is provided with full information about the current situation. We introduce two novel algorithms based on the heuristic search value iteration (HSVI) algorithm that iteratively solve sequences of easier-to-solve approximations of the game using fundamentally different approaches for constructing the sequences: (1) in GoalHorizon, the game approximations are based on a limited number of turns in which players can change their actions, (2) in GoalDiscount, the game approximations are constructed using an increasing discount factor. We provide theoretical qualitative guarantees for algorithms, and we also experimentally demonstrate that these algorithms are able to find near-optimal solutions on pursuit-evasion games and a game modeling privilege escalation problem from computer security.

1. Introduction

Dynamic interactions among competing agents (or players) commonly arise in many real-world situations, including problems appearing in physical security [43,4,1,28,5], computer-network situations [38,47,24], robotics [16,57], or recreational games [49, 36]. In these examples, the interaction among the players is repeated over multiple rounds (or turns of the game), and the players independently choose their action to play, possibly causing changes in the environment in each turn of the game (e.g., deciding whether to attack a particular target in a security game). The length of the interaction is not necessarily limited in advance by a fixed number of turns. Moreover, the players do not always have perfect knowledge about the true state of the environment in many of these examples, and they are able to observe only selected information from the environment (e.g., the pursuers do not necessarily

^{*} Corresponding author.

E-mail address: petr.tomasek@aic.fel.cvut.cz (P. Tomášek).

<https://doi.org/10.1016/j.ijar.2024.109297>

Received 22 February 2024; Received in revised form 5 September 2024; Accepted 8 September 2024

Available online 1 October 2024

0888-613X/© 2024 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

know the exact location of an evader in a pursuit-evasion game). From the perspective of game theory, such situations can be formally modeled as *Partially Observable Stochastic Games* (POSGs) [16,20,24,25].

By solving POSGs, we typically refer to identifying the optimal strategies and/or quantifying the payoffs they achieve. This paper primarily focuses on the latter, aiming to approximate the value of the game. The most common objective when dealing with games that do not have a given limit on the length of the game (the number of turns) is to maximize the *discounted sum of costs/rewards* [16, 25], i.e., the rewards gained in the later stages of the game are discounted by a value decreasing with the length of the game. As we will show later, using the discounted sum of costs/rewards can lead to suboptimal strategies for problems that do not necessarily consider the length of the interaction as the key factor to optimize. As an alternative objective for POSGs, we focus on *undiscounted sum of rewards* (or total sum), where the future rewards are not discounted. However, in order for this objective not to be infinite, we assume that the games have *indefinite horizon* – i.e., every realization of the game is guaranteed to terminate after some turn of the game; however, there is no specified limit on when this termination occurs.

To the best of our knowledge, domain-independent algorithms for solving partially observable stochastic games (or their subclasses) with an indefinite horizon that optimize for the total sum of costs/rewards are not commonly studied. One of the reasons is the problem of *nested beliefs* that is common for all partially observable multi-agent problems. If we are solving a problem with partial observability, agents can base their decisions only on their beliefs about the current state of the problem (usually a probability distribution over possible states of the problem). This can be challenging even in the single-agent case as it was shown that various exact and approximate problems in partially observable Markov decision processes are undecidable [34]. In the non-cooperative multi-agent setting, reasoning about beliefs becomes even more difficult. In such a case, the environment changes as a result of the joint action of all agents. Assume that all players have only partial information about the current state of the problem. If so, it is no longer sufficient for a player to base his decisions only on his own belief. He has to consider the beliefs of other players as well. Since this holds for the beliefs of all players, we end up in a situation where a player has to consider not only his belief over problem states (like in the single-agent case) but also opponents' beliefs, their beliefs over beliefs, and so on. This results in an infinitely deep nesting of the beliefs for which it is not clear when it is safe to stop it and not affect the solution. In our work, we deal with the nested beliefs by restricting ourselves to subclasses of POSGs where only one of the players does not have perfect information about the current state. Thus, the problem of nested beliefs does not appear. This restriction is used in other related works [26,4,1,56] and helps to avoid the problem of nested beliefs as only one player needs to model the uncertainty of the true state. At the same time, this model is not too restrictive and it can be useful in real-world domains (e.g., computer security) where we would like to optimize for the worst-case scenario (i.e., situations where we face a well-informed adversary).

Similarly to other previous work on algorithms for solving games with partial observability [1,17,49,36], we focus on strictly competitive games (or zero-sum games) that again have broad application potential [17,49,36] and are easier to tackle compared to general-sum games. As our main contribution, we introduce two novel algorithms for computing approximately the value of the game for the undiscounted-sum objective with an indefinite horizon. The main idea for the algorithms is to create a sequence of easier-to-solve approximate versions of the game and iteratively solve this sequence, arriving at an approximately optimal solution. We use two fundamentally different approaches for constructing these sequences of games: (1) to approximate the undiscounted-sum game with a game where the players can choose a strategy only in first k turns of the game, and then they have to follow a fixed strategy (termed *GoalHorizon*), or (2) to approximate the undiscounted-sum game with a discounted-sum game with an increasing discount factor (termed *GoalDiscount*).

The paper is organized as follows: We finish this section by describing previous work that we extend, both from the perspective of prior works that focus on problems with undiscounted-sum objective (Section 1.1), and prior works that focus on solving dynamic games with imperfect information (Section 1.2). We then provide the technical background for the concept of one-sided partially observable stochastic games (Section 2) and the state-of-the-art method for their solving (Section 2.1). Then, we show how this method, primarily used for solving infinite-horizon problems with discounted-sum objective, can be modified for solving finite-horizon problems with the total-sum objective (Section 3). We provide a formal definition of POSSPGs (Section 4) and demonstrate that reasoning about exact value of POSSPGs is undecidable. In Section 5 we propose two algorithms for (approximately) solving POSSPGs in terms of (approximately) computing the value of the game for a fixed belief point: we describe the *GoalHorizon* algorithm and extend our previous work on it [54] in Section 5.1, and the *GoalDiscount* algorithm in Section 5.2. Finally, we provide an experimental evaluation (Section 6) of proposed algorithms and compare their results to using the state-of-the-art method for solving one-sided partially observable stochastic games based on both solution quality (Section 6.3) and algorithm scalability (Section 6.4).

1.1. Solving dynamic problems with undiscounted-sum objective

One of the classical real-world problems is the stochastic shortest path (SSP) problem [7,8]. In an SSP, we have an agent that is trying to find an optimal plan in a stochastic environment. To optimize the plan, the agent uses an undiscounted sum of rewards or costs respectively. The optimization of the undiscounted total-sum objective, as we will show later in this section, is crucial for this kind of problems and realizations of solutions obtained assuming a discounted-sum objective can lead to suboptimal results. An example of an agent's goal could be reaching a specific vertex of a graph (or a vertex with some desired dynamic property). Now we present various relevant variants of this problem based on the number of involved controlling agents and the information available to them (visualizations of these variants are shown in Fig. 1).

Let us assume the situation from Fig. 1a. In this setting, the goal of the agent (black stick figure) is to reach the same vertex as the red stick figure. The agent can traverse any of the edges (regardless of the color). Crossing each edge is associated with the cost

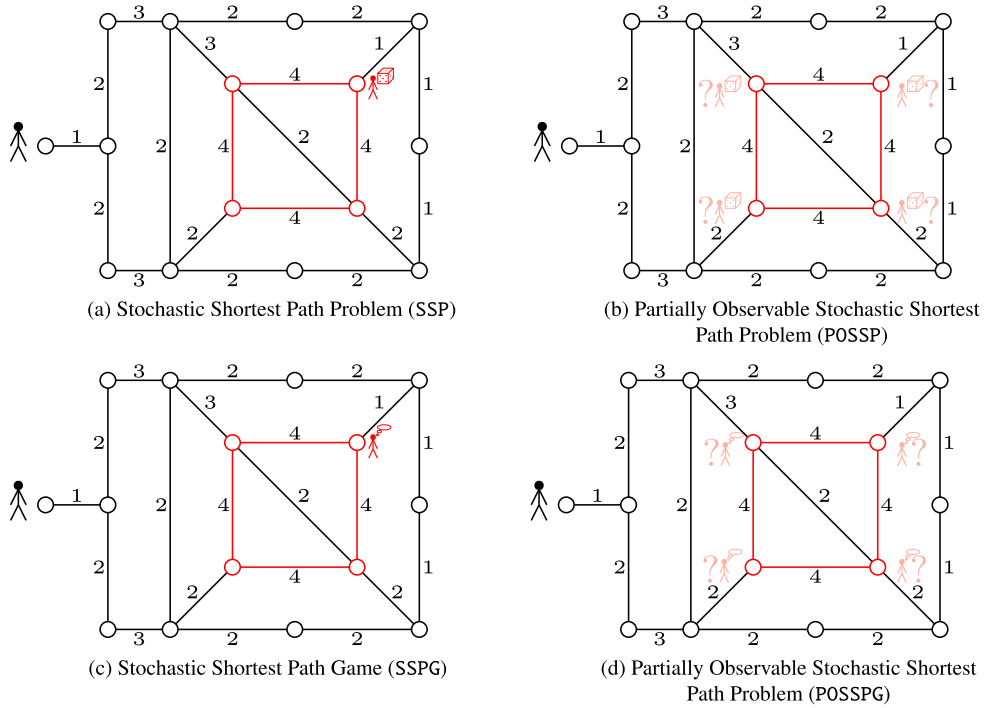


Fig. 1. Variations of Stochastic Shortest Path Problem: Subfigures (a) - (c) represent known variants of the problem, where (a) is a single-agent problem with full information, (b) is a single-agent problem with partial information, and (c) is a two-agent problem (or a game) with full information. We investigate problem (d) – a game with partial information. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

shown near each edge. On the other hand, the red stick figure can visit only red vertices and can traverse only red edges (i.e., at each time step, the red stick figure can only be in one of the red vertices). In each step, both the agent and the red stick figure can either move to a vertex next to their current position or wait (the wait action has for the agent fixed constant cost $c_w > 0$). The agent always has perfect information about the current state of the environment (i.e., the agent knows their own position and the position of the red stick figure in each step). Since we assume a single agent setting, the red stick figure is a part of the environment, and its moves are based on a fixed probability distribution (e.g., uniform) known by the agent. This type of problems can be directly modeled as a Markov decision problem with the objective being an undiscounted sum of rewards (or penalties respectively), and it is well studied [44]. Furthermore, this model naturally arises in many fields, including robotics [33,48], wireless networks [10], or model checking [40].

However, the assumptions of stochastic shortest path problem cannot always be satisfied in real-world conditions that can be more complicated. We focus on two common complications with significant practical implications. In the first one, the agent may have only imperfect information about the current state of the environment. When considering the SSP example, this corresponds to the situation where the position of the red stick figure is no longer known to the agent, and he has only a belief about the current position of the red stick figure (however, the red stick figure is still part of the environment and the probability distribution over its moves is still fixed and known by the agent) — Fig. 1b. Such a class of problems is called partially observable SSP [42,15,23,14] and can be modeled as a partially observable Markov decision process. The second complication is the introduction of an adversary to the problem while retaining perfect information for the agent. The agent still looks for an optimal plan, but at the same time, the agent has to face the adversary that is trying to complicate the planning process. In terms of the SSP example, this will correspond to the situation where the red stick figure is no longer part of the environment and becomes the adversary (also with perfect information) with the goal to avoid ending on the same vertex as the agent — Fig. 1c. The presence of the adversary turns a single-agent problem of stochastic shortest path problem into a two-agent problem called SSP game [41,37,46,11] and such problem is a variant of stochastic games. Furthermore, under the presence of the second agent, the solution is no longer a single optimal plan but usually a randomized strategy.

We focus on an even more complicated scenario that combines partially observable stochastic shortest path problems with stochastic shortest path games, and we term this kind of problems *Partially Observable Stochastic Shortest Path Games* (POSSPGs, Fig. 1d). This means that the agent has to plan in the presence of an adversary while having only partial information about the current state of the environment. In terms of presented examples, everything works like in the SSPG example, with the only difference being that the agent does not know the location of the adversary in each step and has to base his own decision only on a belief on the current position of the adversary (similarly to the POSSP example). In our work, we focus on the two-player, zero-sum setting with only one-sided partial observability. Thus, there is one partially informed player aiming to achieve a goal at minimal cost and one perfectly informed

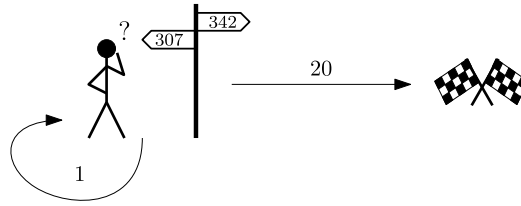


Fig. 2. Example of Stochastic Shortest Path Problem.

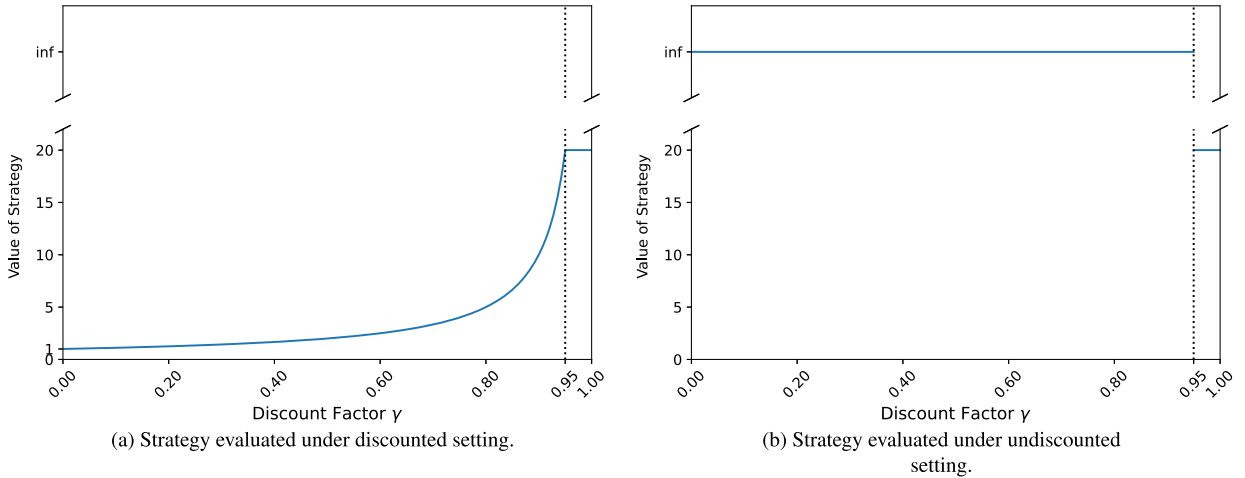


Fig. 3. Value of strategy based on the discount factor γ assumed during the solution process.

adversary intent on increasing the cost paid by the partially informed player. Observe that this makes POSSPGs a variant of partially observable stochastic games (or, in our case, one-sided partially observable stochastic games).

The importance of assuming an undiscounted objective in specific cases can be seen even for the simplest instances of SSP. Let us assume a situation where we have an agent trying to reach the desired destination, and the agent is facing a decision whether to go directly to the destination and pay the traversing cost of $c_m = 20$ or wait at the current position and pay the cost of $c_w = 1$ (see Fig. 2). In this case, we can clearly see that the optimal strategy is to go directly to the destination and pay a cost of 20 since waiting will only increase the paid cost. On the other hand, the situation changes when a discount factor is assumed. If we assume discount factor $\gamma = 0.9$ and attempt to solve the same problem from Fig. 2, we will discover that waiting at the current position becomes a viable move under the discounted setting. Even if the agent waits only once, the cost of such a strategy under a discounted setting will be $c = 1 + \gamma * 20 = 19$, which is less than the cost of 20 paid for going directly to the destination (while under the undiscounted setting, such a strategy has a value of 21). Furthermore, the optimal strategy for this problem while assuming an arbitrary value of the discount factor $\gamma < 0.95$ would always be to wait forever (i.e., going to the final destination is never beneficial for such values of discount factor γ) — see Fig. 3a. Therefore, when deploying these strategies under the undiscounted setting, the paid cost would be *infinity* for all strategies computed with the value of the discount factor $\gamma < 0.95$ — see Fig. 3b.

In the presence of an adversary, the importance of finding an optimal strategy under the undiscounted setting becomes even more crucial. The reason is that the adversary can actively exploit and punish the sub-optimal decisions of the agent and make the strategy highly non-optimal. To avoid such sub-optimal decisions caused by the usage of a discount factor, we have to either solve the problem assuming an undiscounted objective or find such a value of a discount factor γ^* that will be sufficiently large (close to 1) to ensure that the agent will not make sub-optimal decisions. In this work, we present an algorithm for both “*undiscounted objective*” approach (Section 5.1) and “*find the right discount factor*” approach (Section 5.2).

1.2. Other related work

Besides the works mentioned in the previous section that tackle simpler variants of problems with an indefinite horizon and the undiscounted-sum objective, there are other relevant works. For the discounted-sum objective, there exist algorithms for (approximately) solving partially observable problems efficiently [25,53]. However, these do not directly generalize to the undiscounted-sum objective and we extend them in our work.

On the other hand, there are several attempts to solve partially observable problems with variants of history-dependent approaches that are compatible with an undiscounted-sum objective. This includes the usage of history-dependent strategies [20,32]¹ or defining value functions over the probabilities of action-observation histories of the players [59,58]. Even though these approaches can cope with partially observable scenarios under undiscounted setting, the scalability and applicability of such approaches are highly limited as they scale either doubly exponentially (in the case of history-dependent strategies) or exponentially (in the case of history-dependent value functions) with the horizon of the game (i.e., the number of turns in the game). This makes these approaches suitable mostly for problems with small finite horizon. For this particular computational setting, there are also approaches that solve two-player zero-sum POSGs, assuming an undiscounted-sum objective even with both players having only partial information about the current state of the game. The algorithms include HSVI [13], CFR [31,3], or linear programming [2,18]. We focus on solving problems with an indefinite horizon that are expected to have a very long horizon, thus we do not want to consider approaches based on history-dependent strategies/value functions or applicable specifically only to problems with limited finite horizon.

Many game-based problems (e.g., StarCraft II, DotA 2, etc.) share the property of having an indefinite horizon with one-sided partially observable stochastic shortest-path games. In recent years, it was shown that approaches based on neural networks could be used for finding strategies to use in such problems that are comparable to (or even outperform) human experts [6,55]. Despite the successful comparison to human experts, these approaches do not guarantee the quality of the solutions and are highly domain-dependent. On the other hand, our proposed algorithms do have theoretical guarantees to provide ϵ -optimal solutions while being fully domain-independent. Furthermore, our algorithms provide guarantees on the solution quality at any step of the computation (i.e., if the computation is early-terminated, the found solution is guaranteed to be worse than the optimal one at most by some fixed threshold).

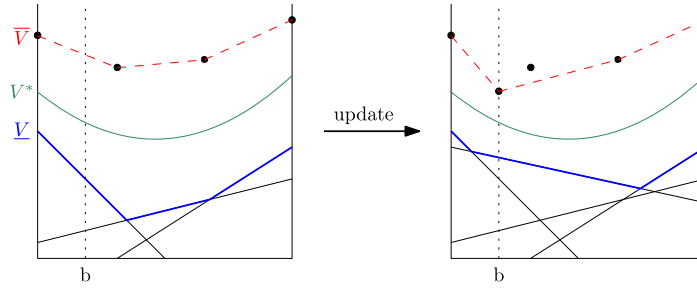
One of the closest related work [23] introduces an algorithm based on HSVI for solving single-agent problems (namely goal partially observable Markov decision processes — Goal-POMDPs) with a total-sum objective. We extended this work in the first conference version of this paper [54] where a simpler variant of the GoalHorizon algorithm was introduced. This submission significantly extends the conference version by: (1) extending the GoalHorizon algorithm to more general games beyond strictly negative costs assumed in [54], (2) we provide a completely new algorithm, GoalDiscount, and (3) we provide a completely new and extensive experimental evaluation of the algorithms.

2. One-sided partially observable stochastic games (OS-POSGs)

One-sided partially observable stochastic game (OS-POSG) [22,25] is a special case of an imperfect-information two-player zero-sum game with infinite horizon and perfect recall. The one-sided partial observability implies that the imperfect information setting is not symmetric. This means that only one player (player 1, P1) has imperfect information while the other player (player 2, P2) is perfectly informed about the current state of the game. The objective of the players is to optimize the *discounted sum* of their rewards obtained during the course of the game. While the discounted-sum objective of OS-POSGs is fundamentally different from the total-sum objective which is in the center of our attention, we decided to introduce OS-POSGs first since our algorithmic approach (as well as notation) is inspired by the OS-POSG literature.

Formally, we define an OS-POSG G as a tuple $G = \langle S, A_1, A_2, O, T, R \rangle$. Initially, the game starts in state $s^{(0)} \in S$, where S is the finite set of states. The state $s^{(0)}$ itself is drawn from the probability distribution over states $b_{\text{init}} \in \Delta(S)$ called the *initial belief*. Then, the game evolves in an infinite number of stages. In each stage t ($t \geq 1$), the game is in one of the states $s \in S$, and players *simultaneously* and *independently* pick their respective actions $a_1 \in A_1$ and $a_2 \in A_2$. The choice of actions determines the new underlying state of the game $s^{(t)}$ and an observation $o^{(t)} \in O$ received by P1. The set O contains all possible observations for P1 that provide P1 with partial information about the new state. Note that both the new state $s^{(t)}$ and the observation $o^{(t)}$ of P1 are a result of the joint action $(a_1^{(t)}, a_2^{(t)})$ and actions of perfectly informed P2 directly affect observations of P1 and thus his belief as well. The transition function T ensures that $s^{(t)}$ and $o^{(t)}$ are generated with probability $T(o^{(t)}, s^{(t)} | s^{(t-1)}, a_1^{(t)}, a_2^{(t)})$. For each transition, P1 receives a stage reward $R(s^{(t-1)}, a_1^{(t)}, a_2^{(t)})$. Due to the one-sided nature of OS-POSGs, P2 can decide about his action $a_2^{(t)}$ based on the entire history of the game, $s^{(0)}(a_1^{(t')}, a_2^{(t')}, o^{(t')})_{t'=1}^{t-1}$ (i.e., his only uncertainty is the action $a_1^{(t)}$ P1 decided to take in the current stage). Thus, his strategy σ_2 conditions on the current state of the game and the entire (potentially infinite) history of the game $(A_1 A_2 O S)^*$ (i.e., $\sigma_2 : S(A_1 A_2 O S)^* \rightarrow \Delta(A_2)$). On the other hand, P1 can only consider his own past actions and observations $(a_1^{(t')}, o^{(t')})_{t'=1}^{t-1}$ when making his decision (i.e., he is uncertain not only about the action $a_2^{(t)}$ but also about all actions previously played by P2 and the current underlying state of the game). Therefore, his strategy σ_1 is based only on the (potentially infinite) history pairs his own actions and observations received from the environment, $(A_1 O)^*$ (i.e., $\sigma_1 : (A_1 O)^* \rightarrow \Delta(A_1)$). Total expected utility for P1 is defined as the expected infinite discounted sum of stage rewards, that is $\mathbb{E}[\sum_{t=1}^{\infty} \gamma^{t-1} R(s^{(t-1)}, a_1^{(t)}, a_2^{(t)})]$. The discount factor γ is assumed to be positive and strictly less than 1. The goal of P1 is to maximize his total expected utility, while P2 aims at minimizing the total expected utility of P1 (note that due to the zero-sum nature of OS-POSGs, this is equivalent to maximizing own utility).

¹ Both [20,32] are about general-sum n-player POSGs with a final focus on their cooperative variant – Dec-POMDPs).

Fig. 4. HSVI local update in the belief b , adapted from [50].

2.1. Solving OS-POSGs

One of the options to solve OS-POSGs (i.e., find the value of a game for a fixed initial belief point) is to utilize the concept of *value function*. A value function $V : \Delta(S) \rightarrow \mathbb{R}$ assigns a payoff (or payoff estimate) to each belief state in the game. When solving an OS-POSG, we want to find the (approximately) optimal value function V^* of the game G that represents the expected cumulative reward that player 1 can attain by following the optimal strategy for each of the beliefs in the game. This can be expressed as:

$$V^*(b) = \max_{\sigma_1} \min_{\sigma_2} \mathbb{E}_{b, \sigma_1, \sigma_2} \left[\sum_{t=1}^{\infty} \gamma^{t-1} R(s^{(t-1)}, a_1^{(t)}, a_2^{(t)}) \right].$$

Function V^* can be characterized by a Bellman-style fixed point equation [22] (which also defines the backup operator H^γ):

$$V^*(b) = [H^\gamma V^*](b) = \max_{\pi_1} \min_{\pi_2} \left[\mathbb{E}_{b, \pi_1, \pi_2} [R(s, a_1, a_2)] + \gamma \sum_{a_1, o} \mathbb{P}_{b, \pi_1, \pi_2} [a_1, o] \cdot V^*(\tau(b, a_1, \pi_2, o)) \right], \quad (1)$$

where $\pi_1 : \Delta(A_1)$ and $\pi_2 : S \rightarrow \Delta(A_2)$ are stage strategies player 1 and 2 use to choose their actions a_1 and a_2 in the current stage of the game and $\tau(b, a_1, \pi_2, o) = \mathbb{P}_{b, \pi_2} [s' | a_1, o]$ is the Bayesian belief update. Note that the backup operator H^γ preserves the ordering of value functions as the following lemma shows:

Lemma 1. *Let us have value functions V and V' such that $V \leq V'$. Then, for every belief $b \in \Delta(S)$, we have $H^\gamma V(b) \leq H^\gamma V'(b)$.*

Proof. Let us have a discount factor $\gamma \in [0, 1]$ and two Nash equilibrium profiles (σ_1, σ_2) and (σ'_1, σ'_2) such that $H^\gamma V(b) = u_b^\gamma(\sigma_1, \sigma_2)$ and $H^\gamma V'(b) = u_b^\gamma(\sigma'_1, \sigma'_2)$, where u_b^γ and $u_b^{\prime\gamma}$ are one-step evaluations of a given strategy profile with respect to the value functions V and V' respectively. Then

$$H^\gamma V(b) = u_b^\gamma(\sigma_1, \sigma_2) \leq u_b^\gamma(\sigma'_1, \sigma_2) \leq u_b^{\prime\gamma}(\sigma'_1, \sigma_2) \leq u_b^{\prime\gamma}(\sigma'_1, \sigma'_2) = H^\gamma V'(b). \quad \square$$

Similarly to the single-player case (POMDP), V^* can be approximated using more scalable algorithms. For OS-POSGs, there exists a modification of the heuristic search value iteration algorithm (HSVI) [22] that extends HSVI for POMDPs [50,51] to the two-player zero-sum games.

This algorithm (Algorithm 1) uses a self-play between P1 and P2 to identify beliefs relevant for the solution of the game and aims at achieving a close approximation of V^* primarily in these beliefs. Piecewise linear and convex lower $\underline{V}$ and upper $\overline{V}$ bounds on V^* are maintained and gradually refined over time (see Fig. 4). Since V^* is δ -Lipschitz continuous for $\delta = (\max R(\cdot) - \min R(\cdot))/2(1 - \gamma)$, δ -Lipschitz bounds $\underline{V}$ and $\overline{V}$ are considered.

The goal of the algorithm is to ensure that a desired precision $\overline{V}(b_{\text{init}}) - \underline{V}(b_{\text{init}}) \leq \varepsilon$ is eventually reached (line 1). To this end, the algorithm performs a sequence of trials (represented by the Explore procedure) that aim at achieving that beliefs $b^{(k)}$ reached at k -th level of recursion are approximated within $\rho(k)$ precision. Sequence ρ is defined as

$$\rho(1) = \varepsilon \quad \rho(k+1) = [\rho(k) - 2\delta D]/\gamma, \quad (2)$$

where δ is the Lipschitz constant of V^* and $D > 0$ is a parameter such that ρ is monotonically increasing and unbounded.

In each stage of a trial, players choose their stage strategies $\pi_1^{(k)}$ and $\pi_2^{(k)}$ (lines 5–6). The maximizing player 1 chooses his strategy based on solving the game corresponding to $[H^\gamma \overline{V}](b)$, while the minimizing player 2 chooses his strategy based on solving $[H^\gamma \underline{V}](b)$ (i.e., each player obtains his own strategy based upon reasoning about an *optimistic* variant of the stage game from his perspective). For piecewise linear and convex bounds $\underline{V}$ and $\overline{V}$ it is possible to obtain these strategies by means of linear programming as shown in [25, Lemma 7.1, Section 8.1]. Since von Neumann's minmax theorem [39] applies to the stage game introduced in Equation (1) as shown in [25, Theorem 1], these linear programs are dual formulations.

Each trial constructs a sequence of beliefs (line 7) such that (1) $b^{(k+1)}$ is reachable from $b^{(k)}$ when employing stage strategies $\pi_1^{(k)}$ and $\pi_2^{(k)}$, (2) $b^{(k+1)}$ has the highest excess gap

Algorithm 1: HSVI for discounted OS-POSGs, adapted from [22].

Input : discounted OS-POSG G , discount factor γ ,
target precision $\epsilon > 0$, initial belief point b_{init}
Output: upper bound $\overline{V}(b_{\text{init}})$, lower bound $\underline{V}(b_{\text{init}})$

```

1 while  $\overline{V}(b_{\text{init}}) - \underline{V}(b_{\text{init}}) > \epsilon$  do
2    $\text{Explore}(b_{\text{init}}, 1)$ 
3 procedure  $\text{Explore}(b^{(t)}, t)$ 
4   if  $\text{excess}_k(b^{(t)}) \leq 0$  then return
5    $\pi_1^{(t)} \leftarrow$  optimal strategy of P1 in  $[H^\gamma \overline{V}](b^{(t)})$ 
6    $\pi_2^{(t)} \leftarrow$  optimal strategy of P2 in  $[H^\gamma \underline{V}](b^{(t)})$ 
7    $b^{(t+1)} \leftarrow \arg \max_{a_1, o} \mathbb{P}_{b^{(t)}, \pi_1^{(t)}, \pi_2^{(t)}}[a_1, o] \cdot \text{excess}_{t+1}(\tau(b^{(t)}, a_1, \pi_2^{(t)}, o))$ 
8    $\text{Explore}(b^{(t+1)}, t+1)$ 
9   Perform point-based update of  $\underline{V}$  and  $\overline{V}$  in  $b^{(t)}$ 

```

$$\text{excess}_{k+1}(\tau(b^{(k)}, a_1, \pi_2^{(k)}, o)) = \overline{V}(\tau(b^{(k)}, a_1, \pi_2^{(k)}, o)) - \underline{V}(\tau(b^{(k)}, a_1, \pi_2^{(k)}, o)) - \rho(k) \quad (3)$$

among the reachable beliefs weighted by the probability $\mathbb{P}_{b^{(k)}, \pi_1^{(k)}, \pi_2^{(k)}}[a_1, o]$ of reaching that belief, where

$$\mathbb{P}_{b^{(k)}, \pi_1^{(k)}, \pi_2^{(k)}}[a_1, o] = \sum_{s \in S} b^{(k)}(s) \cdot \pi_1^{(k)}(a_1) \cdot \sum_{s' \in S} \sum_{a_2 \in A_2} \pi_2^{(k)}(a_2 | s) \cdot T(o, s' | s, a_1, a_2).$$

Upon reaching a belief $b^{(k)}$ where sufficient accuracy is achieved (i.e., it holds $\text{excess}_k(b^{(k)}) \leq 0$), the trial terminates and a sequence of point-based updates in beliefs $b^{(1)}, \dots, b^{(k)}$ is performed on line 9 to refine the bounds $\underline{V}$ and $\overline{V}$ (the updates ensure that $\underline{V}(b) := [H^\gamma \underline{V}](b)$ and $\overline{V}(b) := [H^\gamma \overline{V}](b)$ for selected beliefs b).

The lower bound $\underline{V}$ (blue line in Fig. 4) on V^* is represented by commonly used vector representation as a finite set Γ of linear functions $\alpha_i : \Delta(S) \rightarrow \mathbb{R}$, where the value at a belief state b is the maximum projection of b onto the set Γ . The value of $\underline{V}(b)$ is the point-wise maximum over this set:

$$\underline{V}(b) = \max_{\alpha \in \Gamma} (\alpha \cdot b).$$

These linear functions are termed *alpha vectors* and represent expected outcomes of strategies found by the algorithm (black lines in the bottom part of Fig. 4).

Since the HSVI algorithm relies on the local updates (i.e., updating bounds for specific belief points) and improving bounds in the neighborhood of the local update (as shown in Fig. 4, a local update results not only in improving bounds in a particular belief point but also in its neighborhood), the upper bound cannot be represented by a vector set. Instead, the upper bound $\overline{V}$ on V^* is expressed using a set $\Upsilon = \{(b^{(i)}, y^{(i)}) \mid 1 \leq i \leq |\Upsilon|\}$ of belief/value points $(b^{(i)}, y^{(i)})$ (dots in the upper part of Fig. 4). The lower convex hull of this set of points is then used to obtain the value of $\overline{V}(b)$ (red dashed line in Fig. 4):

$$\overline{V}(b) = \min_{\lambda \in \mathbb{R}_{\geq 0}^{|\Upsilon|}} \left\{ \sum_{1 \leq i \leq |\Upsilon|} \lambda_i y^{(i)} \mid \mathbf{1}^T \lambda = 1, \sum_{1 \leq i \leq |\Upsilon|} \lambda_i b^{(i)} = b \right\}.$$

Finally, HSVI local updates are performed by adding a new vector (for the lower bound) or a point (for the upper bound) to the current sets Γ and Υ , respectively.

Convergence properties. When discussing our proposed modified versions of the HSVI algorithm (FH-HSVI – Section 3, GoalHorizon – Section 5.1, and GoalDiscount – Section 5.2), we will refer to the following key properties of the HSVI algorithm for discounted OS-POSG that suffice to ensure the convergence:

- (1) *Each trial terminates in $b^{(k)}$ with $\text{excess}_k(b^{(k)}) \leq 0$ for some k .* The payoff in the game is bounded, and so is the gap between $\underline{V}$ and $\overline{V}$. As ρ is unbounded, $\rho(k)$ will eventually exceed $\overline{V}(b^{(k)}) - \underline{V}(b^{(k)})$. Observe that this also means that all beliefs reachable from $b^{(k-1)}$ have a negative excess gap on line 4.
- (2) *Point-based update in $b^{(k-1)}$ ensures that all beliefs within hypersphere with some radius D centered in $b^{(k-1)}$ have negative excess gap.* [22, Lemma 4]
- (3) *Eventually, no beliefs with $\text{excess}_k(b^{(k)}) > 0$ remain (unless the algorithm converges earlier).* By (1) and (2), each trial identifies (at least) one belief $b^{(k-1)}$ with a positive excess gap and makes all beliefs within D -neighborhood of $b^{(k-1)}$ have a negative excess gap. By a standard packing argument, the hyperspheres with radius D describing beliefs with a negative excess gap eventually cover the entire belief space. Hence, no beliefs with a positive excess gap remain. [22, Theorem 3]

Observe that the absence of beliefs with a negative excess gap means that it holds $\text{excess}_1(b_{\text{init}}) < 0$ and hence $\overline{V}(b_{\text{init}}) - \underline{V}(b_{\text{init}}) \leq \rho(1) = \epsilon$.

Final improvements. Our implementation of Algorithm 1 uses commonly used approach to prune out dominated alpha vectors and points after each iteration [50,25] (i.e., execution of line 2) to reduce their numbers.

Possible modifications. We can improve exploration in deeper levels of exploring recursion by performing point-based updates to the individual bounds not only after returning from the recursion but also before entering it. In terms of modifications to Algorithm 1, this means performing tasks from line 9 also before executing line 8. On the other hand, this modification can potentially result in using a higher number of alpha vectors or points for representing corresponding bounds and thus increases the memory consumption of the algorithm.

3. Solving finite-horizon OS-POSGs

While the existing results for OS-POSGs consider only infinite-horizon problems with discounted-sum objective, a straightforward reduction can be used to show that most of the results can be transferred to the setting with finite-horizon $\mathcal{H}$ and total sum of rewards $\mathbb{E}[\sum_{t=1}^{\mathcal{H}} R(s^{(t-1)}, a_1^{(t)}, a_2^{(t)})]$ as the optimization objective. Consider a OS-POSG game structure $G = \langle S, A_1, A_2, O, \bar{o}, T, R, b_{\text{init}} \rangle$ with finite-horizon $\mathcal{H}$ and total-sum objective where $\bar{o} \in O$ is an arbitrary observation indicating the game finished. Let us define a strategically equivalent infinite-horizon discounted-sum game $G^\gamma = \langle S^\gamma, A_1, A_2, O, T^\gamma, R^\gamma, b_{\text{init}} \rangle$ by expanding the state space of G :

- $S^\gamma = \{((s, h)) \mid s \in S, 0 \leq h \leq \mathcal{H}\}$ (i.e., (s, h) corresponds to a state s in G when h stages remain to be played),
- $T^\gamma(o, (s', h-1) \mid (s, h), a_1, a_2) = T(o, s' \mid s, a_1, a_2)$ for $h \geq 1$ (i.e., after each transition there is one less stage to be played in the remainder of the game),
- $T^\gamma(\bar{o}, (s, 0) \mid (s, 0), a_1, a_2) = 1$ for a fixed arbitrary observation $\bar{o} \in O$ (i.e., when 0 stages remain in the game, $h = 0$),
- $R^\gamma((s, h), a_1, a_2) = R(s, a_1, a_2)/\gamma^{H-h}$ for $h \geq 1$ and $R^\gamma((s, 0), a_1, a_2) = 0$.

Observe that the definition of rewards R^γ ensures that no reward is allocated after $\mathcal{H}$ stages of the game pass (since a state $(s, 0)$ is reached). Moreover the discounting terms in the discounted-sum objective $\mathbb{E}[\sum_{t=1}^{\mathcal{H}} \gamma^{t-1} R^\gamma(s^{(t-1)}, a_1^{(t)}, a_2^{(t)})]$ cancel out since

$$\begin{aligned} \mathbb{E}[\sum_{t=1}^{\mathcal{H}} \gamma^{t-1} R^\gamma(s^{(t-1)}, a_1^{(t)}, a_2^{(t)})] &= \mathbb{E}[\sum_{t=1}^{\mathcal{H}} \gamma^{t-1} R(s^{(t-1)}, a_1^{(t)}, a_2^{(t)})/\gamma^{H-h}] = \\ &= \mathbb{E}[\sum_{t=1}^{\mathcal{H}} \gamma^{t-1} R(s^{(t-1)}, a_1^{(t)}, a_2^{(t)})/\gamma^{t-1}] = \mathbb{E}[\sum_{t=1}^{\mathcal{H}} R(s^{(t-1)}, a_1^{(t)}, a_2^{(t)})] \end{aligned}$$

and hence the objective of G^γ coincides with the total-sum objective of G .

Applying the HSVI algorithm to the OS-POSG G^γ is, however, impractical as the value of state (s, h) does not correspond to the total-sum payoff Player 1 achieves in the remaining h stages. Due to the multiplication of the rewards by $1/\gamma^{H-h}$, the value of (s, h) is also multiplied by $1/\gamma^{H-h}$. We now show that we can avoid this issue by solving G^γ for $\gamma = 1$ (referred to as $G^{\gamma=1}$) using a modification of the HSVI algorithm that has a different sequence ρ (compared to one presented in Equation (2)) — we term this algorithm FH-HSVI (Algorithm 2).

Algorithm 2: HSVI for finite horizon OS-POSGs (FH-HSVI).

Input : horizon k of game G , target precision $\varepsilon > 0$, initial belief point b_{init}
Output: upper bound $\bar{V}^k(b_{\text{init}})$, lower bound $\underline{V}^k(b_{\text{init}})$

```

1  $\underline{V}^k \leftarrow \text{val}^{\alpha_{\text{inf}}}$ 
2  $\bar{V}^k \leftarrow \text{val}^{\alpha_{\text{sup}}}$ 
3 while  $\bar{V}^k(b_{\text{init}}) - \underline{V}^k(b_{\text{init}}) > \varepsilon$  do
4    $\text{Explore}(b_{\text{init}}, k-1)$ 
5 procedure  $\text{Explore}(b^{(k)}, k)$ 
6   if  $\text{excess}_k(b^{(k)}) \leq 0$  then return
7    $\pi_1^{(k)} \leftarrow$  optimal strategy of P1 in  $[H\bar{V}^k](b^{(k)})$ 
8    $\pi_2^{(k)} \leftarrow$  optimal strategy of P2 in  $[H\underline{V}^k](b^{(k)})$ 
9    $b^{(k-1)} \leftarrow \arg \max_{a_1, o} \mathbb{P}_{b^{(k)}, \pi_1^{(k)}, \pi_2^{(k)}}[a_1, o] \cdot \text{excess}_{k-1}(\tau(b^{(k)}, a_1, \pi_2^{(k)}, o))$ 
10   $\text{Explore}(b^{(k-1)}, k-1)$ 
11  Perform point-based update of  $\underline{V}^k$  and  $\bar{V}^k$  in  $b^{(k)}$ 
```

The FH-HSVI algorithm works along the same lines as the HSVI algorithm with few modifications. Just like in the discounted setting, we can characterize the mapping from beliefs $b \in \Delta(S)$ of P1 to the undiscounted payoff $V^*(b)$ by setting the discount factor $\gamma = 1$ in the Bellman-style fixed point equation from Equation (1) resulting in:

$$V^*(b) = [HV^*](b) = \max_{\pi_1} \min_{\pi_2} \left[\mathbb{E}_{b, \pi_1, \pi_2} [R(s, a_1, a_2)] + \sum_{a_1, o} \mathbb{P}_{b, \pi_1, \pi_2} [a_1, o] \cdot V^*(\tau(b, a_1, \pi_2, o)) \right]. \quad (4)$$

The dynamics of $G^{\gamma=1}$ ensure that a support of any belief cannot contain states with different depth h . This property allows us to partition the optimal value function V^* of $G^{\gamma=1}$ based on the number of remaining stages in the game

$$V_h^*(b) = V^*(b|_h) \quad \text{for } b \in \Delta(S), b|_h \in \Delta(S^\gamma) : b|_h(s, h) = b(s). \quad (5)$$

This means that we can rewrite Equation (4) as

$$V_h^*(b) = [HV_{h-1}^*](b) = \max_{\pi_1} \min_{\pi_2} \left[\mathbb{E}_{b, \pi_1, \pi_2} [R(s, a_1, a_2)] + \sum_{a_1, o} \mathbb{P}_{b, \pi_1, \pi_2} [a_1, o] \cdot V_{h-1}^*(\tau(b, a_1, \pi_2, o)) \right] \quad (6)$$

and obtain a Bellman equation for $G^{\gamma=1}$ in a traditional form for finite-horizon problems. Note that after partitioning the value function V^* , there is no longer a need to consider individual functions V_h^* as defined over (s, h) ; instead, we can revert to the traditional definition of beliefs in OS-POSGs as distributions $b \in \Delta(S)$ over states S of the original problem.

Since we are no longer dealing with infinite horizon, we can use different initialization for $\underline{V}^k$ and $\overline{V}^k$, respectively. In our case, we initialize $\underline{V}^k$ as value of the uniform strategy of imperfectly informed player (i.e., player 1, P1) in this horizon- k game $\text{val}^{\sigma_{\text{unif}}^k}$ (Algorithm 2, line 1). This is indeed a valid lower bound on the value of the game, as the imperfectly informed player can always resort to the uniform strategy and achieve the associated value. For the upper bound, we initialize $\overline{V}^k$ (Algorithm 2, line 2) to the value of the imperfectly informed player (i.e., player 1, P1) assuming that we are solving perfect information variant of the horizon- k game (i.e., the player has no longer only partial information). Throughout this work, we will slightly abuse the notation and refer to the strategy played in the perfect information variant of the game as σ_{perf} (or σ_{perf}^k for horizon- k games) and the value of such strategy as $\text{val}^{\sigma_{\text{perf}}}$ (or $\text{val}^{\sigma_{\text{perf}}^k}$ respectively). Note that the strategy σ_{perf} is only applicable in the perfect information variant of the game. Initializing $\overline{V}^k$ to $\text{val}^{\sigma_{\text{perf}}^k}$ then indeed imposes a valid upper bound on the value of the horizon- k game as the imperfectly informed player cannot perform better as with the perfect information (i.e., additional information can only improve his utility).

Observe that it clearly holds that $V_0^*(b) = 0$ for every $b \in \Delta(S)$ as no rewards are allocated after H stages when the game reaches a state s_0 . We can, therefore, set the initial lower and upper bounds $\underline{V}_0$ and $\overline{V}_0$ on V_0^* in the HSVI algorithm to 0 as well. This means that the Explore procedure eventually reaches a belief $b|_0$ where the gap $\overline{V}_0(b) - \underline{V}_0(b) = 0$ (unless it terminated before reaching H -th stage). Unlike in the discounted setting, the sequence of desired gaps ρ hence need not be strictly increasing. Instead, we let $\rho(t) = \varepsilon - (t-1)\varepsilon/H$ (i.e., $\rho(H+1) = 0$ after H -th stage is considered by Explore procedure which is sufficient for the termination of the trial as the gap $\overline{V}_0(b) - \underline{V}_0(b) = 0$). Similarly to the discounted case (Algorithm 1), the proposed Algorithm 2 explores the belief space based on the maximum excess gap

$$\text{excess}_{k-1}(\tau(b^{(k)}, a_1, \pi_2^{(k)}, o)) = \overline{V}_{k-1}(\tau(b^{(k)}, a_1, \pi_2^{(k)}, o)) - \underline{V}_{k-1}(\tau(b^{(k)}, a_1, \pi_2^{(k)}, o)) - \rho(k). \quad (7)$$

Importantly, since the payoff of any strategy in H' -horizon game can be bounded by $[H' \min R(\cdot), H' \max R(\cdot)]$, a value function $V_{H'}^*$ is $\delta_{H'}$ -Lipschitz continuous for $\delta_{H'} = H'[\max R(\cdot) - \min R(\cdot)]/2$ (cf. [22, Lemma 4]). Similarly to the discounted case, this allows us to consider only $\delta_{H'}$ -Lipschitz continuous lower and upper bounds $\underline{V}_{H'}$ and $\overline{V}_{H'}$ on $V_{H'}^*$. As a consequence, we can verify that the convergence properties discussed in Section 2.1 hold for FH-HSVI:

- (1) Each trial terminates for $t \leq H+1$. After H -th recursion step, the game is in one of the states with zero remaining time, and $\overline{V}_0(b) - \underline{V}_0(b) = 0 \leq \rho(H+1)$.
- (2) Point-based update in $b^{(t-1)}$ ensures that all beliefs within hypersphere with radius $\varepsilon/2H\delta_{H'}$ centered in $b^{(t-1)}$ have negative excess gap (for theoretical details see Proposition 2 and its proof in Appendix A.1).
- (3) By the same packing argument as in [22, Theorem 3], we can see that the hyperspheres induced by the beliefs with the negative excess gap eventually cover the entire belief space (unless we achieve convergence before that).

Theorem 1. *For every $\varepsilon > 0$ the FH-HSVI algorithm for solving finite-horizon OS-POSGs terminates. When it does so, it approximates the value of the game within ε interval.*

For detailed proof, see Appendix A.1.

Final improvements. Just like in the case of traditional discounted HSVI algorithm (Algorithm 1), our implementation of Algorithm 2 prunes out dominated alpha vectors and points after each iteration (i.e., execution of line 4) to reduce their numbers.

Non-zero termination utilities. There might be situations where reaching the horizon produces a one-time non-zero payoff depending on the belief $b^{(k)}$ at that time. This means that such a game is not a finite-horizon OS-POSG where no rewards are assigned after $H = k$ stages. However, we argue that the same algorithm can be applied upon setting $V_0^*(b) = \underline{V}_0(b) = \overline{V}_0(b) = \text{val}^{\sigma_{\text{unif}}^k}(b)$ since

- In the worst case, the Explore procedure reaches a belief $b|_0$ where $\overline{V}_0(b) - \underline{V}_0(b) = 0$,
- The payoff of any strategy in H' -horizon game is bounded by $[\min_b V_0^*(b) + H' \min R(\cdot), \max_b V_0^*(b) + H' \max R(\cdot)]$, and hence $V_{H'}^*$ is Lipschitz continuous.

4. Partially observable stochastic shortest path games (POSSPGs)

Partially observable stochastic shortest path games (POSSPGs) work similarly to stochastic shortest path games (SSPGs). However, instead of extending the stochastic shortest path problems, they represent an extension to the partially observable variant of it. This turns a partially observable stochastic shortest path problem into a zero-sum game with the presence of a well-informed adversary. Similarly to OS-POSGs (Section 2), we assume that one of the players (player 1, P1) has imperfect information and the second player (player 2, P2) is perfectly informed and represents our well-informed adversary. The goal of one of the players is to reach one of the states in the set S_G of goal states while minimizing their cost. On the other hand, the adversary (i.e., the other player) aims to render the cost paid by the minimizing player as high as possible. For now, let us assume that player 2 is the one minimizing the costs. In order to stay consistent with the OS-POSG formalism, we model the costs of player 2 as rewards R of player 1 (recall that in OS-POSGs player 1 maximizes his reward — thus maximizes the cost paid by player 2). We can thus formalize a POSSPG as a tuple $G = \langle S, A_1, A_2, O, T, R \rangle$. Unlike in OS-POSGs, the goal of player 1 in POSSPGs is to maximize the total sum of received rewards (instead of the discounted sum):

$$U_b^\infty(\sigma_1, \sigma_2) = \mathbb{E}_{b, \sigma_1, \sigma_2} \left[\sum_{t=1}^{\infty} R(s^{(t-1)}, a_1^{(t)}, a_2^{(t)}) \right], \quad (8)$$

where b is the current belief of the game, and (σ_1, σ_2) are the strategies used by the players. Note that this formalism is also applicable to the case where player 1 is the one minimizing the paid cost. In that case, the rewards of player 1 are negative values of his costs.

When assuming a well-informed adversary, we aim for the worst-case and assume that player 2 is perfectly informed (just like in OS-POSGs, his strategy σ_2 conditions on the entire history of the game). On the other hand, player 1 has limited observability of the game and can base his decisions (strategy σ_1) only on the history of his own actions and observations received from the environment during the course of the game (again identical to OS-POSGs). Similarly to OS-POSGs, we use $V^*(b)$ to denote the value of a POSSPG with initial belief b (i.e., the payoff the players are guaranteed to achieve under optimal strategies; this payoff is guaranteed to be finite due to the assumptions stated below). We also use the notation $\text{val}^{\sigma_1}(b)$ to denote the payoff strategy σ_1 of player 1 achieves in a game with initial belief b (i.e. $\text{val}^{\sigma_1}(b) = \min_{\sigma_2} U_b^\infty(\sigma_1, \sigma_2)$).

Since the undiscounted total-sum payoff can be potentially infinite, we impose the following assumptions in POSSPGs:

- *The goal is to reach $s \in S_G$.* The game ends upon reaching a state in S_G , and neither player incurs no further costs or rewards (i.e., $R(s, \cdot) = 0$ for all $s \in S_G$). In our model, the game does not technically terminate, but instead, every $s \in S_G$ is an absorbing state. We assume that observation $o_{\text{reach}} \in O$ informs player 1 about reaching the goal and $T(o_{\text{reach}}, s | s, \cdot) = 1$ for every $s \in S_G$.
- *No zero cycles are present.* Compared to previous work on POSSPGs [54] we do not require that all other costs are either positive or negative. Instead, we allow costs to be zero as well, i.e., $R(\cdot) \leq 0$ (in case player 1 is supposed to reach the goal states S_G) or $R(\cdot) \geq 0$ (if player 2 minimizes their costs to reach the goal). We do, however, require that there are no zero cycles that would allow the player to stay in the game indefinitely without paying any cost. Specifically, for the case $R(\cdot) \leq 0$, we require that for every state $s_0 \notin S_G$ and every sequence of actions $\{a_1^{(i)}\}_{i=1}^{|S|}$ of player 1 there exists a sequence of actions $\{a_2^{(i)}\}_{i=1}^{|S|}$ and a sequence of states $\{s^{(i)}\}_{i=1}^{|S|}$ ($S^{(i)} \notin S_G$, $T(\cdot, s^{(i)} | s^{(i-1)}, a_1^{(i)}, a_2^{(i)}) > 0$) such that $R(s^{(i-1)}, a_1^{(i)}, a_2^{(i)}) < 0$ for at least some $1 \leq i < |S|$. Note that, unless a goal state $s \in S_G$ is reached, one has to revisit some state every $|S|$ time steps at the latest and thus form a cycle. The case for $R(\cdot) \geq 0$ is analogous.
- *Uniform strategy σ_{unif} of player 1 is proper and has finite value $\text{val}^{\sigma_{\text{unif}}} > -\infty$.* In other words, if player 1 is trying to reach the goal (case $R(\cdot) \leq 0$), player 2 cannot prevent player 1 from reaching S_G and thus make the cost of player 1 infinite.
- *Value $v(s)$ of every state $s \in S$ in the perfect-information refinement of the game G is finite and satisfies $v(s) < \infty$.* In other words, if player 2 is trying to reach the goal (case $R(\cdot) \geq 0$), player 1 cannot prevent player 2 from reaching S_G even if he had perfect information about the course of the game.

These assumptions help us bound the value of POSSPGs. The first assumption ensures that any play after reaching a goal state has no impact on the value of the objective and is standard in many problems/games [9,42,49,36,24] (especially shortest-path ones). The second assumption creates an incentive for one of the players (the one paying the cost) to reach a goal state from S_G instead of staying in the game forever. The last two assumptions ensure that the player who is incentivized to reach the goal is able to do so from every state in the game (and consequently the player cannot, e.g., make a mistake that would have prevented her from reaching the goal forever). These assumptions are common in goal-oriented problems [9] and are needed for the proposed goal algorithms (GoalHorizon – Section 5.1, and GoalDiscount – Section 5.2) to work. More precisely, they will later directly allow us to define finite lower and upper bounds on the value V^* of the game (i.e., POSSPG).

In Section 6.2, we show two domains inspired by real-world problems which fulfill these assumptions. It is, however, worth noting that there are games that can be interpreted as games with total-sum objective which do not satisfy above assumptions and cannot be solved by the proposed approach (such as Dante's purgatory which is challenging even in the perfect information setting [21]).

4.1. POSSPGs are undecidable

Before we present our algorithmic results that are aimed at approximating the value of POSSPGs, we demonstrate that reasoning about the exact value is undecidable (and hence approximating the value is the only algorithmically viable option).

Proposition 1. Let G be a POSSPG with value function V^* and let $b \in \Delta(S)$. Given a threshold t , determining whether $V^*(b) > t$ is undecidable.

The proof of Proposition 1 can be found in the appendix. The general idea of the proof relies on the facts that the analogous problem is undecidable for discounted POMDPs [34, Section 4.3] and that we can construct a POSSPG that would allow us to reason about the exact value of a discounted POMDP.

5. Solving POSSPGs

Although POSSPGs are similar to OS-POSGs, using discount factor $\gamma = 1$ and applying Algorithm 1 designed for discounted problems to solve POSSPGs is impossible. This is primarily based on the fact that the convergence results for Algorithm 1 rely on the γ -contractivity of the backup operator H^γ (Equation (1)). Even for the single-agent case, the standard HSVI algorithm for discounted problems cannot be applied to undiscounted Goal-POMDPs (as it loses its desirable theoretical guarantees [52,23]). We thus introduce modifications to the HSVI algorithm (Algorithm 1) so that the theoretical guarantees are preserved.

5.1. Increasing horizon (GoalHorizon)

One way to avoid possible convergence issues of Algorithm 1 applied on POSSPGs (e.g., reliance on γ -contractivity of H^γ) is to solve a finite-horizon variant of the game. In [54], it was shown, both theoretically and practically, that using a sufficiently long horizon $\mathcal{H}$ provides a close approximation of the solution of an indefinite-horizon problem without zero-cost actions.

5.1.1. k -cutoff game

Similarly to [23], we introduce a finite-horizon variant of the problem and term it a k -cutoff game. A k -cutoff game (denoted G_k) models a decision-making problem where player 1 can play arbitrarily in the first k stages of POSSPG G and is forced to follow uniform strategy σ_{unif} in the rest of the game. This corresponds to a finite-horizon OS-POSG with non-zero termination utility associated with G with utility function

$$U_b^k(\sigma_1, \sigma_2) = \mathbb{E}_{b, \sigma_1, \sigma_2} \left[\sum_{t=1}^k R(s^{(t-1)}, a_1^{(t)}, a_2^{(t)}) + \text{val}^{\sigma_{\text{unif}}}(b^{(k+1)}) \right], \quad (9)$$

where $b^{(k+1)}$ stands for the belief of player 1 after k -th stage of the game. Observe that in the case the game terminated within first k stages, player 1 has been notified of the termination by o_{reach} observation and $\text{Supp}(b^{(k+1)}) \subseteq S_G$. Since all states from S_G are absorbing states and thus rewards in them are 0, the value of uniform strategy in a belief state $b^{(k)}$ must be $\text{val}^{\sigma_{\text{unif}}}(b^{(k)}) = 0$. This ensures that the gap between the upper bound and the lower bound in any belief $b^{(k)}$ is 0. Similarly to the infinite-horizon case, we define $V_k^*(b)$ as the value of k -cutoff game starting in belief b , and we use $\text{val}_k^{\sigma_1}(b) = \min_{\sigma_2} U_b^k(\sigma_1, \sigma_2)$ to refer to the value of strategy σ_1 .

In this section, we will also be referring to the perfect information variant of an imperfect information game G and the strategy profile for it. In such case, we will slightly abuse notation and refer to the strategy played in the perfect information variant of the game as σ_{perf} , even if this strategy cannot be executed in the imperfect information setting at hand. We will also use $\text{val}^{\sigma_{\text{perf}}}$ to refer to the value of the perfectly informed strategy in the belief space. In this case, we assume that player 1 is informed about the initial state *before* the game starts. Hence $\text{val}^{\sigma_{\text{perf}}}(b) = \sum_{s \in S} b(s) \cdot v_s$ where v_s is the value of state s in the perfect-information game.

5.1.2. Solving infinite-horizon POSSPGs using k -cutoff games

The main idea of the GoalHorizon algorithm is to construct a sequence of k -cutoff games that approximate the POSSPG G and iteratively solve this sequence for increasing k . Note that, by definition, a k -cutoff game is a finite-horizon OS-POSG (as defined in Section 3) with non-zero termination utilities. Hence, each k -cutoff game can be solved using the FH-HSVI algorithm. We now describe the details for the GoalHorizon algorithm, following by the convergence properties of the algorithm.

Algorithm Description. The GoalHorizon algorithm aims to provide a solution ϵ -close to the solution of the original infinite-horizon POSSPG G . Thus, throughout the algorithm we must estimate bounds on the value of the original game G in order to determine whether the solution for the current k -cutoff game G_k is sufficiently close to G . For this reason, we define additional auxiliary upper bound $\bar{V}_k^+$ on the value V^* of the POSSPG G throughout the course of the algorithm. This allows us to terminate as soon as $\bar{V}_k^+(b_{\text{init}}) - \underline{V}_k^-(b_{\text{init}}) \leq \epsilon$, i.e., if the current solution of k -cutoff game G_k represented by the lower bound estimation differs by at most ϵ from upper bound $\bar{V}_k^+$.

The auxiliary upper bound $\bar{V}_k^+(b_{\text{init}})$ is defined as follows. Let G_{k+} be a variant of k -cutoff game in which, if the goal state is not reached in the first k stages, then the reward for player 1 is $\text{val}^{\sigma_{\text{perf}}}(b^{(k)})$ (i.e., assuming the game remains fully observable for player 1) instead of the continuation reward $\text{val}^{\sigma_{\text{unif}}}(b^{(k)})$ (i.e., player 1 following uniform strategy after first k stages). Let V_{k+}^+ denote the value function of G_{k+} . The function $\bar{V}_k^+$ is initialized as $\bar{V}_k^+(b) = \text{val}^{\sigma_{\text{perf}}}(b)$. While the procedure `solvecutoff`(k) is called, every time the point-based updates are performed on $\bar{V}_k^+$ and $\underline{V}_k^-$ at belief b , then the point-based update is also performed on $\bar{V}_k^+$ at belief b .

In order to use $\overline{V}_k^+(b_{\text{init}}) - \underline{V}^k(b_{\text{init}}) \leq \varepsilon$ as termination criterion, the auxiliary upper bound $\overline{V}_k^+(b_{\text{init}})$ must be a valid upper bound on $V^*(b_{\text{init}})$ throughout the whole course of the algorithm. In the original POSSPG G , player 1 has only partial information, while in G_{k+} , he is provided with some additional information. Therefore, his utility can only be improved (when playing G_{k+} compared to the original problem G). This means that after initialization it indeed holds $V^*(b_{\text{init}}) \leq \text{val}_1^{\sigma_{\text{perf}}}(b_{\text{init}}) = \overline{V}_k^+(b_{\text{init}})$. Performing point-based updates throughout the course of the algorithm is equivalent to applying backup operator H (i.e., $H^{\gamma=1}$). From Lemma 1, we know that backup operator H preserves function ordering (i.e., if $V \leq V'$ then $HV \leq HV'$). This means that $\overline{V}_k^+(b_{\text{init}})$ remains a valid upper bound on $V^*(b_{\text{init}})$ even after performing point based updates. Therefore it is guaranteed that it holds $V^*(b_{\text{init}}) \leq \overline{V}_k^+(b_{\text{init}})$ throughout the course of the whole algorithm and termination criterion $\overline{V}_k^+(b_{\text{init}}) - \underline{V}^k(b_{\text{init}}) \leq \varepsilon$ can indeed be used.

For similar reasons, the function $\overline{V}^k$ is then initialized to the value of perfect information strategy of k -cutoff game G_k (i.e., $\overline{V}^k = \text{val}_k^{\sigma_{\text{perf}}}$).

Algorithm 3: GoalHorizon algorithm for POSSPGs.

Input : target precision $\varepsilon > 0$, initial belief point b_{init}
Output: auxiliary upper bound $\overline{V}_k^+(b_{\text{init}})$, k -cutoff lower bound $\underline{V}^k(b_{\text{init}})$

```

1  $k \leftarrow 1$ 
2  $\underline{V}^k \leftarrow \text{val}^{\sigma_{\text{inf}}}$ 
3  $\overline{V}_k^+ \leftarrow \text{val}^{\sigma_{\text{perf}}}$ 
4 while  $\overline{V}_k^+(b_{\text{init}}) - \underline{V}^k(b_{\text{init}}) > \varepsilon$  do
5    $\overline{V}^k \leftarrow \text{val}_k^{\sigma_{\text{perf}}}$ 
6    $\text{solvecutoff}(k)$ 
7    $k \leftarrow k + 1$ 
8 procedure  $\text{solvecutoff}(k)$ 
9   while  $\overline{V}^k(b_{\text{init}}) - \underline{V}^k(b_{\text{init}}) > \eta * \varepsilon$  and  $\overline{V}_k^+(b_{\text{init}}) - \underline{V}^k(b_{\text{init}}) > \varepsilon$  do
10     $\text{Explore}(b_{\text{init}}, k - 1)$ 
11 procedure  $\text{Explore}(b^{(k)}, k)$ 
12   if  $\text{excess}_t(b^{(k)}) \leq 0$  then return
13    $\pi_1^{(k)} \leftarrow$  optimal strategy of P1 in  $[H\overline{V}^k](b^{(k)})$ 
14    $\pi_2^{(k)} \leftarrow$  optimal strategy of P2 in  $[H\underline{V}^k](b^{(k)})$ 
15    $b^{(k-1)} \leftarrow \arg \max_{a_1, o} \mathbb{P}_{b^{(k)}, \pi_1^{(k)}, \pi_2^{(k)}}[a_1, o] \cdot \text{excess}_{k-1}(\tau(b^{(k)}, a_1, \pi_2^{(k)}, o))$ 
16    $\text{Explore}(b^{(k-1)}, k - 1)$ 
17   Perform point-based update of  $\underline{V}^k$  and  $\overline{V}^k$  in  $b^{(k)}$ 
18   Perform point-based update of  $\overline{V}_k^+$  in  $b^{(k)}$ 

```

The GoalHorizon algorithm (Algorithm 3) starts by setting $k = 1$ and initializing $\underline{V}^k$, $\overline{V}^k$, and $\overline{V}_k^+$. For each value of k , we perform the following operations.

1. Solve k -cutoff game G_k using FH-HSVI from Section 3. This gives us a lower bound $\underline{V}^k(b_{\text{init}})$ on $V^*(b_{\text{init}})$ and consequently also a lower bound on $V^*(b_{\text{init}})$.² The gap $\overline{V}^k(b_{\text{init}}) - \underline{V}^k(b_{\text{init}})$ converges faster than the gap $\overline{V}_k^+(b_{\text{init}}) - \underline{V}^k(b_{\text{init}})$ used for the algorithm termination. Therefore, when solving G_k , we aim for a gap that is tighter than the target gap ε .³ This ensures that the termination gap $\overline{V}_k^+(b_{\text{init}}) - \underline{V}^k(b_{\text{init}})$ can reach the desired value, and the algorithm can terminate.
2. For every point-based update on $\overline{V}^k$ and $\underline{V}^k$ at belief b (line 17 of Algorithm 3), perform a point-based update on $\overline{V}_k^+$ at belief b (line 18 of Algorithm 3)
3. If the gap $\overline{V}_k^+(b_{\text{init}}) - \underline{V}^k(b_{\text{init}}) < \varepsilon$, then the algorithm terminates. Otherwise, increment k by 1.

Remember that the k -cutoff games correspond to finite-horizon OS-POSGs. This means their value function is partitioned based on the number of remaining stages. This has to be accounted for when representing bounds $\underline{V}^k$, $\overline{V}^k$, and $\overline{V}_k^+$ on the value of the POSSPG V^* . For $\underline{V}^k$, we use a separate set Γ_i for each stage i to represent a lower bound value function instead of just one set Γ for the whole problem like in the HSVI algorithm for discounted OS-POSGs. Similarly, for upper bounds $\overline{V}^k$ and $\overline{V}_k^+$, we use a separate set Υ_i for each stage i to represent an upper bound value function instead of just one set Υ for the whole problem.

² Note that the course of the future exploration (line 16) solely relies on strategies computed for k -cutoff game (lines 13 and 14).

³ The amount of tightness is controlled by the parameter η , $0 < \eta \leq 1$ (line 9 of Algorithm 3). Setting $\eta < 1$ usually allows Algorithm 3 to reach the desired precision for smaller k (compared to k needed for $\eta = 1$).

Convergence properties. In this section, we provide a theoretical analysis of the convergence properties of the proposed algorithm for solving one-sided POSSPGs (Algorithm 3). For simplicity, we will assume the case where the costs are non-zero (i.e., all $R(\cdot) > 0$ or all $R(\cdot) < 0$). We briefly discuss the case where zero rewards are allowed at the end of this section.

Our goal is to prove that for some k , Algorithm 3 approximates the value of the game within an ε interval (Theorem 2). Observe that the inner loop of Algorithm 3 for a fixed k (lines 9–17) corresponds to the algorithm for solving finite-horizon POSSGs with terminal rewards (Algorithm 2). Therefore, according to Theorem 1, this inner loop terminates, and once it does, the difference $\bar{V}^k(b_{init}) - \underline{V}^k(b_{init}) \leq \varepsilon_{inner}$. Due to the way the value functions are initialized, it holds $\underline{V}^k \leq V^* \leq \bar{V}^k$ for $k = 0$. This claim can be naturally extended to all $k \geq 0$ thanks to Lemma 1 (note that its proof is valid even for $\gamma = 1$ as is the case for the undiscounted objective). To prove that the algorithm converges, it is sufficient to show that $\bar{V}_k^+(b_{init}) - \bar{V}^k(b_{init})$ goes to zero. This ensures that the difference $\bar{V}_k^+(b_{init}) - \bar{V}^k(b_{init})$ becomes eventually less than $\varepsilon - \varepsilon_{inner}$ and the gap $\bar{V}_k^+(b_{init}) - \underline{V}^k(b_{init}) \leq \varepsilon$.

To prove Theorem 2, we characterize the point-based updates to the value functions $\bar{V}_k^+$ and $\bar{V}^k$ performed by the algorithm. During every call to the `Explore` procedure (Algorithm 3, lines 11–18), the algorithm calculates $y_k = [HV_{k-1}](b)$ and $y_k^+ = [HV_{k-1}^+](b)$. These values are used to update $\bar{V}_k^+$ and $\bar{V}^k$ by adding new points (b, y_k) and (b, y_k^+) to Υ_k and Υ_k^+ , respectively. Since both updates are performed in the same belief point b at the same time, we refer to these points jointly as (b, y_k, y_k^+) . We define sets of feasible updates $\mathcal{F}_k$, which characterize feasible combinations of values of y_k , y_k^+ and $y_k^+ - y_k$ in the sense of Lemma 2. We use this characterization to show that for sufficiently large k , the gap $y_k^+ - y_k$ can only be larger than $\varepsilon - \varepsilon_{inner}$ if y_k^+ or y_k attained impossible values that exceeded global lower or upper bounds on the value of the game.

Definition $\mathcal{F}_k$. Let G be a POSSPG and let $\bar{V}_0, \bar{V}_0^+$ be its k -cutoff and auxiliary upper bounds for the remaining horizon $k = 0$. Let $\bar{y}_0^+ = \max_{b \in \Delta(S)} \bar{V}_0^+(b)$ and $\underline{y}_0 = \min_{b \in \Delta(S)} \bar{V}_0(b)$ be bounds on $\bar{V}_0^+$ and $\bar{V}_0$, respectively. We define the sets of feasible updates $\mathcal{F}_k$ as follows:

$$\mathcal{F}_0 = \{(\bar{\Delta}_0 = \bar{y}_0^+ - \underline{y}_0, \bar{y}_0^+, \underline{y}_0)\} \quad (10)$$

$$\mathcal{F}_k = \left\{ (\kappa \bar{\Delta}_{k-1}, \bar{r} + \kappa \bar{y}_{k-1}^+, \underline{r} + \kappa \underline{y}_{k-1}) \mid \forall \kappa \in [0, 1], (\bar{\Delta}_{k-1}, \bar{y}_{k-1}^+, \underline{y}_{k-1}) \in \mathcal{F}_{k-1} \right\}, \quad (11)$$

where $\underline{r} = \min R(\cdot)$ and $\bar{r} = \max R(\cdot)$ are minimum and maximum rewards, respectively.

We can now proceed to formulate one of the key lemmas that characterizes the relation between value functions $\bar{V}^k$ and $\bar{V}_k^+$ (and the point-based updates used to form them) and the sets $\mathcal{F}_k$.

Lemma 2. Let G be a POSSPG and let $k \geq 0$. Then for every point-based update (b, y_k, y_k^+) of value functions V^k and V_k^+ , respectively, there exists $(\bar{\Delta}_k, \bar{y}_k^+, \underline{y}_k) \in \mathcal{F}_k$ such that $y_k^+ - y_k \leq \bar{\Delta}_k$, $y_k^+ \leq \bar{y}_k^+$ and $y_k \geq \underline{y}_k$. Furthermore, at any point of time, for every belief $b \in \Delta(S)$ there exists $(\bar{\Delta}_k, \bar{y}_k^+, \underline{y}_k) \in \mathcal{F}_k$ such that $\bar{V}_k^+(b) - \bar{V}^k(b) \leq \bar{\Delta}_k$, $\bar{V}_k^+(b) \leq \bar{y}_k^+$ and $\bar{V}^k(b) \geq \underline{y}_k$.

For detailed proof of Lemma 2, see Appendix A.3.

Definition $\mathcal{F}_k$, together with Lemma 2, already provides us with a useful characterization of the relation between gap $\bar{V}_k^+(b) - \bar{V}^k(b)$ and the values of $\bar{V}_k^+(b)$ and $\bar{V}^k$. For example, let us assume that Player 2 is trying to reach the goal while minimizing their cost (i.e., all rewards $R(\cdot) > 0$). With every increase in k , either the lower bound $\underline{r} + \kappa \underline{y}_{k-1}$ on $\underline{V}_k$ increases by $\underline{r}$ (and eventually becomes excessively high), or $\kappa < 1$, which results in the decrease of the gap $\bar{\Delta}_k = \kappa \bar{\Delta}_{k-1}$. Similarly, when $R(\cdot) < 0$, we either see a decrease in the gap or the upper bound $\bar{r} + \kappa \bar{y}_{k-1}^+$ decreases by at least $\bar{r}$ (eventually becoming excessively low).

Theorem 2. Let G be a POSSPG with non-zero rewards. For every $0 < \varepsilon_{inner} < \varepsilon$ there exists k such that $\bar{V}_k^+(b_{init}) - \underline{V}^k(b_{init}) \leq \varepsilon$, and Algorithm 3 thus terminates and yields ε approximation of $V^*(b_{init})$.

For detailed proof of Theorem 2, see Appendix A.4.

Observe that the results can be generalized to the case with zero rewards but no zero-cost cycles in a straightforward manner: Since there is no cycle with zero cost under any strategy profile, this means that the player had to pay the minimum cost between any two visits of the same state—this happens (at least) every $|S|$ steps. We can thus modify the theory by, e.g., modifying the Definition $\mathcal{F}_k$ by expressing $\mathcal{F}_{k+|S|}$ in terms of $\mathcal{F}_k$ (instead of $\mathcal{F}_{k+1}$ in terms of $\mathcal{F}_k$).

Final improvements. We now suggest some modifications that we use in our implementation, which decrease the runtime of the algorithm. Instead of initializing k -cutoff game upper bound $\bar{V}^k$ (line 5) as $\text{val}_k^{\sigma_{\text{perf}}}$ and auxiliary upper bound $\bar{V}^+$ (line 3) as $\text{val}^{\sigma_{\text{perf}}}$ for $k > 1$, we initialize them using points representing $\bar{V}^+$ in $(k-1)$ -cutoff game. This initialization is indeed an upper bound on V^* since adding an additional round for which the game is not played as perfect information version of POSSPG can only result in the same or worse utility of player 1 in k -cutoff game compared to $(k-1)$ -cutoff game. To avoid increased memory complexity of the

algorithm, we keep only points that are not dominated when moving from k -cutoff game to $(k + 1)$ -cutoff game. Note that to be able to achieve this, we have to use the same representation (individual sets Υ_i for each stage i) for both $\bar{V}^k$ and $\bar{V}^+$.

As V_k^* is a lower bound of V_{k+1}^* , the lower bound $\underline{V}^k$ of V_k^* (generated in line 6) is also a lower bound of V_{k+1}^* . So while performing `solvecutoff(k+1)` (line 6), we use $\underline{V}^k$ to initialize $\underline{V}^{k+1}$, instead of initializing as the value of uniform strategy $\text{val}^{\sigma_{\text{unif}}}$. As $\underline{V}^k$ is greater than or equal to the value of the uniform strategy, this initialization tends to decrease the runtime of the algorithm.

Finally, the target value $\eta * \epsilon$ of the gap $\bar{V}^k(b_{\text{init}}) - \underline{V}^k(b_{\text{init}})$ has effect on the runtime too. In our case, we observed the best results with the parameter $\eta = 0.9$. Furthermore, to reduce the number of alpha vectors and points for representing corresponding bounds, we prune out all dominated alpha vectors and points after each execution of line 10.

Possible modifications. Similarly to the discounted HSVI algorithm, we can optionally modify the GoalHorizon algorithm to perform point-based updates on individual bounds not only after returning from exploring recursion but also before going to the recursion (line 16). In terms of the actual Algorithm 3, this would mean executing commands from lines 17 and 18 also before executing line 16. Like the discounted HSVI algorithm, this modification can improve exploration in the deeper levels of exploring recursion. On the other hand, it can still result in using a higher number of alpha vectors or points for representing corresponding bounds and thus increase the memory consumption of the algorithm.

5.2. Increasing discount factor (GoalDiscount)

In order to (approximately) solve one-sided POSSPGs, we need to ensure that all distant costs that are potentially relevant for the computation of the near-equilibrium solution are reflected in the utility function of the proxy game with sufficient weight. In the previous section, we approached this by gradually increasing the horizon of the game (thus including increasingly distant costs in the computation) until an ϵ -optimal solution is found. Instead of this binary approach where a cost is either considered (if it is before the horizon) or not, we may opt to use discounting and assume that cost c_t paid at time step t is considered with a weight γ^{t-1} (where $0 < \gamma < 1$). If the discount factor is low, the weight γ^t of distant costs is negligible. However, as $\gamma \rightarrow 1$, the weights $\gamma^{t-1} \rightarrow 1$ as well and the game thus becomes strategically similar to the undiscounted POSSPG.

In this section, we use this observation and solve one-sided POSSPGs by considering their discounted counterparts G_γ (with value functions V_γ^*) as a proxy—i.e., we assume that the utility of the game G_γ is $\mathbb{E}[\sum_{t=1}^{\infty} \gamma^{t-1} r_t]$. To approach the solution of the undiscounted problem, we start with a small value of discount factor $0 < \gamma < 1$ and gradually increase it until the ϵ -optimal solution of the undiscounted game G is found. According to Theorem 3, the value of the discounted game G_γ will eventually become close to the value of the undiscounted POSSPG G .

Theorem 3. *Let G be a POSSPG with value function V^* and let $\epsilon > 0$. There exists $\underline{\gamma} < 1$ such that for every $\gamma \geq \underline{\gamma}$ and $b \in \Delta(S)$ we have $|V_\gamma^*(b) - V^*(b)| \leq \epsilon$.*

For proof of Theorem 3 see Appendix A.5.

The Algorithm. Our proposed algorithm (Algorithm 4) leverages the fact that every discounted proxy game G_γ (for $0 < \gamma < 1$) is a discounted one-sided POSG and thus the HSVI algorithm from [22,25] can be applied to solve it (see procedure `solvediscount( $\gamma$ )`). However, since our ultimate goal is to solve an undiscounted game, we also maintain bounds $\underline{V}_1$ and $\bar{V}_1$ on the value V^* of the original game G :

We initialize these bounds similarly to the discounted bounds. The lower bound $\underline{V}_1$ is initialized to the value of the uniform strategy in the *undiscounted* setting while the upper bound is set using the values of the perfect-information refinement of the POSSPG G .⁴ We update these bounds alongside the updates to the discounted bounds. Whenever an update to $\underline{V}_\gamma$ and $\bar{V}_\gamma$ is performed (line 19, using the solution of stage games $[H_\gamma \underline{V}_\gamma](b)$ and $[H_\gamma \bar{V}_\gamma](b)$), we perform analogous undiscounted updates to $\underline{V}_1$ and $\bar{V}_1$ (line 20, using the solution of *undiscounted* stage games $[H_1 \underline{V}_1](b)$ and $[H_1 \bar{V}_1](b)$).

The presence of undiscounted bounds $\underline{V}_1$ and $\bar{V}_1$ allows us to determine when we reached the desired approximation quality (line 11). Unless the desired precision ϵ is reached, we increase the discount factor γ to γ' ($\gamma < \gamma' < 1$) and solve the resulting discounted game $G_{\gamma'}$ (line 8).

Note that similarly to GoalHorizon, the gap between bounds on the easier-to-solve approximation G_γ of the game G (i.e., discounted bounds $\underline{V}_\gamma$ and $\bar{V}_\gamma$) converges faster than the gap between undiscounted bounds $\underline{V}_1$ and $\bar{V}_1$. Therefore, when solving discounted proxy game G_γ , we aim for a gap that is tighter than the target gap ϵ .⁵ This helps to ensure that the termination gap $\bar{V}_1(b_{\text{init}}) - \underline{V}_1(b_{\text{init}})$ can reach the desired value, and the algorithm can terminate (line 8 of Algorithm 4).

Final improvements. To improve the run-time of Algorithm 4, we suggest the following modifications. Instead of re-initializing the discounted lower bound as $\underline{V}_{\gamma'} = \text{val}_{\gamma'}^{\sigma_{\text{unif}}}$ (line 5) and the discounted upper bound as $\bar{V}_{\gamma'} = \text{val}_{\gamma'}^{\sigma_{\text{perf}}}$ (line 6) every time we increase

⁴ Assuming v_s is the value of state $s \in S$ of the *undiscounted* perfect information stochastic game, the initial $\bar{V}_1$ is set to $\bar{V}_1(b) := \sum_{s \in S} b(s)v_s$.

⁵ The amount of tightness is controlled by the parameter η , $0 < \eta \leq 1$ (line 11 of Algorithm 4). Setting $\eta < 1$ usually allows Algorithm 4 to reach the desired precision for a smaller discount factor γ (compared to γ needed for $\eta = 1$).

Algorithm 4: GoalDiscount algorithm for POSSPGs.

Input : initial discount factor γ_{init} , target precision $\epsilon > 0$,
 schedule for increasing the discount factor γ , initial belief point b_{init}

Output: undiscounted upper bound $\overline{V}_1(b_{\text{init}})$,
 undiscounted lower bound $\underline{V}_1(b_{\text{init}})$

```

1  $\gamma \leftarrow \gamma_{\text{init}}$ 
2  $\underline{V}_1 \leftarrow \text{val}_1^{\sigma_{\text{unif}}}$ 
3  $\overline{V}_1 \leftarrow \text{val}_1^{\sigma_{\text{perf}}}$ 
4 while true do
5    $\underline{V}_\gamma \leftarrow \text{val}_\gamma^{\sigma_{\text{unif}}}$ 
6    $\overline{V}_\gamma \leftarrow \text{val}_\gamma^{\sigma_{\text{perf}}}$ 
7   solvediscount( $\gamma$ )
8   if  $\overline{V}_1(b_{\text{init}}) - \underline{V}_1(b_{\text{init}}) > \epsilon$  then Increase  $\gamma$  according to provided schedule
9   else break
10 procedure solvediscount( $\gamma$ )
11   while  $\overline{V}_\gamma(b_{\text{init}}) - \underline{V}_\gamma(b_{\text{init}}) > \eta * \epsilon$  and  $\overline{V}_1(b_{\text{init}}) - \underline{V}_1(b_{\text{init}}) > \epsilon$  do
12     Explore( $b_{\text{init}}, 1$ )
13 procedure Explore( $b^{(t)}, t$ )
14   if  $\text{excess}_s(b^{(t)}) \leq 0$  then return
15    $\pi_1^{(t)} \leftarrow$  optimal strategy of P1 in  $[H^\gamma \overline{V}_\gamma](b^{(t)})$ 
16    $\pi_2^{(t)} \leftarrow$  optimal strategy of P2 in  $[H^\gamma \underline{V}_\gamma](b^{(t)})$ 
17    $b^{(t+1)} \leftarrow \arg \max_{a_1, o} \mathbb{P}_{b^{(t)}, \pi_1^{(t)}, \pi_2^{(t)}}[a_1, o] \cdot \text{excess}_{t+1}(\tau(b^{(t)}, a_1, \pi_2^{(t)}, o))$ 
18   Explore( $b^{(t+1)}, t+1$ )
19   Perform (discounted) point-based update of  $\underline{V}_\gamma$  and  $\overline{V}_\gamma$  in  $b^{(t)}$ 
20   Perform (undiscounted) point-based update of  $\underline{V}_1$  and  $\overline{V}_1$  in  $b^{(t)}$ 

```

discount factor γ to a new value γ' , we can leverage computations done for the previous discount factor γ . We distinguish two scenarios: $R(\cdot) \leq 0$ and $R(\cdot) \geq 0$. If $R(\cdot) \leq 0$, then the value obtained for a larger discount factor γ' will always be less than or equal to the value obtained for the smaller γ (i.e., $V_\gamma \geq V_{\gamma'} \geq \underline{V}_1$). Therefore, in such a case, we can initialize discounted lower bound as $\underline{V}_{\gamma'} = \max(\text{val}_{\gamma'}^{\sigma_{\text{unif}}}, \underline{V}_1)$ and discounted upper bound as $\overline{V}_{\gamma'} = \min(\text{val}_{\gamma'}^{\sigma_{\text{perf}}}, \overline{V}_\gamma)$. Similarly, if $R(\cdot) \geq 0$, then the value obtained for a larger discount factor γ' will always be greater than or equal to the value obtained for the smaller γ (i.e., $V_\gamma \leq V_{\gamma'} \leq \overline{V}_1$). Therefore, in such a case, we can initialize discounted lower bound as $\underline{V}_{\gamma'} = \max(\text{val}_{\gamma'}^{\sigma_{\text{unif}}}, \underline{V}_\gamma)$ and discounted upper bound as $\overline{V}_{\gamma'} = \min(\text{val}_{\gamma'}^{\sigma_{\text{perf}}}, \overline{V}_1)$.

Furthermore, to reduce the number of alpha vectors and points for representing corresponding bounds, we prune out all dominated alpha vectors and points after each execution of line 12.

Possible modifications. Just like in the case of the discounted HSVI algorithm and the GoalHorizon algorithm, we can optionally modify the GoalDiscount algorithm to perform point-based updates on individual bounds not only after returning from exploring recursion but also before going to the recursion (line 18). In terms of the actual Algorithm 4, this would mean executing commands from lines 19 and 20 also before executing line 18. Again, this modification can improve exploration in the deeper levels of exploring recursion. On the other hand, it can result in using more alpha vectors or points for representing corresponding bounds and thus increase the memory consumption of the algorithm.

Additionally to the modification to point-based updates, it is possible to tweak the way how the discount factor γ is increased on line 8. If the increases to the discount factor γ are done by smaller increments, this modification may result in finding a smaller discount factor γ required for finding a sufficient solution. Being able to find a sufficient solution for a smaller discount factor γ can lead to faster computational times since the computation complexity increases with higher values of γ . In Section 6.1, we provide two specific examples of possible schedules for increasing discount factor γ that differ in the size of the increments.

6. Experiments

In this section, we present experimental results of both proposed algorithms for solving POSSPGs (GoalHorizon algorithm - Section 5.1, GoalDiscount algorithm - Section 5.2). First, we provide details on the experimental environment. Second, we describe the testing domains we used. Then we discuss experimental results on solution quality and algorithm scalability. Finally, we show the larger picture of the experimental results.

The results presented in this section compare both the GoalHorizon algorithm and the GoalDiscount algorithm to the traditional discounted HSVI algorithm (we refer to it also as dHSVI) which will serve as our baseline (we present more details on this algorithm, and in particular the choice of discount factor γ , in Section 6.1). When comparing two algorithms on a given domain, we are interested in the behavior of the algorithms with increasing difficulty of the problems from that domain. Therefore, some reasonable metric is required to determine the difficulty of individual instances. Often, it is reasonable to determine the difficulty

Algorithm 5: Modified HSVI for POSSPGs (udHSVI).

Input : discount factor γ , target precision $\varepsilon > 0$, initial belief point b_{init}
Output: undiscounted upper bound $\overline{V}_1(b_{\text{init}})$,
undiscounted lower bound $\underline{V}_1(b_{\text{init}})$

```

1  $\underline{V}_1 \leftarrow \text{val}_1^{\sigma_{\text{init}}}$ 
2  $\overline{V}_1 \leftarrow \text{val}_1^{\sigma_{\text{init}}}$ 
3  $\underline{V}_\gamma \leftarrow \text{val}_\gamma^{\sigma_{\text{init}}}$ 
4  $\overline{V}_\gamma \leftarrow \text{val}_\gamma^{\sigma_{\text{init}}}$ 
5 while  $\overline{V}_1(b_{\text{init}}) - \underline{V}_1(b_{\text{init}}) > \varepsilon$  do
6    $\text{Explore}(b_{\text{init}}, 1)$ 
7 procedure  $\text{Explore}(b^{(t)}, t)$ 
8   if  $\text{excess}_t(b^{(t)}) \leq 0$  then return
9    $\pi_1^{(t)} \leftarrow \text{optimal strategy of P1 in } [H' \overline{V}_\gamma](b^{(t)})$ 
10   $\pi_2^{(t)} \leftarrow \text{optimal strategy of P2 in } [H' \underline{V}_\gamma](b^{(t)})$ 
11   $b^{(t+1)} \leftarrow \arg \max_{a_1, o} \mathbb{P}_{b^{(t)}, \pi_1^{(t)}, \pi_2^{(t)}}[a_1, o] \cdot \text{excess}_{t+1}(\tau(b^{(t)}, a_1, \pi_2^{(t)}, o))$ 
12   $\text{Explore}(b^{(t+1)}, t+1)$ 
13  Perform (discounted) point-based update of  $\underline{V}_\gamma$  and  $\overline{V}_\gamma$  in  $b^{(t)}$ 
14  Perform (undiscounted) point-based update of  $\underline{V}_1$  and  $\overline{V}_1$  in  $b^{(t)}$ 

```

of an instance based on the number of states. This particular metric is especially fitting for extensive form games since these can be solved using a linear program that is linear in the size of the problem instance (i.e., the number of states) [29].

However, using a metric based on the number of states is not that fitting for determining the difficulty of POSSPGs. This is mostly due to the fact that the computational complexity of HSVI-based algorithms directly increases with the increased discount factor γ . The reason is that higher values of discount factor γ result in longer realizations of games as the future rewards/costs will not be discounted that quickly and will remain relevant for longer. This implies longer strategies and larger strategy space. Therefore, using a metric based on the discount factor is more fitting. In our work, we determine the difficulty of individual instances based on the maximum discount factor γ reached by GoalDiscount algorithm.⁶ There might still be a connection between instance size and the value of discount factor γ^* , but in general, having a larger instance of a game does not necessarily imply higher γ^* .

As stated before, we are interested in the behavior of algorithms in terms of solution quality and scalability. First (see Section 6.3), we show that both the GoalHorizon algorithm and the GoalDiscount algorithm outperform the naive application of the traditional discounted HSVI algorithm (dHSVI) presented in Section 2.1 in terms of solution quality, where the discounted HSVI algorithm is unable to find a solution with desired properties under the undiscounted setting for the majority of instances.

Since the traditional discounted HSVI algorithm is unable to achieve solution quality comparable to the proposed goal algorithms, we modify it to look for the solution with desired properties under the *undiscounted setting* (we will call this modified algorithm udHSVI). To do so, we take advantage of being able to compute undiscounted counterparts for each discounted alpha vector (for the lower bound) and point (for the upper bound) – the same way we do it in GoalDiscount algorithm (Section 5.2). Besides keeping track of undiscounted counterparts (i.e., undiscounted lower bound $\underline{V}_1$ and undiscounted upper bound $\overline{V}_1$) for both discounted bounds (i.e., discounted lower bound $\underline{V}_\gamma$ and discounted upper bound $\overline{V}_\gamma$), the modified algorithm (Algorithm 5) works the same way as the traditional discounted HSVI algorithm but terminates the computation (line 5) only when the undiscounted counterparts of both bounds are sufficiently close to each other. Note that this does not simply work for any value of discount factor but it requires discount factor with a value equal or greater than some instance-specific threshold. If such a discount factor is not available, fulfillment of the termination condition $\overline{V}_1(b_{\text{init}}) - \underline{V}_1(b_{\text{init}}) > \varepsilon$ (line 5) is not guaranteed. The algorithm can still be used with a different discount factor (that is below the instance-specific threshold). In such a case, however, the termination condition has to be changed to $\overline{V}_\gamma(b_{\text{init}}) - \underline{V}_\gamma(b_{\text{init}}) > \varepsilon$ in order to guarantee algorithm termination and provide bounds on the undiscounted evaluation of the discounted solution. Just like in the case of our implementations of all previously presented algorithms, our implementation of Algorithm 5 prunes out dominated alpha vectors and points after each iteration (i.e., execution of line 6) to reduce their number. Also note that, similarly to the discounted HSVI algorithm, the udHSVI algorithm can be optionally modified to perform point-based updates on individual bounds not only after returning from exploring recursion but also before going to the recursion (line 12). In terms of the actual Algorithm 5, this would mean executing commands from lines 13 and 14 also before line 12. Just like the discounted HSVI algorithm, this modification can improve exploration in the deeper levels of the exploring recursion. On the other hand, it can result in using a higher number of alpha vectors or points to represent corresponding bounds and thus increase the memory consumption of the algorithm. Then we compare both proposed goal algorithms to udHSVI in terms of scalability (see Section 6.4).⁷

⁶ If the GoalDiscount algorithm did not find the solution, we used the largest value of discount factor γ with started computation.

⁷ We compared the scalability of both proposed goal algorithms also with the original discounted HSVI algorithm (dHSVI) and provide the results in the appendix – see Appendix B.3.

6.1. Experiment settings

All computational results have been obtained on computers equipped with *Intel Xeon Scalable Gold 6146* processors while limiting the runtime to 10 hours (36000 seconds) and RAM to 128 GB. We used CPLEX 12.9 to solve linear programs.

We also assume several computation settings for each algorithm based on possible modifications available to individual algorithms.

For all considered algorithms (GoalHorizon, GoalDiscount, dHSVI and udHSVI), we distinguish two settings based on whether point-based updates were applied to individual bounds only after the return from recursion (one-way update) or also before entering the recursion (both-way update).

For GoalDiscount, in addition, we try two different ways of increasing the value of the discount factor γ (Algorithm 4, line 8). The first way is referred to as *rough increase* and the new value of the discount factor is computed as $\gamma' = \gamma * 10^{-1} + 0.9$. The second one is *smooth increase* and it computes the new value of the discount factor in the following way:

$$\gamma' = \begin{cases} \gamma + g & \text{if } \gamma + g < 1, \\ 1 - 10^{\lfloor \log_{10} g \rfloor} & \text{otherwise } \wedge g = g * 10^{-1} \text{ for future increases.} \end{cases}$$

For the experiments, we set the starting value of discount factor $\gamma = 0.9$ for both *rough increase* and *smooth increase*, and the initial value of increment g for *smooth increase* was set to $g = 0.025$. Remember that after each increase the value of the discount factor becomes $\gamma = \gamma'$ (see Algorithm 4, line 8). For these initial values, we obtain the following increasing sequences of discount factor γ . For *rough increase*, we get $\{0.9, 0.99, 0.999, \dots\}$ and for *smooth increase* we get $\{0.9, 0.925, 0.95, 0.975, 0.99, 0.9925, 0.995, \dots\}$.

Despite the fact that performing either one-way update or both-way update is the only presented possible modification to the discounted HSVI algorithm, we will also be distinguishing the *rough increase* and *smooth increase* settings. This is because running the discounted HSVI algorithm requires providing the value of the discount factor γ . For all problem instances in our experiments, we provided the discounted HSVI algorithm with the value of γ needed by the GoalDiscount algorithm to find the solution on that particular instance.⁸ Thus, in terms of the discounted HSVI algorithm, *rough increase* and *smooth increase* settings represent the way how the used discount factor was obtained.

Identically to the discounted HSVI algorithm (dHSVI), running the udHSVI algorithm also requires providing the value of the discount factor γ . For all problem instances, we do the same thing as for the dHSVI algorithm and provide udHSVI with the value of γ needed by the GoalDiscount algorithm to find the solution on that particular instance⁸. This means that, just like for the dHSVI algorithm, we distinguish *rough increase* and *smooth increase* settings based on the way how the used discount factor was obtained.

In the paper, we show results only for *two-way update*, *rough increase* and *two-way update*, *smooth increase* as the behavior under *one-way update* and *two-way update* is similar. The results for the remaining computational settings combinations (*one-way update*, *rough increase* and *one-way update*, *smooth increase*) can be found in the appendix (see Appendix B).

6.2. Testing domains

The experimental evaluation of the presented algorithms was done on two different domains of POSSPGs. Both domains are inspired by real-world security problems. First, pursuit-evasion games appear in mobile robotics [12] or security [26] where a group of agents is trying to find and capture another evading agent. Second, privilege-escalation games model the security problem of discovering an attacker in a computer system [30]. We give a detailed description of both domains including the solution quality the algorithms were required to achieve.

6.2.1. Pursuit-evasion games (peg)

In pursuit-evasion games [12,26], a team of K centrally controlled pursuers (we consider a team of $K = 2$ pursuers) is trying to locate and capture the evader, who is trying to avoid getting captured. We assume the game being played on a grid (dimensions $3 \times N$; $N \in \{3, 4, 5, 6\}$), with the pursuers starting in the top-left corner and the evader in the bottom-right corner – see Fig. 5. In each step, the units move to one of their adjacent locations (i.e., the actions of the evader are $A_2 = \{\text{left, right, up, down}\}$, while the actions available to the team of pursuers are joint actions for all units in the team, $A_1 = (A_2)^K$). The game ends when one of the units from the team of pursuers enters the same cell as the evader or directly swaps position with the evader. The reward for all transitions in the game is -1 . The pursuer knows the location of their units, but the current location of the evader is not known.

Domain settings. For each domain size (i.e., value of N) used in experiments, we generated 2 instances using different fixed seeds.⁹ We denote instances of pursuit-evasion game played on a grid of a given size $3 \times N$, as *pegN*. All solution methods were required to find an ϵ -optimal solution where ϵ was set to 1.

⁸ If the GoalDiscount algorithm did not find the solution, we used the largest value of discount factor γ with started computation.

⁹ Note that, for this domain, the different seed values do not affect the solution of the generated instance (i.e., its value) and can only change the structure of the representation of the generated instance (e.g., order of pursuers in the team) which can result in HSVI-based algorithms exploring beliefs in a different order.

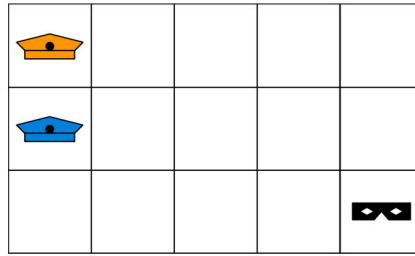


Fig. 5. Example of the pursuit-evasion game.

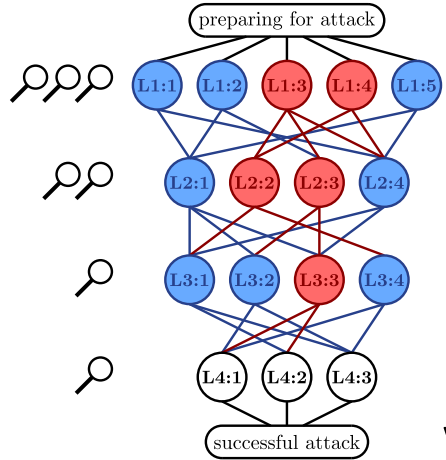


Fig. 6. Example of the privilege escalation game.

6.2.2. Privilege escalation games (privEsc)

The privilege escalation games domain is inspired by the computer security problem of privilege escalation on networks and corresponding attack graphs and network privilege graphs [30,27,60]. It simulates a situation where an attacker has to pass through several levels of security or privilege checks in a computer network to be able to reach the desired host computer and, for example, steal data. In Fig. 6, we depict an example instance of such a game with four layers. The dynamics of these games work as follows. The game starts with the attacker preparing for an attack on the topmost layer. The goal of the attacker is to perform a sequence of successful attacks to move through these layers and reach the “successful attack” state (this could represent for example reading a database). On the other hand, the goal of the defender is to catch the attacker as many times as possible, where each catch results in the game moving back to the first layer (i.e., to the “preparing for attack” state) and yields a positive reward for the defender. On each layer, the defender has a limited number of available security resources (strictly smaller than the number of available targets) to protect nodes in that layer. While the defender deploys security resources on the current layer (the layer the attacker has to bypass next), the attacker picks a target of his attack on this layer from the set of possible nodes. Note that except for the first layer this set does not need to include all nodes of the particular layer and it depends on the successful attack performed on the previous layer. Furthermore, the set of nodes available for the attack is unknown by the defender. If the attack is successful (i.e., no security resource is deployed to its target), the game moves to the next layer. The defender is provided with information that there was a successful attack on the current layer and a set of possible targets of that attack (i.e., the defender knows that the successful attack was performed on one of the targets from the set but cannot precisely distinguish on which one, in Fig. 6, we mark these sets of “indistinguishable” targets by different colors¹⁰). If the attack is unsuccessful (i.e., there is a security resource deployed to its target), the attacker is caught and moved back to the “preparing for attack” state.

Example. To further illustrate the dynamics of privilege escalation games, we present the following example. Let us assume the game in Fig. 6. The game starts with the attacker being in the “preparing for attack” state and the defender deploying his three available security resources on the first layer (L1). Assume that the defender deploys security resources to the last three nodes (L1:3, L1:4, and L1:5). If the attacker attacks one of these nodes, the defender receives a positive reward for catching the attacker, and the attacker is returned back to the “preparing for attack” state. On the other hand, if the attacker attacks, for example, the leftmost node (L1:1), such an attack would be successful (the defender is not protecting that target), and the game will move to the next layer (L2).

¹⁰ We used two sets of “indistinguishable” targets in the Fig. 6 to keep things simple. In general, there can be a higher number of indistinguishable sets.

After assumed successful attack on blue node $L1:1$ and moving to layer $L2$, the defender receives information that there was a successful attack on layer $L1$ and this attack was performed on one of the blue nodes (similarly, if the successful attack would target a red node, the defender would know that the attack was performed on one of the red nodes). The only unprotected blue nodes were $L1:1$ and $L1:2$, which means that on layer $L2$, the attacker can possibly attack only nodes $L2:1$, $L2:3$, and $L2:4$ (in Fig. 6 these constraints on possible attacks on the next layer are represented by connection lines — these lines have the same color as the target from layer $L1$). Similarly to layer $L1$, the defender deploys his available security resources (for layer $L2$, there are only 2 of them), and the attacker picks his new target (only from the set of possible targets). So, this time, let us assume that the defender would protect both blue nodes (i.e., nodes $L2:1$ and $L2:4$). If the attacker picks one of these protected nodes as his target, he gets caught, the game returns back to the first layer with the attacker in the “preparing for attack” state, and the defender receives a positive reward. Again, if the attack is successful, the game proceeds to the next layer. In this case, assuming the aforementioned deployment of security resources, the successful attack can be performed only on the red node $L2:3$.

The defender receives information that there was a successful attack on one of the red nodes on layer $L2$. However, since only one red node was available for attack (based on previously obtained information), the defender knows exactly that node $L2:3$ was attacked on the second layer $L2$ and based on the fact that only nodes $L3:2$ and $L3:3$ can be attacked on the third layer $L3$. On layer $L3$, the defender deploys security resources once again (this time only one), and let us assume that he decides to protect the blue node $L3:2$. In case of an unsuccessful attack, the game moves back to the first layer $L1$, and the defender receives a positive reward. If the attack is successful (i.e., the attacker attacked red node $L3:3$), the game proceeds to the very last layer, and the defender receives information that there was a successful attack on a red node on layer $L3$. Since there is only one red node on layer $L3$, this means that on the last layer, $L4$, only nodes $L4:1$ and $L4:2$ can be attacked. Again, the defender will deploy available security resources, and the attacker will pick the target of his attack. If the attack is unsuccessful, the game moves back to the first layer, and the defender receives a positive reward. Finally, if this attack on the last layer is successful, the game ends, and the defender receives a negative reward (i.e., penalty) for the attacker successfully passing through all layers of defense and obtaining desired data.

Domain settings. For each domain size (number of layers) used in experiments, we randomly generated 10 instances using different fixed seeds. We denote instances of privilege escalation game with a given number of layers L (for the experiments we assume values $L \in \{4, 5, 6, 7\}$) as *privEsc* L . All solution methods were required to find an ϵ -optimal solution where ϵ was set to 1.

6.3. Solution quality

Due to the nature of individual algorithms, only solutions provided by the proposed undiscounted algorithms (GoalHorizon and GoalDiscount) are guaranteed to have desired properties under *undiscounted setting* (i.e., providing sufficient approximation of the undiscounted solution represented by a sufficiently tight gap under the undiscounted setting). For the traditional discounted HSVI algorithm (dHSVI), this does not hold, and desired properties of found solutions are guaranteed only for the *discounted setting* (i.e., naive application of the traditional discounted HSVI algorithm guarantees only a sufficient approximation of the discounted solution). Therefore, in this section, we are focusing on the comparison of algorithms solely on the solution quality. The metric we use for this is whether the algorithm succeeds in producing a solution with a gap of at most 1 between the upper and lower bound when these are evaluated under the undiscounted setting. If the gap is sufficiently small, we do not further differentiate its precise value and consider all such gaps to be equal to 1. On the other hand, for all undiscounted gaps larger than 1, we consider their precise value, which allows us to compare unfinished solutions as well. The precise gap values achieved by compared algorithms on all individual instances are presented in Appendix B.1.3.

When comparing *goal algorithms* (GoalHorizon, GoalDiscount) with the *baseline* (discounted HSVI algorithm - dHSVI), we visualize the results using scatter plots of gap ratio $\frac{\text{goal_algorithm}}{\text{baseline}}$. If both compared algorithms find a sufficient solution in terms of the undiscounted gap, the ratio will be 1, and thus, the corresponding point will land on the red 1-line. If the point is above the red line, the solution provided by the discounted HSVI algorithm happens to be better under the undiscounted setting than the solution found by the GoalHorizon algorithm or the GoalDiscount algorithm respectively. Similarly, if the point is below the red line, the solution found by the discounted HSVI algorithm is worse than the one provided by the GoalHorizon algorithm or the GoalDiscount algorithm, respectively.

Note that all HSVI-based algorithms provide any-time solutions (i.e., the last finished iteration of the algorithm still provides some solution that can be used). Therefore, we can compare the algorithms even in a case when neither of the algorithms finds the sufficient answer (for GoalHorizon, GoalDiscount and udHSVI this happens only if the algorithm exceeds the given time limit of 10 hours or runs out of 128 GB of memory). In such a case, the ratio shows which one of the algorithms provided a solution closer to the sufficient one (the discounted algorithm dHSVI – if the point is above the red 1-line, or the considered undiscounted algorithm – if the point is below the red 1-line).

6.3.1. GoalHorizon vs discounted HSVI

First, let us compare the discounted HSVI algorithm to the GoalHorizon algorithm. We can see that solutions provided by the GoalHorizon algorithm are generally better than those of the discounted HSVI algorithm (dHSVI) when evaluated under undiscounted setting regardless of the computation setting (Fig. 7), except for solutions for *peg6*, for which the GoalHorizon algorithm was not able to finish in the given time limit. Furthermore, we can observe that GoalHorizon compares to the discounted HSVI algorithm better under the *smooth increase* computational setting. The reason for this is that the discounted HSVI algorithm uses smaller values of the discount factor γ (compared to the *rough increase* computational setting). As the computational complexity

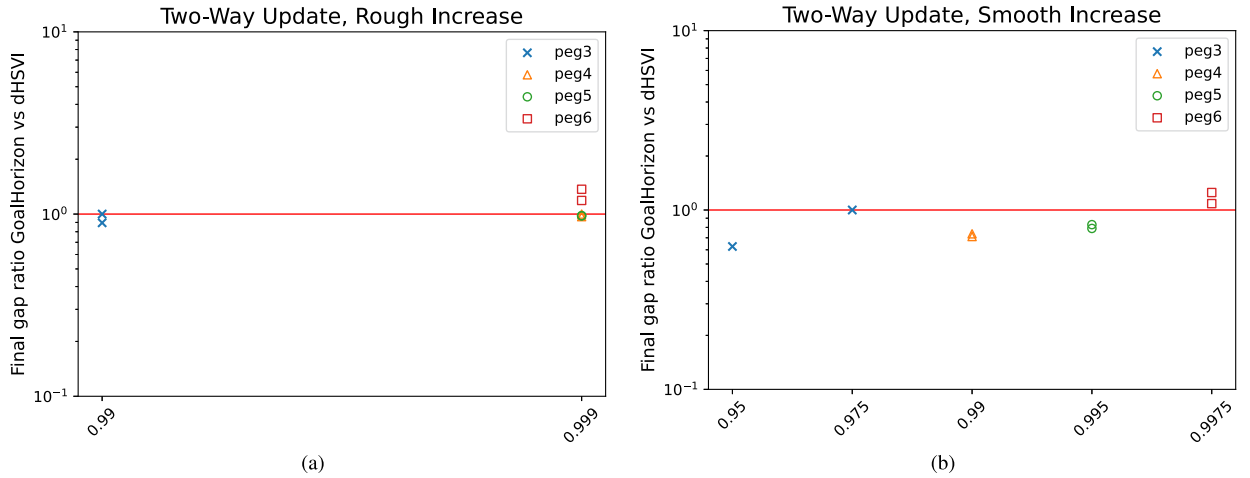


Fig. 7. Comparison of the quality of solutions found by GoalHorizon algorithm and the discounted HSVI (dHSVI) algorithm when evaluated under the undiscounted setting on the domain of pursuit-evasion games.

of HSVI-based algorithms directly increases with the increased discount factor γ and smaller values of the discount factor γ decrease the runtime of discounted HSVI algorithm. However, as the value of the discount factor γ becomes smaller (i.e., further from 1), the chance of a solution found by the discounted HSVI algorithm having the desired properties when evaluated under an undiscounted setting decreases. This can be seen on instances of *peg4* and *peg5*. If the discounted HSVI algorithm uses a larger discount factor (rough increase computational setting), the found solutions have desired properties also when evaluated under the undiscounted setting (just like those obtained by the GoalHorizon algorithm — corresponding points in the scatter plot lie on the red 1-line, Fig. 7a). On the other hand, when using smaller values of discount factor γ (that are sufficient for the GoalDiscount algorithm to find a solution with desired properties), we can see that this no longer holds and the undiscounted evaluation of solutions provided by the discounted HSVI algorithm do not have desired properties (corresponding points in the scatter plot are below the red 1-line, Fig. 7b).

The difference in the quality of provided solutions between the GoalHorizon algorithm and the discounted HSVI algorithm is even more pronounced when comparing these two algorithms on the domain of privilege escalation games (Fig. 8). We can see that on this particular domain, the solutions found by the GoalHorizon algorithm are always better in terms of quality with no exceptions regardless of the computational setting. The biggest difference in terms of solution quality is present for larger values of discount factor γ where the discounted HSVI algorithm was not able even to find the solution in the given time limit while the GoalHorizon algorithm solved corresponding instances without any issue. Furthermore, the solutions provided by the GoalHorizon algorithm are better not only for instances successfully solved in the given time limit but also for the “unfinished” (partial) ones. Under the two-way update, smooth increase computational setting combination, there are two instances of *privEsc7* unsolved by both algorithms for $\gamma = 0.99975$ due to exceeding the given time limit of 10 hours. For these two particular instances, the GoalHorizon algorithm found solutions with gaps 4.814 and 6.766, while the discounted HSVI algorithm produced solutions with undiscounted gaps 67.281 and 51.415.

6.3.2. GoalDiscount vs discounted HSVI

When comparing the solution quality of the GoalDiscount algorithm and discounted HSVI algorithm, we can observe similar results as for the GoalHorizon algorithm. On the pursuit-evasion domain (Fig. 9), the GoalDiscount algorithm is capable of finding solutions with better quality for all instances and discount factors than the discounted HSVI algorithm. This is true not only for instances solved by the GoalDiscount algorithm but also for the two instances of *peg6* which were unsolved under the two-way update, smooth increase computational setting combination due to exceeding the memory limit of 128 GB. The corresponding solutions found by the GoalDiscount algorithm (despite being only partial ones) reached gap 1.005 and 1.024, respectively. On the other hand, solutions provided by the discounted HSVI algorithm for these particular instances under the same computational setting had undiscounted gaps of 1.163 and 1.164, respectively.

We can observe that the GoalDiscount algorithm outperforms the discounted HSVI algorithm even more significantly on the domain of privilege escalation games (Fig. 10). Just like in the case of the GoalHorizon algorithm, the biggest difference in solution quality is on instances requiring larger values of the discount factor γ . Similarly to the pursuit-evasion domain, the GoalDiscount algorithm has better performance in terms of solution quality not only for solutions completed in the given time limit but also for the partial ones. Under the two-way update, smooth increase computational setting combination, there is only one instance of *privEsc7* unsolved by both algorithms for $\gamma = 0.99975$. For this instance, the GoalDiscount algorithm achieves a gap of 3.742 while the undiscounted gap of the solution provided by the discounted HSVI algorithm is 51.415 (this particular instance was not solved in the given time limit by the GoalHorizon algorithm as well and provided partial solution achieved gap of 6.766).

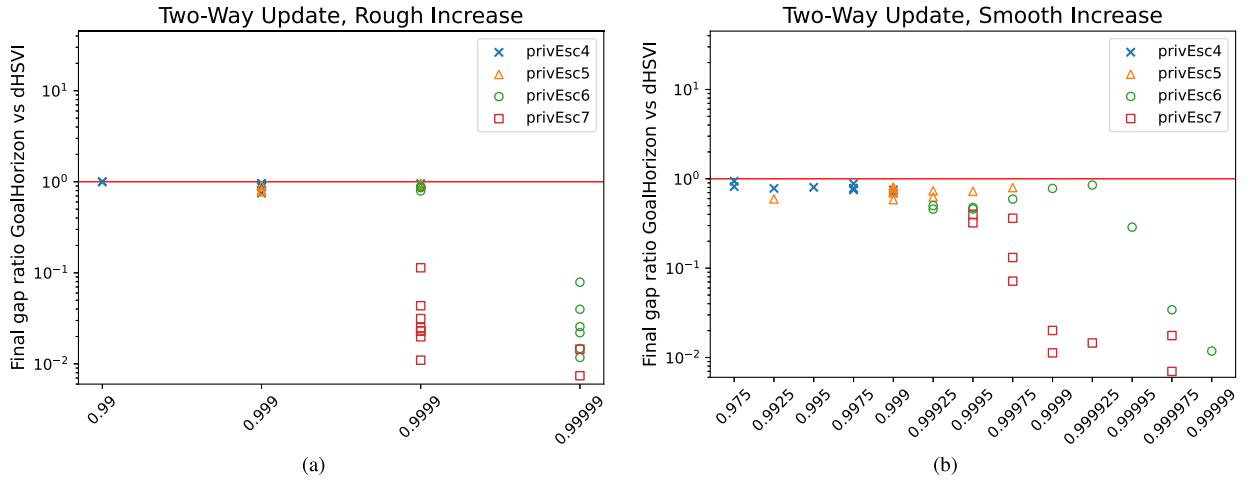


Fig. 8. Comparison of the quality of solutions found by GoalHorizon algorithm and the discounted HSVI (dHSVI) algorithm when evaluated under the undiscounted setting on the domain of privilege escalation games.

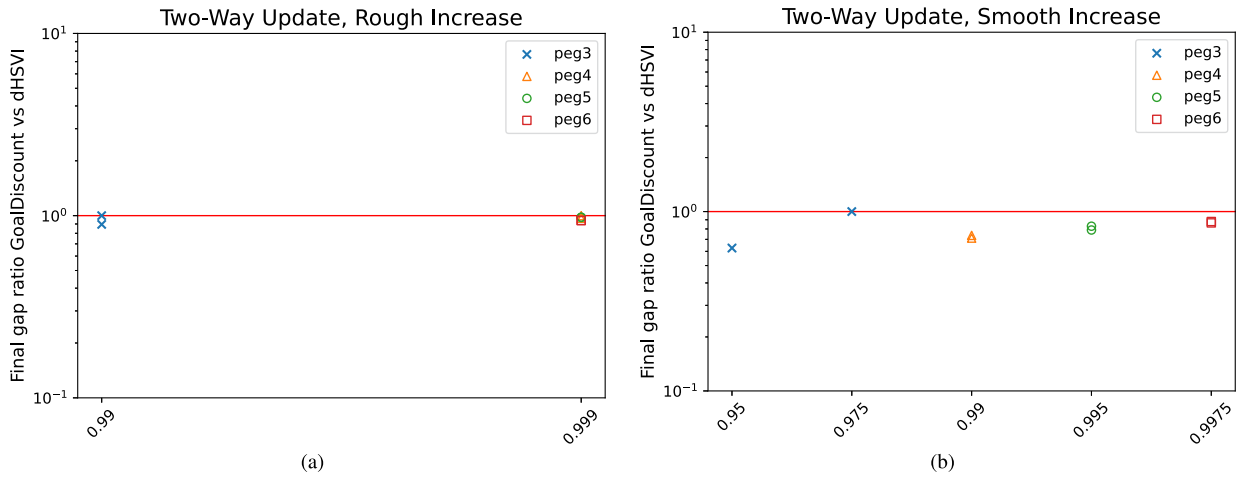


Fig. 9. Comparison of the quality of solutions found by the GoalDiscount algorithm and the discounted HSVI (dHSVI) algorithm when evaluated under the undiscounted setting on the domain of pursuit-evasion games.

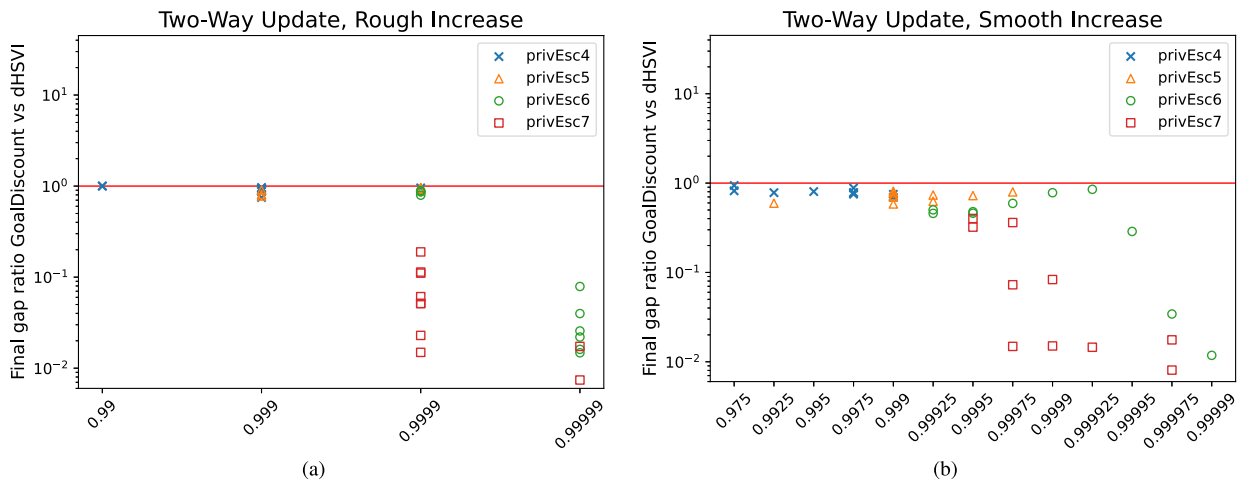


Fig. 10. Comparison of the quality of solutions found by the GoalDiscount algorithm and the discounted HSVI (dHSVI) algorithm when evaluated under the undiscounted setting on the domain of privilege escalation games.

6.4. Scalability with guaranteed solution quality

As we showed in Section 6.3, the quality of the solutions provided by the discounted HSVI algorithm is indeed worse than of those provided by POSSPG algorithms (GoalHorizon and GoalDiscount). However, as noted in Section 5.2, we can take advantage of being able to compute undiscounted counterparts for each discounted alpha vector (for the lower bound) and point (for the upper bound) and modify the discounted HSVI algorithm to look for the solution with desired properties under the *undiscounted setting* (we introduced this modified algorithm in the beginning of this section as udHSVI — Algorithm 5). The only requirement is to have the “right” discount factor for which it is achievable to find such a solution. For instances successfully solved by the GoalDiscount algorithm, we know the value of such “right” discount factor γ . For instances that the GoalDiscount algorithm was not able to solve (due to exceeding the time limit or running out of memory), we assume the largest value of discount factor γ with started computation as this is the best estimate on the “right” discount factor γ and both GoalDiscount and udHSVI are guaranteed to achieve similar solution quality for such discount factor. This allows us to compare proposed POSSPG algorithms with the discounted HSVI algorithm (udHSVI) in terms of scalability in situations when the provided solutions have similar quality.¹¹

We cannot directly compare the computation times of the proposed *goal algorithms* (GoalHorizon, GoalDiscount) to the *baseline* (udHSVI) in situations where one of the algorithms fails to find the solution in the given time limit. However, we can still compare them based on the percentage of unsolved games. To be able to connect results for situations where one of the algorithms failed with those where both algorithms succeeded, we visualize obtained scalability results using dual charts. On the primary y-axis (the left one) is the ratio $\frac{\text{goal algorithm}}{\text{baseline}}$ of the times needed to find a solution. This ratio is visualized as a scatter plot in the upper part of the chart. Whenever a point ends up being below the red 1-line, it means that the particular goal algorithm is better than the baseline (in this case, udHSVI). Situations where one of the compared algorithms was not able to find the solution in the given time limit are omitted from the scatter plot. On the secondary y-axis (the right one) is the respective percentage of games for which the individual algorithms were unable to find a solution in the given time limit, and this is visualized as a grouped bar plot in the bottom part of the charts.

6.4.1. GoalHorizon vs udHSVI

When comparing the GoalHorizon algorithm with udHSVI in terms of algorithm scalability on the domain of pursuit-evasion games (Fig. 11), we can see that the GoalHorizon algorithm keeps up with udHSVI on easier instances, but with the increased complexity of problem instances, udHSVI scales better and thus the GoalHorizon is not the best choice for pursuit-evasion games in terms of scalability. This can be clearly seen for instances of *peg6* for which the GoalHorizon algorithm cannot solve any instance regardless of the setting. We think that this is mostly due to no additional information (besides whether the game ended or still continues) provided to the partially informed player during the course of pursuit-evasion games. Thus, there can be not enough shared information between individual k -cutoff games to have similarities among the corresponding computed strategies (i.e., the strategy computed for $k + 1$ -cutoff game highly differs from the one computed for the k -cutoff game).

Another thing we can notice is that the GoalHorizon algorithm achieves better results (compared to udHSVI) under the *rough increase* setting. The reason for the better performance under this particular setting is that the udHSVI algorithm solves the same problem instances but with a higher discount factor γ (due to the fact that these values of γ were obtained by the GoalDiscount algorithm assuming *rough increase* setting). This can be seen when comparing values of discount factor gamma for fixed update setting based on the *increase* setting (e.g., comparison two-way update, *rough increase* vs. two-way update, *smooth increase*).

For example, let us look at the two *peg4* instances under the two-way update setting. When comparing corresponding figures (Fig. 11a and Fig. 11b), we can observe that, for the *rough increase* setting, we obtained value of discount factor $\gamma = 0.999$ for both of these *peg4* instances. However, under the *smooth increase* setting, it appeared that it is sufficient to solve both of them with the value of the discount factor $\gamma = 0.99$. Since the computational complexity of HSVI-based algorithms directly increases with the value of the discount factor γ and only udHSVI assumes discount factor (i.e., run times of the GoalHorizon algorithm does not rely on the discount factor γ and are identical for both *rough increase* and *smooth increase* setting) it is expected to see results to be more in favor of the udHSVI algorithm under the *smooth increase* setting (assuming fixed update setting). Better scalability results of udHSVI under the *smooth increase* setting are expected in general since *smooth increase* setting can only lead to smaller values of the discount factor γ and thus decreased computational complexity (the same instances will be solved for a smaller value of discount factor γ under the *smooth increase* setting).

When moved to the domain of privilege escalation games, the situation gets completely different (see Fig. 12). Based on the individual chart, we can see that the GoalHorizon algorithm is still capable of keeping up with udHSVI for smaller discount factors, but this time, as the value of the discount factor increases, it outperforms udHSVI by a significant margin.

This is further highlighted by the percentage of unsolved instances for larger discount factors where we can see that udHSVI struggles to solve the games within the time limit while the GoalHorizon algorithm is able to solve almost all of them. This can be clearly seen for values of discount factor $\gamma \geq 0.99995$. For these values, the udHSVI algorithm does not solve any of the instances, while the GoalHorizon algorithm is capable of solving all of them. Furthermore, on the corresponding instances (namely *privEsc6* and *privEsc7* instances), the GoalHorizon algorithm does not get even close to the time limit. In terms of actual runtimes, the GoalHorizon algorithm is capable of finding solutions for *privEsc6* instances in 417–1663 seconds and for *privEsc7* instances in

¹¹ We provide the scalability comparison to the original discounted HSVI algorithm (dHSVI) in the appendix — see Appendix B.3.

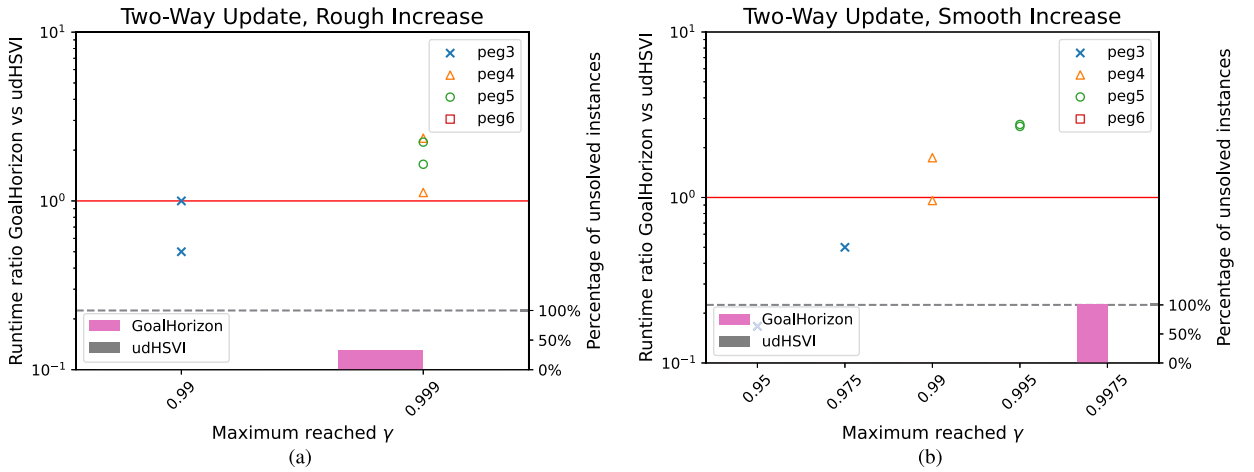


Fig. 11. Scalability comparison between the GoalHorizon algorithm and the udHSVI algorithm on the domain of pursuit-evasion games.

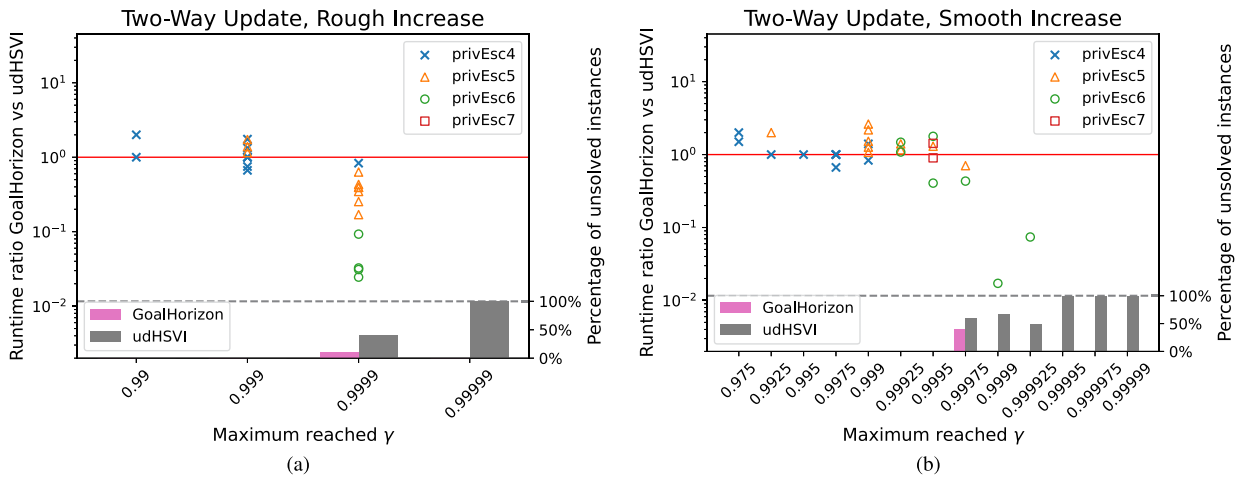


Fig. 12. Scalability comparison between the GoalHorizon algorithm and the udHSVI algorithm on the domain of privilege escalation games.

3455–24477 seconds. If the GoalHorizon algorithm did not find the solution (instances of *privEsc7*) in the given time limit (Fig. 12a — $\gamma = 0.9999$, and Fig. 12b — $\gamma = 0.99975$), it is always the case that udHSVI was not able to find a solution in the given time limit either. We think that the better performance of the GoalHorizon algorithm on the privilege escalation domain compared to the pursuit-evasion domain is due to more information provided to the partially informed player (the defender) during the course of the game (after a successful attack on any layer, the defender knows the set of possible targets of that attack which helps with the estimation of possible attack targets on the following layer) and thus more structured belief space.

6.4.2. GoalDiscount vs udHSVI

For the GoalDiscount algorithm on the pursuit-evasion domain (Fig. 13), we can see that, unlike the GoalHorizon algorithm, it scales significantly better with the increased discount factor, to the point it is able to match or even outperform udHSVI. This is mostly visible for situations where the maximum reached discount factor γ was obtained by the GoalDiscount algorithm under the rough increase computational setting (Fig. 13a). This is mostly due to the fact that udHSVI has to operate with higher discount factors compared to the smooth increase. Because of this, we can also spot the same trend of better scalability with a higher discount factor for situations where the maximum reached discount factor γ was obtained under the smooth increase setting (Fig. 13b) but it is not that pronounced and the GoalDiscount algorithm has roughly similar scalability as udHSVI.¹²

¹² Unsolved instances for the GoalDiscount algorithm (Fig. 13b) are not due to exceeding the time limit but running out of memory due to two-way update computational setting and thus using a larger number of alpha vectors and points to represent corresponding bounds during the computation. The algorithm was able to find the solution with a memory limit increased to 256 GB with corresponding runtime ratios $\frac{\text{goal algorithm}}{\text{baseline}}$ being 1.57 and 1.33 respectively.

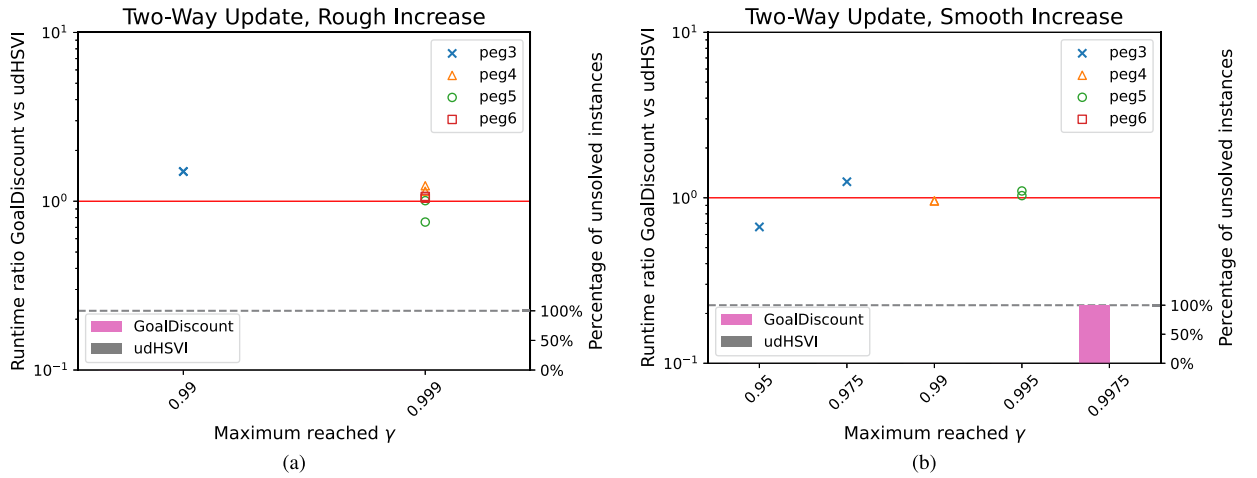


Fig. 13. Scalability comparison between the GoalDiscount algorithm and the udHSVI algorithm on the domain of pursuit-evasion games.

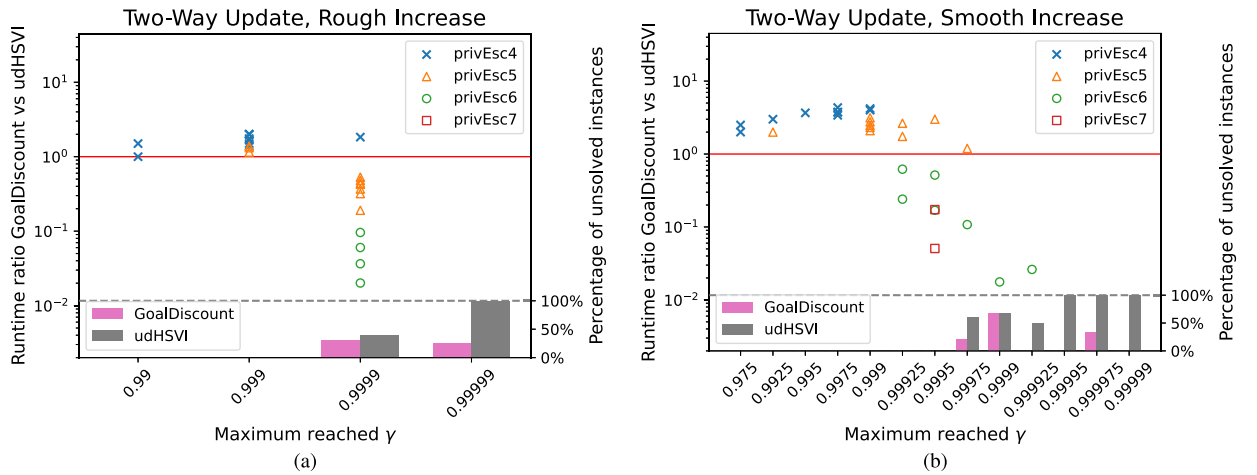


Fig. 14. Scalability comparison between the GoalDiscount algorithm and the udHSVI algorithm on the domain of privilege escalation games.

Better scalability with increased discount factor for the GoalDiscount algorithm compared to udHSVI is further highlighted on the privilege escalation domain (Fig. 14).¹³ Similarly to the comparison on pursuit-evasion games, we can observe that the GoalDiscount algorithm compares to udHSVI better when considering the rough increase computational setting. The results for smooth increase computational setting (Fig. 14b) then allow us to distribute the results to more values of discount factor γ and better observe how both algorithms perform. We can see that there are some instances that the GoalDiscount algorithm was not able to solve in the given time limit. However, these instances do not require the largest values of discount factor γ (assuming smooth increase computational setting) which was hidden for the rough increase computational setting and the inability to solve them in the given time limit is caused rather by being larger (all of them are instances of *privEsc7*) and more complex rather than requiring a larger discount factor.¹⁴ For the largest value of the discount factor γ we can observe that the GoalDiscount algorithm solves all the instances while udHSVI is unable to solve any of them. Furthermore, the GoalDiscount algorithm solves these instances with a large portion of the given time limit being unused. If assuming one-way update, smooth increase computational setting combination, the instances requiring the largest value of discount factor $\gamma = 0.999975$ were all solved by the GoalDiscount algorithm under 8600 seconds.

6.5. Experiments summary

In Section 6.3, we showed that both presented goal algorithms (GoalHorizon and GoalDiscount) can outperform the discounted HSVI algorithm on both testing domains in terms of the solution quality due to the fact that the traditional discounted HSVI

¹³ However, compared to the results of the GoalHorizon algorithm (Fig. 12) we can observe that the performance of the GoalDiscount algorithm is slightly worse in terms of unsolved instances.

¹⁴ Note that the same instances were not solved by udHSVI either.

Table 1

Best achieved runtimes for all game instances ignoring the computational setting.

Instance	dHSVI (s)	udHSVI (s)	GoalHorizon (s)	GoalDiscount (s)
peg3_0	1	2	1	3
peg3_1	1	2	2	3
peg4_0	14	34	61	42
peg4_1	15	41	46	45
peg5_0	117	239	812	291
peg5_1	95	250	717	177
peg6_0	5461	8064	NA	7709
peg6_1	6479	8215	NA	8263
privEsc4_01	1	3	2	5
privEsc4_02	2	5	5	7
privEsc4_03	3	6	5	11
privEsc4_04	1	3	3	6
privEsc4_05	2	3	4	6
privEsc4_06	2	4	6	8
privEsc4_07	2	4	4	7
privEsc4_08	1	2	4	3
privEsc4_09	1	2	3	3
privEsc4_10	1	3	3	6
privEsc5_01	7	13	13	18
privEsc5_02	15	43	50	51
privEsc5_03	15	25	30	44
privEsc5_04	11	20	20	35
privEsc5_05	4	9	14	15
privEsc5_06	40	88	42	86
privEsc5_07	12	21	44	32
privEsc5_08	14	22	32	29
privEsc5_09	9	19	21	26
privEsc5_10	19	33	59	69
privEsc6_01	249	781	611	188
privEsc6_02	5029	9973	639	261
privEsc6_03	83	197	291	122
privEsc6_04	13058	27191	417	514
privEsc6_05	NA	NA	1315	5006
privEsc6_06	1925	5363	1286	578
privEsc6_07	30260	NA	1005	404
privEsc6_08	544	1837	591	311
privEsc6_09	NA	NA	1188	1857
privEsc6_10	135	412	718	212
privEsc7_01	2194	7793	4563	395
privEsc7_02	NA	NA	31589	9860
privEsc7_03	NA	NA	3455	4313
privEsc7_04	3023	10724	10298	1854
privEsc7_05	NA	NA	NA	NA
privEsc7_06	NA	NA	14064	21732
privEsc7_07	NA	NA	4705	NA
privEsc7_08	15038	NA	7137	2240
privEsc7_09	NA	NA	21585	NA
privEsc7_10	NA	NA	14384	NA

algorithm does not guarantee to produce solutions with desired properties under the undiscounted setting. And even though we can modify the discounted HSVI algorithm in a way to make it produce solutions that are guaranteed to have desired properties under undiscounted setting as well,¹⁵ it still cannot compete with presented goal algorithms most of the time. The most important reason for that is that the used modification itself is not sufficient, and we also have to know the right discount factor γ^* to be able to get solutions with guaranteed desired undiscounted properties using the modified discounted HSVI algorithm (udHSVI).¹⁶ This is not an issue for the goal algorithms since they either iteratively increase the discount factor γ until the solution has desired undiscounted properties (GoalDiscount algorithm) or does not require a discount factor at all (GoalHorizon algorithm). Furthermore, even if

¹⁵ This is possible thanks to the principle of undiscounted counterparts for alpha vectors (lower bound) and points (upper bound) introduced for the GoalDiscount algorithm.

¹⁶ During the experiments we used maximum values of discount factor γ needed by the proposed GoalDiscount algorithm.

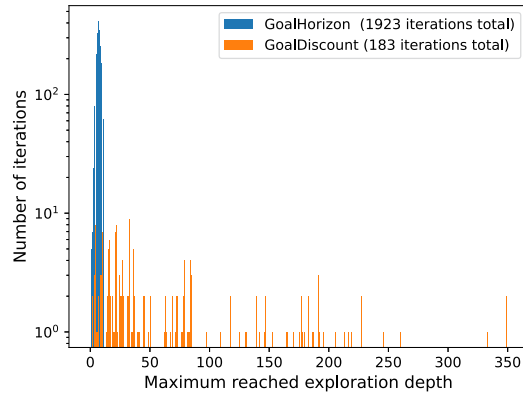


Fig. 15. Typical example of comparisons of depth histograms of the GoalHorizon algorithm and the GoalDiscount algorithm on domain of pursuit evasion games (here instance peg5_1).

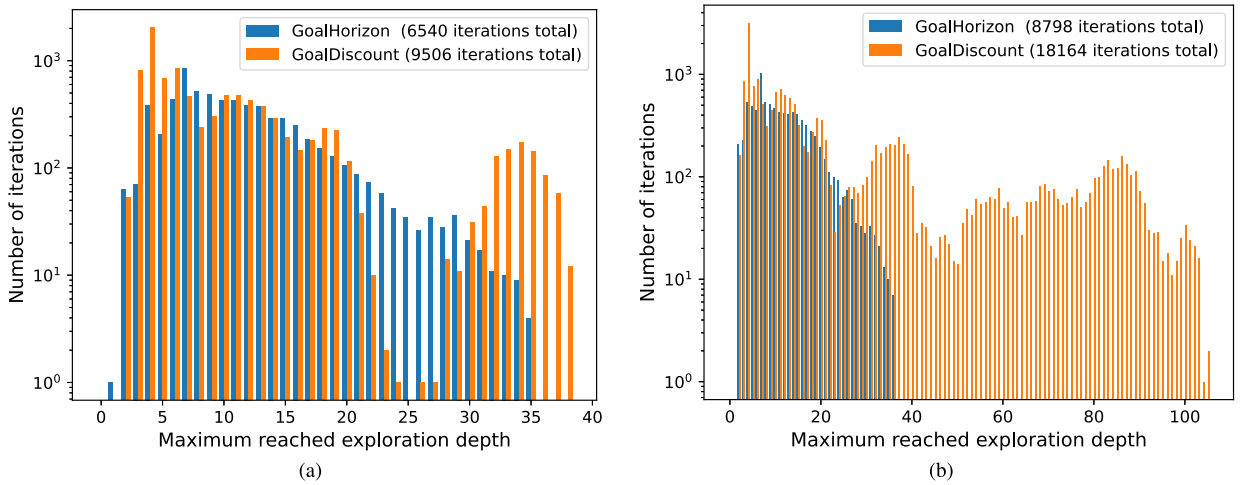


Fig. 16. Typical examples of comparisons of depth histograms of the GoalHorizon algorithm and the GoalDiscount algorithm on domain of privilege escalation games. Shown results are for instances of privEsc7 namely (a) privEsc7_08 and (b) privEsc7_06.

the right discount factor γ^* is known, the goal algorithms shine in situations where it gets closer to 1 and they are able to outperform `udHSVI` in terms of algorithm scalability (especially on more complex domains like privilege escalation games).

To further highlight the differences between algorithms in terms of time needed for finding a solution, we present the best-achieved runtimes for individual games achieved by each algorithm, disregarding the computational setting (Table 1).

We can see that modifying the discounted HSVI algorithm (`dHSVI`) to guarantee that produced solutions will have desired properties under the undiscounted setting (`udHSVI`) increases the best runtimes roughly 2 or 3 times. These increased runtimes then show that using presented goal algorithms (in situations where we request properties of produced solutions under undiscounted setting) is at the worst equivalent in terms of runtime to this approach.¹⁷

When comparing presented goal algorithms, we can observe the following. The GoalHorizon algorithm is quite stable in solving POSSPGs and is capable of solving the highest number of instances out of all algorithms. On the other hand, the GoalDiscount algorithm is usually faster than the GoalHorizon algorithm on the larger instances. However, there are some instances (namely instances of `privEsc7`) that it cannot solve in the given runtime while the GoalHorizon algorithm is capable of doing so.

We were not able to identify the specific reasons behind this behavior. Based on the conducted investigation we believe that it is a result of a combination of the following two reasons. The first one is the complexity of individual instances that is tightly connected to the size of the belief space and the portion of it required to be visited in order to be able to solve a particular instance. The second one is the way how each of the goal algorithms works in terms of belief space exploration. Because of the limited depth, the GoalHorizon algorithm is forced to work only with beliefs reachable before reaching the limited horizon and thus perform rather wider explorations, making its behavior similar to breadth-first search. On the other hand, the GoalDiscount algorithm explores rather deeper than wider due to the usage of discount factor, and its behavior can be viewed as a depth-first search. This is also supported by looking at the comparison of depth (maximum depths of individual iterations) histograms of the GoalHorizon

¹⁷ Note that this assumes we know the right value of discount factor γ^* for which we can find a solution with desired undiscounted properties using `udHSVI`.

algorithm and the GoalDiscount algorithm. With very few exceptions, we can observe that the GoalDiscount algorithm usually requires performing deeper iterations for both the pursuit-evasion domain (see Fig. 15) and the privilege escalation one (see Fig. 16). We can also notice that deeper iterations do not necessarily mean a higher number of performed iterations, which supports our claim about the impact of belief space structure on a solution for each individual instance. Identifying how the combination of these two conditions affects the solution process and in which situations will be one of the presented goal algorithms better than the other one is a highly challenging problem, and it is closely tied to similar problems from other areas of the game theory field making it a good candidate for follow up research.

7. Conclusion

Solving partially observable stochastic games (POSG) is a challenging task. The most common objective that agents optimize is usually based on a discounted sum of rewards that gives lower weight to the costs and rewards obtained in the later stages of the problem. We focus on the objective based on an undiscounted sum for problems with an indefinite horizon and present a formulation for such scenarios using partially observable stochastic shortest path games (POSSPGs).

We introduce two novel algorithms based on the heuristic search value iteration (HSVI) algorithm that iteratively solve sequences of easier-to-solve approximations of the game using fundamentally different approaches for constructing the sequences: (1) in GoalHorizon, the game approximations are based on a limited number of turns in which players can change their actions, (2) in GoalDiscount, the game approximations are constructed using an increasing discount factor and we show that we can use such an approach to obtain a solution with guarantees under the undiscounted setting. We show experimentally (Section 6) that both our methods for solving POSSPGs can approximate the value of the game for the undiscounted objective for non-trivial games in a time comparable with solving a discounted variant of the game. Moreover, by using approximations based on discounted variants of the games, we demonstrate that simply solving a discounted version of the game (even with a large-enough discount factor) yields strategies that do not necessarily reach the same value in the undiscounted variant of the game compared to our algorithms.

Considering the proposed algorithms and their experimental evaluation, a good candidate for future work would be analyzing the exact impact of instance complexity (belief space structure) and belief space exploration on the behavior of both GoalHorizon algorithm and GoalDiscount algorithm. Such analysis can open ways to further improve the scalability of these algorithms and potentially extend their applicability to more general settings. It might also help with problems from other areas of the game theory field that deal with game complexity and belief selection. Another possible direction for future work could be further theoretical research leading to unifying our proposed algorithms with ideas on computing value along a sequence of distributions over the naturals of decreasing patience [45].

CRedit authorship contribution statement

Petr Tomášek: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Formal analysis. **Karel Horák:** Writing – review & editing, Writing – original draft, Validation. **Branislav Božanský:** Writing – review & editing, Writing – original draft, Validation, Supervision.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Petr Tomasek reports financial support was provided by Czech Science Foundation (no. 19-24384Y, GA22-26655S). Petr Tomasek reports financial support was provided by Ministry of Education Youth and Sports (no. CZ.02.1.01/0.0/0.0/16 019/0000765) of the Czech Republic. Petr Tomasek reports financial support was provided by US Army Combat Capabilities Development Command Army Research Laboratory (no. W911NF-13-2-0045). Branislav Bosansky reports financial support was provided by Czech Science Foundation (no. 19-24384Y, GA22-26655S). If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This research was supported by the Czech Science Foundation (no. 19-24384Y, GA22-26655S), by the OP VVV MEYS funded project CZ.02.1.01/0.0/0.0/16 019/0000765 “Research Center for Informatics”, and by the Combat Capabilities Development Command Army Research Laboratory and was accomplished under Cooperative Agreement Number W911NF-13-2-0045 (ARL Cyber Security CRA). The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the Combat Capabilities Development Command Army Research Laboratory or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes not withstanding any copyright notation here on.

Appendix A. Proofs

A.1. Proof of Theorem 1

Proposition 2. Let $b^{(t-1)} \in \Delta(S)$ and $\pi_1^{(t)}, \pi_2^{(t)}$ be Nash equilibrium strategies of P1 and P2 in $[H\bar{V}](b^{(t-1)})$ and $[H\underline{V}](b^{(t-1)})$, respectively. Assume that all beliefs $\tau(b^{(t-1)}, a_1, \pi_2^{(t)}, o)$ reachable when following $(\pi_1^{(t)}, \pi_2^{(t)})$ have negative excess gap $\text{excess}_\tau(\tau(b^{(t-1)}, a_1, \pi_2^{(t)}, o)) \leq 0$. Then after performing the point-based update in $b^{(t-1)}$ we have $\text{excess}_{t-1}(b') \leq 0$ for every b' such that $\|b^{(t-1)} - b'\| \leq \varepsilon/2H\delta_H$.

Proof. Let $u_{V,b}(\pi_1, \pi_2)$ denote the utility in stage game $[HV](b)$ when the players use stage strategies (π_1, π_2) . Let $(\underline{\pi}_1^{(t)}, \underline{\pi}_2^{(t)})$ and $(\bar{\pi}_1^{(t)}, \bar{\pi}_2^{(t)})$ be Nash equilibrium strategies in stage games $[H\underline{V}_{H-t}](b^{(t-1)})$ and $[H\bar{V}_{H-t}](b^{(t-1)})$, respectively. By similar argument to the proof of [22, Lemma 4], we have

$$\begin{aligned} [H\bar{V}_{H-t}](b^{(t-1)}) - [H\underline{V}_{H-t}](b^{(t-1)}) &= \\ &= u_{\bar{V}_{H-t}, b^{(t-1)}}(\bar{\pi}_1^{(t)}, \bar{\pi}_2^{(t)}) - u_{\underline{V}_{H-t}, b^{(t-1)}}(\underline{\pi}_1^{(t)}, \underline{\pi}_2^{(t)}) \\ &\leq u_{\bar{V}_{H-t}, b^{(t-1)}}(\bar{\pi}_1^{(t)}, \bar{\pi}_2^{(t)}) - u_{\underline{V}_{H-t}, b^{(t-1)}}(\bar{\pi}_1^{(t)}, \bar{\pi}_2^{(t)}) \\ &= \sum_{a_1, o} \mathbb{P}_{b^{(t-1)}, \bar{\pi}_1^{(t)}, \bar{\pi}_2^{(t)}}[a_1, o] [\bar{V}_{H-t}(\tau(b^{(t-1)}, a_1, \bar{\pi}_2^{(t)}, o)) - \\ &\quad \underline{V}_{H-t}(\tau(b^{(t-1)}, a_1, \bar{\pi}_2^{(t)}, o))] \\ &\leq \rho(t) = \rho(t-1) - \varepsilon/H. \end{aligned}$$

The point-based update in $b^{(t-1)}$ hence ensures that $\bar{V}_{H-t+1}(b^{(t-1)}) - \underline{V}_{H-t+1}(b^{(t-1)}) \leq \rho(t-1) - \varepsilon/H$. Since $\underline{V}_{H-t+1}$ and $\bar{V}_{H-t+1}$ are δ_H -Lipschitz continuous (observe that $\delta_H \geq \delta_{H'}$ for every $0 \leq H' \leq H$), we have that $\bar{V}_{H-t+1}(b^{(t-1)}) - \underline{V}_{H-t+1}(b^{(t-1)}) \leq \rho(t-1)$ for every belief b' such that $\|b' - b^{(t-1)}\| \leq \varepsilon/2H\delta_H$. $\square$

Theorem 1. For every $\varepsilon > 0$ the FH-HSVI algorithm for solving finite-horizon OS-POSGs terminates. When it does so, it approximates the value of the game within ε interval.

Proof. In each iteration, the FH-HSVI algorithm (Algorithm 2) performs a forward exploration until it reaches belief $b^{(t)}$ with negative excess gap $\text{excess}_{b^{(t)}} \leq 0$ (line 6). Since there are no further costs or rewards issued after reaching the maximum depth H , the excess gap is $\text{excess}_H(b^{(H)}) = 0$ for every belief $b^{(H)}$ in depth H . This means that the forward exploration (recursive procedure `Explore`) is guaranteed to always terminate.

Let us assume sets Ψ_t of all beliefs with a negative excess gap at time t . Whenever the procedure `Explore` terminates at recursion level t (i.e., the condition on line 6 is satisfied), it is guaranteed that corresponding belief $b^{(t)}$ and all beliefs b' such that $\|b' - b^{(t-1)}\| \leq \varepsilon/2H\delta_H$ have negative excess gap (Proposition 2) and can be added to Ψ_t . To prove that the whole algorithm terminates, it is sufficient to show that expanding sets Ψ_t guarantees that eventually $\Psi_t = \Delta(S)$ (unless the desired precision ε is achieved beforehand — line 3) and the number of required expansions is finite.

In every iteration, Ψ_t is expanded by one explored belief and all beliefs in its $\varepsilon/2H\delta_H$ -neighborhood. The number of such expansions is finite. The problem of finding the maximum set of explored beliefs to ensure $\Psi_t = \Delta(S)$ can be seen as a *hypersphere packing* (more precisely, a higher dimensional version of the sphere packing — [19]) filling the belief space using non-overlapping hyperspheres of radius $\varepsilon/4H\delta_H$. When no hypersphere can be further inserted, there is no belief with a positive excess gap. Hence, the desired precision was achieved in the whole belief space. $\square$

A.2. Proof of Proposition 1

Proposition 1. Let G be a POSSPG with value function V^* and let $b \in \Delta(S)$. Given a threshold t , determining whether $V^*(b) > t$ is undecidable.

Proof. The problem of determining whether a policy with expected value greater than a given threshold t exists is undecidable for discounted POMDPs [34, Section 4.3]. This problem is analogous to deciding whether the value $V^*(b_{init}) > t$. We prove the undecidability of the exact value problem in POSSPGs by outlining a reduction that allows us to reason about the exact value of a discounted POMDP using the exact value of a corresponding POSSPG. The reduction is inspired by [9, Section 5].

Let us assume a discounted POMDP $\Pi = (S, A, T, R, O, \gamma)$. First, let us modify the reward function of Π by subtracting a constant value $C > \max_{s,a} R(s, a)$ from every reward: $R'(s, a) \leftarrow R(s, a) - C$. It is easy to verify that the payoff associated with every policy in $\Pi' = (S, A, T, R', O, \gamma)$ decreases by $C/(1 - \gamma)$ compared to the original problem Π , and that all rewards of the modified problem satisfy $R'(s, a) < 0$.

Next, let us introduce an absorbing state g (i.e., $S' \leftarrow S \cup \{g\}$) such that $R(g, a) = 0$ for every action $a \in A$ and $T'(g, a, g) = 1$. Furthermore, let us introduce an observation o_{end} that notifies the agent about reaching g . Let us modify the transition function in a way that the world transitions to g with probability of $1 - \gamma$ in every step, while with probability γ it continues as before:

$$\begin{aligned} T'(s, a, o_{end}, g) &= 1 - \gamma & \forall s \in S \setminus \{g\}, a \in A \\ T'(s, a, o, s') &= \gamma T(s, a, o, s') & \forall s, s' \in S \setminus \{g\}, a \in A, o \in O \\ T'(s, a, o, s') &= T(s, a, o, s') & \text{otherwise} . \end{aligned}$$

Using a standard construction, we can see that the *undiscounted* payoffs in a POMDP $\Pi'' = (S \cup \{g\}, A, T', R', O \cup \{o_{end}\})$ are identical to the *discounted* payoffs in Π' .

We have now arrived at an undiscounted POMDP Π'' such that the value $V_{\Pi''}^*(b)$ of Π'' satisfies $V_{\Pi''}^*(b) = V_{\Pi}^*(b) - C/(1 - \gamma)$ (where $V_{\Pi}^*(b)$ is the value of the original discounted POMDP Π). To represent this POMDP as a POSSPG it suffices to introduce the perfectly informed player 2 who cannot influence neither transitions nor rewards (e.g., by having only a single action a_{noop} such that $R_{POSSPG}(s, a, a_{noop}) = R'(s, a)$ and $T_{POSSPG}(s, a, a_{noop}, o, s') = T'(s, a, o, s')$). It remains to verify that the POSSPG assumptions hold:

- The game ends upon reaching state g and no further rewards/costs are incurred afterwards since g is always reached in the limit and $R(g, \cdot) = 0$.
- All rewards in the game are negative (with the exception of the rewards received in the goal state g). No zero cycles are therefore present.
- The payoff of every policy of player 1 in the POSSPG is identical to the discounted payoff in Π except for the additive constant $-C/(1 - \gamma)$. Therefore the utilities as considered in the assumptions 3 and 4 are finite.

We have proved that deciding whether the exact value of the constructed POSSPG is greater than $t - C/(1 - \gamma)$ would allow us to decide whether the value of Π is greater than t . Thus this is undecidable for POSSPGs which concludes the proof. $\square$

A.3. Proof of Lemma 2

Lemma 3. For every $k \geq 0$ set $\mathcal{F}_k$ is convex.

Proof. To prove this claim, we will use induction on k and the recursive definition of sets $\mathcal{F}_k$ (Definition $\mathcal{F}_k$). For the means of the proof, it is sufficient to simplify (11) as

$$\mathcal{F}_k = \{\kappa x \mid \forall \kappa \in [0, 1], x \in \mathcal{F}_{k-1}\} = [0, 1]\mathcal{F}_{k-1}. \quad (\text{A.1})$$

From Definition $\mathcal{F}_k$, we have $\mathcal{F}_0 = \{(\bar{\Delta}_0 = \bar{y}_0^+ - \underline{y}_0, \bar{y}_0^+, \underline{y}_0)\}$. Therefore, the base case $k = 0$ clearly holds, as $\mathcal{F}_0$ contains only one element. Let $k \geq 1$ and, for the sake of induction, assume that the statement from Lemma 3 holds for every $k - 1$. To prove that it also works for every k we will use proof from [35], that we verified to be valid and correct. Let us have two elements from $\mathcal{F}_k$ being $x = \kappa a$ and $y = \kappa' b$ where $a, b \in \mathcal{F}_{k-1}$ and $\kappa, \kappa' \in [0, 1]$. To prove that $\mathcal{F}_k$ is convex we need to show that $\iota x + (1 - \iota)y \in \mathcal{F}_k$ for all $\iota \in [0, 1]$.

Let us set $\mu = \kappa \iota + \kappa'(1 - \iota)$ and $\nu = \frac{\kappa \iota}{\kappa \iota + \kappa'(1 - \iota)}$, where both μ and ν are guaranteed to be in $[0, 1]$. Using constants μ and ν we can get $\iota x + (1 - \iota)y = \nu \mu a + (1 - \nu) \mu b$. Using the inductive assumption, set $\mu \mathcal{F}_{k-1}$ is convex. Therefore, it holds $\nu \mu a + (1 - \nu) \mu b \in \mu \mathcal{F}_{k-1}$ for all $\nu \in [0, 1]$. Thus, it also holds $\iota x + (1 - \iota)y = \nu \mu a + (1 - \nu) \mu b \in [0, 1]\mathcal{F}_{k-1} = \mathcal{F}_k$. $\square$

Lemma 2. Let G be a POSSPG and let $k \geq 0$. Then for every point-based update (b, y_k, y_k^+) of value functions V^k and V_k^+ , respectively, there exists $(\bar{\Delta}_k, \bar{y}_k^+, \underline{y}_k) \in \mathcal{F}_k$ such that $y_k^+ - y_k \leq \Delta_k$, $y_k^+ \leq \bar{y}_k^+$ and $y_k \geq \underline{y}_k$. Furthermore, at any point of time, for every belief $b \in \Delta(S)$ there exists $(\bar{\Delta}_k, \bar{y}_k^+, \underline{y}_k) \in \mathcal{F}_k$ such that $\bar{V}_k^+(b) - \bar{V}_k^-(b) \leq \Delta_k$, $\bar{V}_k^+(b) \leq \bar{y}_k^+$ and $\bar{V}_k^-(b) \geq \underline{y}_k$.

Proof. We will prove the claim by induction on k . By definition of $\mathcal{F}_0$ the base case $k = 0$ clearly holds. For $k \geq 1$ we focus on the point-based updates $y = [H\bar{V}_k](b)$ and $y^+ = [H\bar{V}_k^+](b)$ performed by Algorithm 3 on lines 17 and 18.

Let $k \geq 1$ and $b \in \Delta(S)$ and, for the sake of induction, assume that the statement from the Lemma 2 holds for every $k - 1$. Let

$$\begin{aligned} y^+ &= [H\bar{V}_k^+](b) = u_b^{\bar{V}_{k-1}^+}(\pi_1^+, \pi_2^+) \\ &= \mathbb{E}_{b, \pi_1^+, \pi_2^+}[R(s, a_1, a_2)] + \sum_{a_1, o} \Pr_{b, \pi_1^+, \pi_2^+}[a_1, o] \bar{V}_{k-1}^+(\tau(b, a_1, \pi_2^+, o)) \\ y &= [H\bar{V}_k](b) = u_b^{\bar{V}_{k-1}}(\pi_1, \pi_2) \\ &= \mathbb{E}_{b, \pi_1, \pi_2}[R(s, a_1, a_2)] + \sum_{a_1, o} \Pr_{b, \pi_1, \pi_2}[a_1, o] \bar{V}_{k-1}(\tau(b, a_1, \pi_2, o)), \end{aligned}$$

where (π_1, π_2) and (π_1^+, π_2^+) are Nash equilibrium strategy profiles in stage games $[H\bar{V}_k](b)$ and $[H\bar{V}_k^+](b)$, respectively. By deviating from his/her prescribed Nash-equilibrium strategy, each player can only worsen their payoff. We thus have

$$y^+ = u_b^{\bar{V}^{k-1}}(\pi_1^+, \pi_2^+) \leq u_b^{\bar{V}^{k-1}}(\pi_1^+, \pi_2) = \mathbb{E}_{b, \pi_1^+, \pi_2} [R(s, a_1, a_2)] \\ + \sum_{a_1, o} \Pr_{b, \pi_1^+, \pi_2} [a_1, o] \bar{V}_{k-1}^+(\tau(b, a_1, \pi_2, o))$$

$$y = u_b^{\bar{V}^{k-1}}(\pi_1, \pi_2) \geq u_b^{\bar{V}^{k-1}}(\pi_1^+, \pi_2) = \mathbb{E}_{b, \pi_1^+, \pi_2} [R(s, a_1, a_2)] \\ + \sum_{a_1, o} \Pr_{b, \pi_1^+, \pi_2} [a_1, o] \bar{V}_{k-1}(\tau(b, a_1, \pi_2, o)).$$

Observe that observation o_{end} indicates reaching the goal state and the end of the game. In this case, no future costs are paid and $\bar{V}_{k-1}^+(\tau(b, a_1, \pi_2, o)) = \bar{V}_{k-1}(\tau(b, a_1, \pi_2, o)) = 0$. Hence

$$y^+ \leq \mathbb{E}_{b, \pi_1^+, \pi_2} [R(s, a_1, a_2)] \\ + \Pr_{b, \pi_1^+, \pi_2} [o \neq o_{end}] \sum_{\substack{a_1, o \\ o \neq o_{end}}} \Pr_{b, \pi_1^+, \pi_2} [a_1, o | o \neq o_{end}] \bar{V}_{k-1}^+(\tau(b, a_1, \pi_2, o)) \quad (\text{A.2})$$

$$y \geq \mathbb{E}_{b, \pi_1^+, \pi_2} [R(s, a_1, a_2)] \\ + \Pr_{b, \pi_1^+, \pi_2} [o \neq o_{end}] \sum_{\substack{a_1, o \\ o \neq o_{end}}} \Pr_{b, \pi_1^+, \pi_2} [a_1, o | o \neq o_{end}] \bar{V}_{k-1}(\tau(b, a_1, \pi_2, o)). \quad (\text{A.3})$$

Subtracting $y^+ - y$ we get

$$y^+ - y \leq \Pr_{b, \pi_1^+, \pi_2} [o \neq o_{end}] \sum_{\substack{a_1, o \\ o \neq o_{end}}} \Pr_{b, \pi_1^+, \pi_2} [a_1, o | o \neq o_{end}] [\bar{V}_{k-1}^+(\tau(b, a_1, \pi_2, o)) \\ - \bar{V}_{k-1}(\tau(b, a_1, \pi_2, o))]. \quad (\text{A.4})$$

Using the inductive assumption, we know that for every b' there exists $\langle \bar{\Delta}_{k-1}(b'), \bar{y}_{k-1}^+(b'), \underline{y}_{k-1}(b') \rangle \in \mathcal{F}_{k-1}$ such that $\bar{V}_k^+(b') - \bar{V}_k(b') \leq \bar{\Delta}_{k-1}(b')$, $\bar{V}_k^+(b') \leq \bar{y}_{k-1}^+(b')$ and $\bar{V}_k(b') \geq \underline{y}_{k-1}(b')$. Moreover, since $\mathcal{F}_{k-1}$ from Definition $\mathcal{F}_k$ is a convex set (see Lemma 3) we also have that there exists $\langle \bar{\Delta}_{k-1}(*), \bar{y}_{k-1}^+(*), \underline{y}_{k-1}(*) \rangle \in \mathcal{F}_{k-1}$ such that

$$\sum_{\substack{a_1, o \\ o \neq o_{end}}} \Pr_{b, \pi_1^+, \pi_2} [a_1, o | o \neq o_{end}] [\bar{V}_{k-1}^+(\tau(b, a_1, \pi_2, o)) - \bar{V}_{k-1}(\tau(b, a_1, \pi_2, o))] \leq \bar{\Delta}_{k-1}(*), \\ \sum_{\substack{a_1, o \\ o \neq o_{end}}} \Pr_{b, \pi_1^+, \pi_2} [a_1, o | o \neq o_{end}] \bar{V}_{k-1}^+(\tau(b, a_1, \pi_2, o)) \leq \bar{y}_{k-1}^+(*), \text{ and} \\ \sum_{\substack{a_1, o \\ o \neq o_{end}}} \Pr_{b, \pi_1^+, \pi_2} [a_1, o | o \neq o_{end}] \bar{V}_{k-1}(\tau(b, a_1, \pi_2, o)) \leq \underline{y}_{k-1}(*).$$

Since $\Pr_{b, \pi_1^+, \pi_2} [o \neq o_{end}] \in [0, 1]$, Equations (A.2)–(A.4) imply that

$$\exists \langle \bar{\Delta}_{k-1}(*), \bar{y}_{k-1}^+(*), \underline{y}_{k-1}(*) \rangle \in \mathcal{F}_{k-1}, \kappa \in [0, 1] : \quad y^+ - y \leq \kappa \bar{\Delta}_{k-1}(*) \\ y^+ \leq \bar{r} + \kappa \bar{y}_{k-1}^+(*), \\ y \geq \underline{r} + \kappa \underline{y}_{k-1}(*),$$

where $\bar{r} = \max R(\cdot)$ and $\underline{r} = \min R(\cdot)$ are maximum and minimum non-zero rewards, respectively. Observe that according to Definition $\mathcal{F}_k$, $\mathcal{F}_k$ contains points $\langle \kappa \bar{\Delta}_{k-1}, \bar{r} + \kappa \bar{y}_{k-1}^+, \underline{r} + \kappa \underline{y}_{k-1} \rangle$ for all possible values $\langle \bar{\Delta}_{k-1}, \bar{y}_{k-1}^+, \underline{y}_{k-1} \rangle \in \mathcal{F}_{k-1}$ and $\kappa \in [0, 1]$. This directly implies that there exists $\langle \bar{\Delta}_k, \bar{y}_k^+, \underline{y}_k \rangle \in \mathcal{F}_k$ such that $y^+ - y \leq \bar{\Delta}_k$, $y^+ \leq \bar{y}_k^+$ and $y \geq \underline{y}_k$.

The second part of the claim follows from the convexity of sets $\mathcal{F}_k$ and the fact that the value functions $\bar{V}_k^+$ and $\bar{V}_k$ are obtained as convex combinations of points $(b, y^+) \in \Upsilon_k^+$ and $(b, y) \in \Upsilon_k$. We thus have that for every belief b there exists $\langle \bar{\Delta}_k, \bar{y}_k^+, \underline{y}_k \rangle \in \mathcal{F}_k$ such that $\bar{V}_k^+(b) - \bar{V}_k(b) \leq \bar{\Delta}_k$, $\bar{V}_k^+(b) \leq \bar{y}_k^+$ and $\bar{V}_k(b) \geq \underline{y}_k$. $\square$

A.4. Proof of Theorem 2

To prove Theorem 2, we first define auxiliary set of feasible updates $\mathcal{F}_k^\square$ which dominates $\mathcal{F}_k$ in the sense that for every $(\bar{\Delta}_k, \bar{y}_k^+, \underline{y}_k) \in \mathcal{F}_k$ there exists $(\bar{\Delta}_k^\square, \bar{y}_k^{+\square}, \underline{y}_k^\square) \in \mathcal{F}_k^\square$ such that $\bar{\Delta}_k \leq \bar{\Delta}_k^\square$, $\bar{y}_k^+ \leq \bar{y}_k^{+\square}$ and $\underline{y}_k \geq \underline{y}_k^\square$.

Lemma 4. *Let*

$$\mathcal{F}_k^\square = \left\{ \left\langle \begin{array}{l} \max \left\{ \bar{r} + \kappa[(k-1)\bar{r} + \bar{y}_0^+], k\bar{r} + \bar{y}_0^+ \right\}, \\ \min \left\{ \underline{r} + \kappa[(k-1)\underline{r} + \underline{y}_0], k\underline{r} + \underline{y}_0 \right\} \end{array} \right\rangle \mid \forall \kappa \in [0, 1] \right\}. \quad (\text{A.5})$$

Then for every $k \in \{1, 2, \dots\}$ and every $(\bar{\Delta}_k, \bar{y}_k^+, \underline{y}_k) \in \mathcal{F}_k$ there exists $(\bar{\Delta}_k^\square, \bar{y}_k^{+\square}, \underline{y}_k^\square) \in \mathcal{F}_k^\square$ such that $\bar{\Delta}_k \leq \bar{\Delta}_k^\square$, $\bar{y}_k^+ \leq \bar{y}_k^{+\square}$ and $\underline{y}_k \geq \underline{y}_k^\square$.

Since we will be referring to the fact that for every element in set $x \in X$ there exists element in $y \in Y$ that dominates in the sense of the above lemma, we introduce the following shorthand.

Definition $\leq$. Let $P, Q \subseteq \mathbb{R}^3$. We say that $P \leq Q$ if and only if for every element $(\bar{\Delta}_p, \bar{y}_p^+, \underline{y}_p) \in P$ there exists $(\bar{\Delta}_q, \bar{y}_q^+, \underline{y}_q) \in Q$ such that $\bar{\Delta}_p \leq \bar{\Delta}_q$, $\bar{y}_p^+ \leq \bar{y}_q^+$ and $\underline{y}_p \geq \underline{y}_q$.

In the sense of the Lemma 4 the above definition implies $\mathcal{F}_k \leq \mathcal{F}_k^\square$. Observe that $\leq$ is a transitive relation, i.e., if $P \leq Q$ and $Q \leq R$ we also have $P \leq R$.

Proof. We prove the claim by induction on k . The base case $k = 1$ clearly holds since after expanding $\mathcal{F}_1^\square$ and substituting $k = 1$ equation (A.5) is identical to $\mathcal{F}_1$ from Definition $\mathcal{F}_k$ except for the maximization and minimization involved in $\mathcal{F}_1^\square$. This maximization and minimization only increases and decreases the second and third coordinates, thus preserving $\mathcal{F}_1 \leq \mathcal{F}_1^\square$. We will now use the induction on k to prove two claims:

1. We introduce a set $\mathcal{F}'_k$ and we show that $\mathcal{F}_k \leq \mathcal{F}'_k$.
2. We show that $\mathcal{F}'_k \leq \mathcal{F}_k^\square$.

Observe that due to the transitivity of $\leq$ proving these two claims for given k implies that $\mathcal{F}_k \leq \mathcal{F}_k^\square$. We will be proving both claims simultaneously, i.e., we will be assuming that both claims hold for all $k' < k$ when proving their validity for given k .

Proof of claim 1: Let us introduce set $\mathcal{F}'_k$ as follows

$$\mathcal{F}'_k = \left\{ \langle \kappa' \bar{\Delta}_{k-1}^\square, \bar{r} + \kappa' \bar{y}_{k-1}^{+\square}, \underline{r} + \kappa' \underline{y}_{k-1}^\square \rangle \mid \forall \kappa' \in [0, 1], \langle \bar{\Delta}_{k-1}^\square, \bar{y}_{k-1}^{+\square}, \underline{y}_{k-1}^\square \rangle \in \mathcal{F}_{k-1}^\square \right\}. \quad (\text{A.6})$$

Observe that the definition of $\mathcal{F}'_k$ is identical to definition of $\mathcal{F}_k$ (see equation (11)) except for quantifying over $\langle \bar{\Delta}_{k-1}^\square, \bar{y}_{k-1}^{+\square}, \underline{y}_{k-1}^\square \rangle \in \mathcal{F}_{k-1}^\square$ instead of $\langle \bar{\Delta}_{k-1}, \bar{y}_{k-1}^+, \underline{y}_{k-1} \rangle \in \mathcal{F}_{k-1}$. It is easy to see that the induction assumption $\mathcal{F}_{k-1} \leq \mathcal{F}_{k-1}^\square$ implies $\mathcal{F}_k \leq \mathcal{F}'_k$:

Let $(\bar{\Delta}_k, \bar{y}_k^+, \underline{y}_k) \in \mathcal{F}_k$ be arbitrary. According to Definition $\mathcal{F}_k$, there exists $\kappa \in [0, 1]$ and $(\bar{\Delta}_{k-1}, \bar{y}_{k-1}^+, \underline{y}_{k-1}) \in \mathcal{F}_{k-1}$ such that $\bar{\Delta}_k = \kappa \bar{\Delta}_{k-1}$, $\bar{y}_k^+ = \bar{r} + \kappa \bar{y}_{k-1}^+$ and $\underline{y}_k = \underline{r} + \kappa \underline{y}_{k-1}$. Using the assumption $\mathcal{F}_{k-1} \leq \mathcal{F}_{k-1}^\square$, there exists $(\bar{\Delta}_{k-1}^\square, \bar{y}_{k-1}^{+\square}, \underline{y}_{k-1}^\square) \in \mathcal{F}_{k-1}^\square$ such that $\bar{\Delta}_k \leq \bar{\Delta}_k^\square$, $\bar{y}_k^+ \leq \bar{y}_k^{+\square}$ and $\underline{y}_k \geq \underline{y}_k^\square$. We now have

$$\begin{aligned} \bar{\Delta}_k &= \kappa \bar{\Delta}_{k-1} \leq \kappa \bar{\Delta}_{k-1}^\square, \\ \bar{y}_k^+ &= \bar{r} + \kappa \bar{y}_{k-1}^+ \leq \bar{r} + \kappa \bar{y}_{k-1}^{+\square}, \text{ and} \\ \underline{y}_k &= \underline{r} + \kappa \underline{y}_{k-1} \geq \underline{r} + \kappa \underline{y}_{k-1}^\square. \end{aligned}$$

Element $(\kappa \bar{\Delta}_{k-1}^\square, \bar{r} + \kappa \bar{y}_{k-1}^{+\square}, \underline{r} + \kappa \underline{y}_{k-1}^\square) \in \mathcal{F}'_k$ is thus a witness showing $\mathcal{F}_k \leq \mathcal{F}'_k$.

Proof of claim 2: Proving $\mathcal{F}'_k \leq \mathcal{F}_k^\square$ is more involved. Consider definition of $\mathcal{F}'_k$ (equation (A.6)) and let us substitute for $\mathcal{F}_{k-1}^\square$ using definition of $\mathcal{F}_{k-1}^\square$ (equation (A.5)). This gives us

$$\mathcal{F}'_k = \left\{ \left\langle \begin{array}{l} \kappa' \bar{\Delta}_0, \\ \bar{r} + \kappa' \max \left\{ \bar{r} + \kappa[(k-2)\bar{r} + \bar{y}_0^+], (k-1)\bar{r} + \bar{y}_0^+ \right\}, \\ \underline{r} + \kappa' \min \left\{ \underline{r} + \kappa[(k-2)\underline{r} + \underline{y}_0], (k-1)\underline{r} + \underline{y}_0 \right\} \end{array} \right\rangle \mid \forall \kappa' \in [0, 1], \kappa \in [0, 1] \right\}. \quad (\text{A.7})$$

We will now break the proof in two cases $R(\cdot) < 0$ and $R(\cdot) > 0$ which helps us simplify the maximization and minimization terms.

We will start with $R(\cdot) < 0$. In this case, we have also that the initial $\bar{y}_0^+ < 0$ (the value of perfect information game) and $\underline{y}_0 < 0$. Hence

$$\mathcal{F}'_k = \left\{ \left\langle \begin{array}{c} \kappa' \kappa \bar{\Delta}_0, \\ \bar{r} + \kappa' [\bar{r} + \kappa[(k-2)\bar{r} + \bar{y}_0^+]], \\ \underline{r} + \kappa'[(k-1)\underline{r} + \underline{y}_0] \end{array} \right\rangle \mid \forall \kappa' \in [0, 1], \kappa \in [0, 1] \right\}. \quad (\text{A.8})$$

Every point in $\mathcal{F}'_k$ is identified with $\kappa, \kappa' \in [0, 1]$. Let κ, κ' be arbitrary. We have

$$\bar{r} + \kappa' [\bar{r} + \kappa[(k-2)\bar{r} + \bar{y}_0^+]] \leq \bar{r} + \kappa' \kappa [\bar{r} + (k-2)\bar{r} + \bar{y}_0^+] \quad (\text{A.9})$$

$$= \bar{r} + \kappa' \kappa [(k-1)\bar{r} + \bar{y}_0^+] \quad (\text{A.10})$$

$$= \max \{ \bar{r} + \kappa' \kappa [(k-1)\bar{r} + \bar{y}_0^+], k\bar{r} + \bar{y}_0^+ \} = \bar{y}_k^{+\square} \quad (\text{A.11})$$

$$\underline{r} + \kappa' [(k-1)\underline{r} + \underline{y}_0] \geq \underline{r} + [(k-1)\underline{r} + \underline{y}_0] = k\underline{r} + \underline{y}_0 \quad (\text{A.12})$$

$$= \min \{ \underline{r} + \kappa' [(k-1)\underline{r} + \underline{y}_0], k\underline{r} + \underline{y}_0 \} = \underline{y}_k^{\square}. \quad (\text{A.13})$$

Observe that $\kappa \kappa' \in [0, 1]$ and $(\kappa' \kappa \bar{\Delta}_0, \bar{y}_k^{+\square}, \underline{y}_k^{\square})$ is thus both element of $\mathcal{F}_k^{\square}$ as well as witness of $\mathcal{F}'_k \leq \mathcal{F}_k^{\square}$ (for case $R(\cdot) < 0$).

The proof for $R(\cdot) > 0$ is very similar. In this case

$$\mathcal{F}'_k = \left\{ \left\langle \begin{array}{c} \kappa' \kappa \bar{\Delta}_0, \\ \bar{r} + \kappa' [(k-1)\bar{r} + \bar{y}_0^+], \\ \underline{r} + \kappa' [\underline{r} + \kappa[(k-2)\underline{r} + \underline{y}_0]] \end{array} \right\rangle \mid \forall \kappa' \in [0, 1], \kappa \in [0, 1] \right\} \quad (\text{A.14})$$

and

$$\bar{r} + \kappa' [(k-1)\bar{r} + \bar{y}_0^+] \leq \bar{r} + [(k-1)\bar{r} + \bar{y}_0^+] = k\bar{r} + \bar{y}_0^+ \quad (\text{A.15})$$

$$= \max \{ \bar{r} + \kappa' [(k-1)\bar{r} + \bar{y}_0^+], k\bar{r} + \bar{y}_0^+ \} = \bar{y}_k^{+\square} \quad (\text{A.16})$$

$$\underline{r} + \kappa' [\underline{r} + \kappa[(k-2)\underline{r} + \underline{y}_0]] \leq \underline{r} + \kappa' \kappa [\underline{r} + (k-2)\underline{r} + \underline{y}_0] \quad (\text{A.17})$$

$$= \underline{r} + \kappa' \kappa [(k-1)\underline{r} + \underline{y}_0] \quad (\text{A.18})$$

$$= \max \{ \underline{r} + \kappa' \kappa [(k-1)\underline{r} + \underline{y}_0], k\underline{r} + \underline{y}_0 \} = \underline{y}_k^{\square} \quad (\text{A.19})$$

and $(\kappa' \kappa \bar{\Delta}_0, \bar{y}_k^{+\square}, \underline{y}_k^{\square})$ is the witness of $\mathcal{F}'_k \leq \mathcal{F}_k^{\square}$ for $R(\cdot) > 0$. This concludes the proof of the second claim as well as the proof of the entire lemma as we now have $\mathcal{F}_k \leq \mathcal{F}_k^{\square}$. $\square$

Theorem 2. Let G be a POSSPG with non-zero rewards. For every $0 < \varepsilon_{\text{inner}} < \varepsilon$ there exists k such that $\bar{V}_k^+(b_{\text{init}}) - \underline{V}_k(b_{\text{init}}) \leq \varepsilon$, and Algorithm 3 thus terminates and yields ε approximation of $V^*(b_{\text{init}})$.

Proof. To prove the theorem, we leverage the fact that

1. $\bar{y}_k^+$ cannot get lower than, e.g., $\min_b \underline{V}_0(b)$. Otherwise upper bound $\bar{V}_k^+$ on the value of the game would become lower than the lower bound.
2. $\bar{y}_k$ cannot get higher than $\bar{y}_0^+ + \varepsilon_{\text{inner}}$. Since we know that the inner loop of the algorithm terminates with $\bar{V}_k - \underline{V}_k \leq \varepsilon_{\text{prime}}$, this would mean that global lower bound $\underline{V}_k$ became higher than global upper bound $\bar{V}_k^+$ —which is impossible.

We break the proof in two cases, $R(\cdot) < 0$ and $R(\cdot) > 0$. If $R(\cdot) < 0$, Lemmas 2 and 4 imply that every update at depth k satisfies $\bar{y}_k^+ - \bar{y}_k \leq \kappa \bar{\Delta}_0$ and $\bar{y}_k^+ \leq \bar{r} + \kappa[(k-1)\bar{r} + \bar{y}_0^+]$ for some $\kappa \in [0, 1]$. Since we know that $\bar{y}_k^+$ cannot get lower than $\min_b \underline{V}_0(b)$, we are able to bound the value of κ :

$$\bar{r} + \kappa[(k-1)\bar{r} + \bar{y}_0^+] \geq \min_b \underline{V}_0(b) \quad (\text{A.20})$$

$$\kappa[(k-1)\bar{r} + \bar{y}_0^+] \geq \min_b \underline{V}_0(b) - \bar{r} \quad (\text{A.21})$$

$$\kappa \leq \frac{\min_b \underline{V}_0(b) - \bar{r}}{(k-1)\bar{r} + \bar{y}_0^+}. \quad (\text{A.22})$$

Observe that the numerator is negative and the denominator tends to $-\infty$ as $k \rightarrow \infty$. This means that also $\kappa \rightarrow 0$. Note that the bound on difference between $\bar{V}_0^+ - \bar{V}_0 \leq \kappa \bar{\Delta}_0$ depends on κ and thus goes to 0 as well. From the definition of limit, we can thus find k such that $\bar{V}_0^+ - \bar{V}_0 \leq \varepsilon - \varepsilon_{inner}$.

The proof for case $R(\cdot) > 0$ is analogous. From Lemmas 2 and 4 we know that every update at depth k satisfies $\bar{y}_k^+ - \bar{y}_k \leq \kappa \bar{\Delta}_0$ and $\bar{y}_k \geq \underline{r} + \kappa[(k-1)\underline{r} + \underline{y}_0]$ for some $\kappa \in [0, 1]$. Since $\bar{y}_k$ cannot exceed $\bar{y}_0^+ + \varepsilon_{inner}$ as discussed above, we can bound κ using

$$\kappa \leq (\bar{y}_0^+ + \varepsilon_{inner} - \underline{r}) / [(k-1)\underline{r} + \underline{y}_0]. \quad (\text{A.23})$$

The rest of the proof is identical to the case of $R(\cdot) < 0$. $\square$

A.5. Proof of Theorem 3

Theorem 3. Let G be a POSSPG with value function V^* and let $\varepsilon > 0$. There exists $\underline{\gamma} < 1$ such that for every $\gamma \geq \underline{\gamma}$ and $b \in \Delta(S)$ we have $|V_\gamma^*(b) - V^*(b)| \leq \varepsilon$.

Proof. To prove this claim, let us first consider a game $G_{k,\gamma}$ with value $V_{k,\gamma}^*(b)$ which differs from the discounted game G_γ by forcing player 1 to play uniform strategy after k -stages. Since player 1 has full control over her actions in the first k stages of $G_{k,\gamma}$, she can enforce that the expected payoff in the first k stages of $G_{k,\gamma}$ will be identical to the payoff in G_γ . Hence player 1 can only lose at most $|V_\gamma^*(b) - V_{k,\gamma}^*(b)| \leq \sum_{l=k+1}^{\infty} \gamma^{l-1}(\bar{r} - \underline{r}) = \gamma^k(\bar{r} - \underline{r})/(1 - \gamma)$ where $\bar{r}$ and $\underline{r}$ are maximum and minimum rewards in the game. Observe that by appropriate choice of k the difference $|V_\gamma^*(b) - V_{k,\gamma}^*(b)|$ can therefore be made arbitrarily small.

Furthermore, let us consider a k -cutoff game G_k (see Section 5.1.1) with value $V_k^*(b)$. This game is identical to $G_{k,\gamma}$ with the exception that the payoffs of the players are not discounted by γ^l . Observe that as we increase the discount factor $\gamma \rightarrow 1$, the difference between the payoff $u_{k,\gamma}(\sigma_1, \sigma_2)$ of a strategy profile (σ_1, σ_2) in the discounted game $G_{k,\gamma}$ and the payoff $u_k(\sigma_1, \sigma_2)$ of the same strategy profile in the undiscounted game G_k goes to zero. Moreover, since the players only have control over the behavior in the first k stages (player 1 has to play uniformly afterwards while player 2 best-responds), the number of pure strategies in G_k and $G_{k,\gamma}$ is finite. For arbitrary value of $\varepsilon_\gamma > 0$ we can thus find a value of discount factor γ' such for every pure strategy profile (σ_1, σ_2) and for every $\gamma \geq \gamma'$ we have $|u_{k,\gamma}(\sigma_1, \sigma_2) - u_k(\sigma_1, \sigma_2)| \leq \varepsilon_\gamma$ and consequently also $|V_{k,\gamma}^*(b) - V_k^*(b)| \leq \varepsilon_\gamma$.

Finally, we use Theorem 2 to show that the difference $|V_k^*(b) - V^*(b)|$ can be made arbitrarily small as well. Theorem 2 indicates that for every ε_k there exists k such that $\bar{V}_k^+(b) - \underline{V}_k(b) \leq \varepsilon_k$. Recall that $\underline{V}_k \leq V_k^* \leq \bar{V}_k \leq \bar{V}_k^+$ and also $\underline{V}_k \leq V^* \leq \bar{V}_k^+$. Therefore we also have $|V_k^*(b) - V^*(b)| \leq \varepsilon_k$.

In the previous paragraphs we have shown that we can make the differences $|V_\gamma^*(b) - V_{k,\gamma}^*(b)|$, $|V_{k,\gamma}^*(b) - V_k^*(b)|$ and $|V_k^*(b) - V^*(b)|$ arbitrarily small by choosing sufficiently large values of k and γ . This also implies that we can make the difference $|V_\gamma^*(b) - V^*(b)| \leq |V_\gamma^*(b) - V_{k,\gamma}^*(b)| + |V_{k,\gamma}^*(b) - V_k^*(b)| + |V_k^*(b) - V^*(b)|$ arbitrarily small (i.e., smaller than ε from the statement of the theorem) which concludes the proof. $\square$

Appendix B. Experimental results

Here, we provide additional results and data of the experimental comparison of our proposed goal algorithms. First, we show a comparison with the traditional discounted HSVI algorithm (dHSVI) in terms of solution quality for computational settings *one-way update, rough increase* and *one-way update, smooth increase*. Additionally, we provide exact data (i.e., achieved gaps) for each instance for all combinations of computational settings (i.e., *one-way update, rough increase*, *one-way update, smooth increase*, *two-way update, rough increase* and *two-way update, smooth increase*). Then, for the same computational settings, we show a comparison with the modified discounted HSVI (udHSVI) in terms of algorithm scalability. Finally, we provide a scalability comparison with the traditional discounted HSVI algorithm for the full range of computational settings.

B.1. Solution quality

In terms of solution quality, dHSVI compares to both goal algorithms under *one-way update* setting similarly as under *two-way update*. For both domains, it still gets outperformed by both goal algorithms. However, for the pursuit-evasion domain, this is even more pronounced compared to the *two-way update*. On the other hand, for the privilege escalation domain, dHSVI comes out slightly better from the comparison with goal algorithms under *one-way update*.

B.1.1. GoalHorizon vs discounted HSVI (Figs. B.17 and B.18)

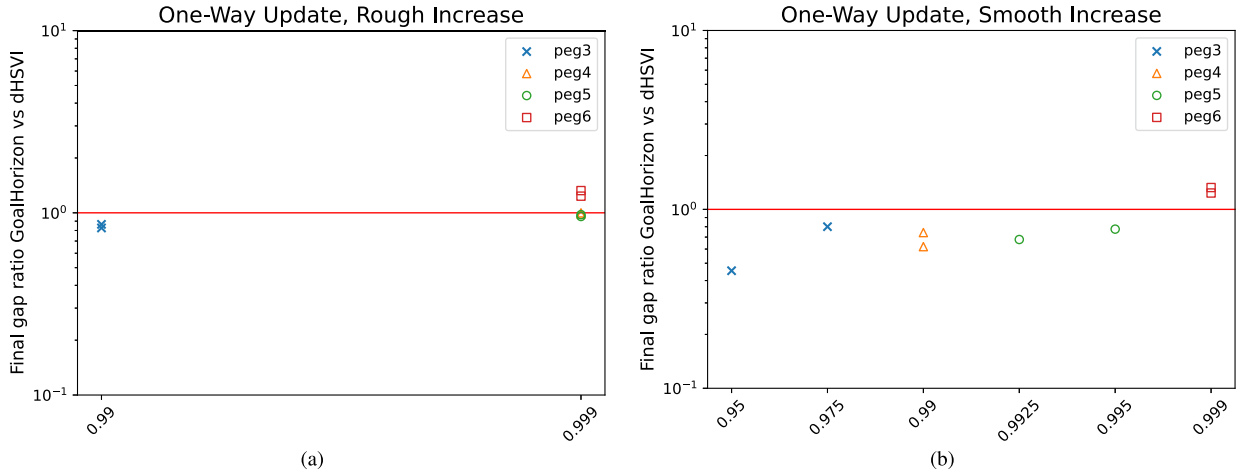


Fig. B.17. Comparison of the quality of solutions found by the GoalHorizon algorithm and the discounted HSVI (dHSVI) algorithm when evaluated under the undiscounted setting on the domain of pursuit-evasion games.

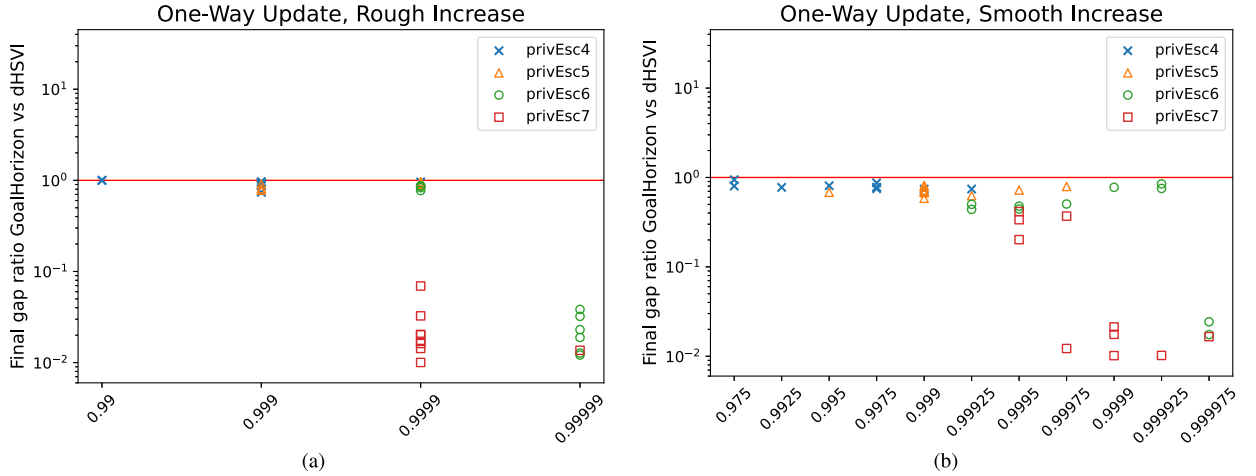


Fig. B.18. Comparison of the quality of solutions found by the GoalHorizon algorithm and the discounted HSVI (dHSVI) algorithm when evaluated under the undiscounted setting on the domain of privilege escalation games.

B.1.2. GoalDiscount vs discounted HSVI (Figs. B.19 and B.20)

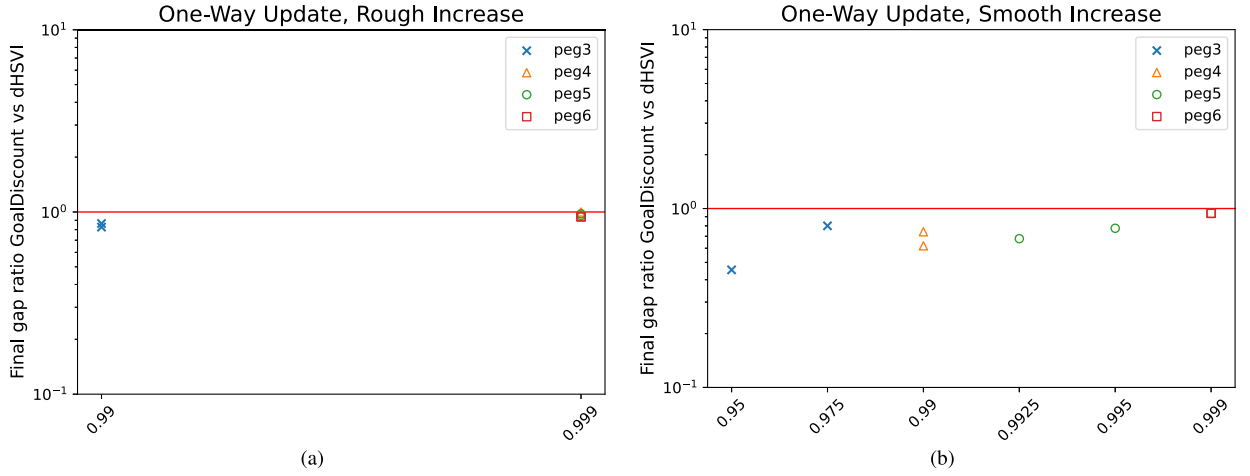


Fig. B.19. Comparison of the quality of solutions found by GoalDiscount algorithm and the discounted HSVI (dHSVI) algorithm when evaluated under the undiscounted setting on the domain of pursuit-evasion games.

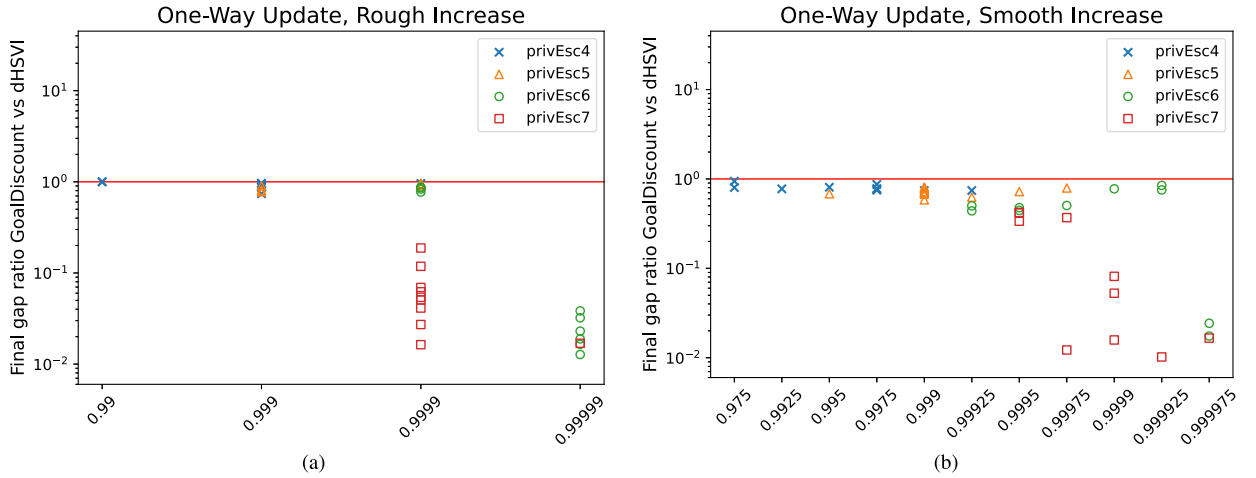


Fig. B.20. Comparison of the quality of solutions found by GoalDiscount algorithm and the discounted HSVI (dHSVI) algorithm when evaluated under the undiscounted setting on the domain of privilege escalation games.

B.1.3. Solution quality data (Tables B.2–B.7)

Table B.2

Solution quality (achieved undiscounted gaps) of individual algorithms on pursuit-evasion games domain under rough increase setting. If the gap is not sufficiently tight (i.e., less or equal 1), we present the reason for not finishing in the bracket (TO - time out, OoM - out of memory).

Instance	Update Setting	Increase Setting	dHSVI	GoalHorizon	GoalDiscount
peg3_0	One-way Update	Rough Increase	1.15680	0.93819	0.84706
peg3_1	One-way Update	Rough Increase	1.21019	0.91646	0.76695
peg3_0	Two-way Update	Rough Increase	1.11705	0.96007	0.54082
peg3_1	Two-way Update	Rough Increase	0.51075	0.95192	0.91778
peg4_0	One-way Update	Rough Increase	1.01631	0.99504	0.86209
peg4_1	One-way Update	Rough Increase	0.97438	0.99669	0.87379
peg4_0	Two-way Update	Rough Increase	1.03313	0.99935	0.95351
peg4_1	Two-way Update	Rough Increase	0.90900	0.99816	0.97890

(continued on next page)

Table B.2 (continued)

Instance	Update Setting	Increase Setting	dHSVI	GoalHorizon	GoalDiscount
peg5_0	One-way Update	Rough Increase	1.04752	0.99580	0.95348
peg5_1	One-way Update	Rough Increase	1.02366	0.99703	0.92154
peg5_0	Two-way Update	Rough Increase	1.01953	0.99698	0.95267
peg5_1	Two-way Update	Rough Increase	1.02336	0.97664	0.98737
peg6_0	One-way Update	Rough Increase	1.06347	1.31278 (TO)	0.99235
peg6_1	One-way Update	Rough Increase	1.06405	1.40928 (TO)	0.99443
peg6_0	Two-way Update	Rough Increase	1.06346	1.45509 (TO)	0.99412
peg6_1	Two-way Update	Rough Increase	1.06343	1.26307 (TO)	0.99898

Table B.3

Solution quality (achieved undiscounted gaps) of individual algorithms on pursuit-evasion games domain under smooth increase setting. If the gap is not sufficiently tight (i.e., less or equal 1), we present the reason for not finishing in the bracket (TO - time out, OoM - out of memory).

Instance	Update Setting	Increase Setting	dHSVI	GoalHorizon	GoalDiscount
peg3_0	One-way Update	Smooth Increase	2.20287	0.93819	0.89338
peg3_1	One-way Update	Smooth Increase	1.24979	0.91646	0.45672
peg3_0	Two-way Update	Smooth Increase	1.59742	0.96007	0.98400
peg3_1	Two-way Update	Smooth Increase	0.87472	0.95192	0.96213
peg4_0	One-way Update	Smooth Increase	1.34832	0.99504	0.91402
peg4_1	One-way Update	Smooth Increase	1.61583	0.99669	0.93078
peg4_0	Two-way Update	Smooth Increase	1.35763	0.99935	0.94719
peg4_1	Two-way Update	Smooth Increase	1.40576	0.99816	0.93240
peg5_0	One-way Update	Smooth Increase	1.28979	0.99580	0.99548
peg5_1	One-way Update	Smooth Increase	1.47393	0.99703	0.99575
peg5_0	Two-way Update	Smooth Increase	1.26716	0.99698	0.98948
peg5_1	Two-way Update	Smooth Increase	1.20677	0.97664	0.96344
peg6_0	One-way Update	Smooth Increase	1.06347	1.31278 (TO)	0.99307
peg6_1	One-way Update	Smooth Increase	1.06405	1.40928 (TO)	0.99513
peg6_0	Two-way Update	Smooth Increase	1.16264	1.45509 (TO)	1.00527 (OoM)
peg6_1	Two-way Update	Smooth Increase	1.16370	1.26307 (TO)	1.02409 (OoM)

Table B.4

Solution quality (achieved undiscounted gaps) of individual algorithms on privilege escalation games domain under one-way update, rough increase setting. If the gap is not sufficiently tight (i.e., less or equal 1), we present the reason for not finishing in the bracket (TO - time out, OoM - out of memory).

Instance	Update Setting	Increase Setting	dHSVI	GoalHorizon	GoalDiscount
privEsc4_01	One-way Update	Rough Increase	1.07452	0.95980	0.97358
privEsc4_02	One-way Update	Rough Increase	1.13292	0.98717	0.97597
privEsc4_03	One-way Update	Rough Increase	1.04340	0.98759	0.98904
privEsc4_04	One-way Update	Rough Increase	1.04140	0.99610	0.99802
privEsc4_05	One-way Update	Rough Increase	1.12495	0.87633	0.98867
privEsc4_06	One-way Update	Rough Increase	1.35049	0.97661	0.99741
privEsc4_07	One-way Update	Rough Increase	1.15192	0.99429	0.99860
privEsc4_08	One-way Update	Rough Increase	0.81746	0.94554	0.99647
privEsc4_09	One-way Update	Rough Increase	0.97732	0.98744	0.99255
privEsc4_10	One-way Update	Rough Increase	1.04014	0.99545	0.99612
privEsc5_01	One-way Update	Rough Increase	1.28406	0.99884	0.99992
privEsc5_02	One-way Update	Rough Increase	1.08404	0.99730	0.99144
privEsc5_03	One-way Update	Rough Increase	1.05182	0.99251	0.99817
privEsc5_04	One-way Update	Rough Increase	1.07775	0.99394	0.99883
privEsc5_05	One-way Update	Rough Increase	1.10604	0.99690	0.99110
privEsc5_06	One-way Update	Rough Increase	1.10665	0.99738	0.99926
privEsc5_07	One-way Update	Rough Increase	1.04754	0.98643	0.99717
privEsc5_08	One-way Update	Rough Increase	1.23136	0.98990	0.99890
privEsc5_09	One-way Update	Rough Increase	1.07645	0.99991	0.99658
privEsc5_10	One-way Update	Rough Increase	1.03706	0.98906	0.99784
privEsc6_01	One-way Update	Rough Increase	1.14534	0.99914	0.99997
privEsc6_02	One-way Update	Rough Increase	26.05254 (TO)	0.99817	0.99783
privEsc6_03	One-way Update	Rough Increase	1.15676	0.99925	0.99928
privEsc6_04	One-way Update	Rough Increase	43.46806 (TO)	0.99897	0.99769
privEsc6_05	One-way Update	Rough Increase	82.58478 (TO)	0.99690	1.36195 (TO)
privEsc6_06	One-way Update	Rough Increase	1.29362	0.99633	0.99884
privEsc6_07	One-way Update	Rough Increase	52.93067 (TO)	0.99964	0.99966
privEsc6_08	One-way Update	Rough Increase	1.20251	0.99888	0.99999

Table B.4 (continued)

Instance	Update Setting	Increase Setting	dHSVI	GoalHorizon	GoalDiscount
privEsc6_09	One-way Update	Rough Increase	78.48841 (TO)	0.99862	0.99980
privEsc6_10	One-way Update	Rough Increase	31.08412 (TO)	0.99986	0.99977
privEsc7_01	One-way Update	Rough Increase	14.42682 (TO)	0.99794	0.99986
privEsc7_02	One-way Update	Rough Increase	210.13834 (TO)	0.99994	3.43426 (TO)
privEsc7_03	One-way Update	Rough Increase	73.31795 (TO)	0.99830	1.24134 (TO)
privEsc7_04	One-way Update	Rough Increase	49.59110 (TO)	0.99971	1.34814 (TO)
privEsc7_05	One-way Update	Rough Increase	166.89702 (TO)	2.38747 (TO)	8.45028 (TO)
privEsc7_06	One-way Update	Rough Increase	61.69744 (TO)	0.99913	3.82904 (TO)
privEsc7_07	One-way Update	Rough Increase	49.09049 (TO)	0.99892	5.80566 (TO)
privEsc7_08	One-way Update	Rough Increase	30.69030 (TO)	0.99798	1.26509 (TO)
privEsc7_09	One-way Update	Rough Increase	58.34337 (TO)	0.99923	10.95752 (TO)
privEsc7_10	One-way Update	Rough Increase	99.62517 (TO)	0.99741	5.38458 (TO)

Table B.5

Solution quality (achieved undiscounted gaps) of individual algorithms on privilege escalation games domain under one-way update, smooth increase setting. If the gap is not sufficiently tight (i.e., less or equal 1), we present the reason for not finishing in the bracket (TO - time out, OoM - out of memory).

Instance	Update Setting	Increase Setting	dHSVI	GoalHorizon	GoalDiscount
privEsc4_01	One-way Update	Smooth Increase	1.15011	0.95980	0.98177
privEsc4_02	One-way Update	Smooth Increase	1.29822	0.98717	0.99934
privEsc4_03	One-way Update	Smooth Increase	1.34701	0.98759	0.99438
privEsc4_04	One-way Update	Smooth Increase	1.23795	0.99610	0.99934
privEsc4_05	One-way Update	Smooth Increase	1.28439	0.87633	0.99145
privEsc4_06	One-way Update	Smooth Increase	1.35049	0.97661	0.99245
privEsc4_07	One-way Update	Smooth Increase	1.33612	0.99429	0.99641
privEsc4_08	One-way Update	Smooth Increase	1.05995	0.94554	0.99003
privEsc4_09	One-way Update	Smooth Increase	1.24765	0.98744	0.98118
privEsc4_10	One-way Update	Smooth Increase	1.29105	0.99545	0.99580
privEsc5_01	One-way Update	Smooth Increase	1.28406	0.99884	0.99853
privEsc5_02	One-way Update	Smooth Increase	1.59889	0.99730	0.99760
privEsc5_03	One-way Update	Smooth Increase	1.47951	0.99251	0.99875
privEsc5_04	One-way Update	Smooth Increase	1.38243	0.99394	0.99709
privEsc5_05	One-way Update	Smooth Increase	1.46680	0.99690	0.99707
privEsc5_06	One-way Update	Smooth Increase	1.26200	0.99738	0.99803
privEsc5_07	One-way Update	Smooth Increase	1.43686	0.98643	0.99677
privEsc5_08	One-way Update	Smooth Increase	1.23136	0.98990	0.99556
privEsc5_09	One-way Update	Smooth Increase	1.71220	0.99991	0.99789
privEsc5_10	One-way Update	Smooth Increase	1.38766	0.98906	0.99082
privEsc6_01	One-way Update	Smooth Increase	2.27615	0.99914	0.99785
privEsc6_02	One-way Update	Smooth Increase	1.17959	0.99817	0.99994
privEsc6_03	One-way Update	Smooth Increase	1.99841	0.99925	0.99904
privEsc6_04	One-way Update	Smooth Increase	1.28931	0.99897	0.99939
privEsc6_05	One-way Update	Smooth Increase	57.22029 (TO)	0.99690	0.99917
privEsc6_06	One-way Update	Smooth Increase	1.98199	0.99633	0.99987
privEsc6_07	One-way Update	Smooth Increase	1.32745	0.99964	0.99857
privEsc6_08	One-way Update	Smooth Increase	2.09464	0.99888	0.99977
privEsc6_09	One-way Update	Smooth Increase	41.15668 (TO)	0.99862	0.99932
privEsc6_10	One-way Update	Smooth Increase	2.25152	0.99986	0.99981
privEsc7_01	One-way Update	Smooth Increase	2.40517	0.99794	0.99999
privEsc7_02	One-way Update	Smooth Increase	81.92511 (TO)	0.99994	0.99966
privEsc7_03	One-way Update	Smooth Increase	60.30604 (TO)	0.99830	0.99993
privEsc7_04	One-way Update	Smooth Increase	2.96909	0.99971	0.99917
privEsc7_05	One-way Update	Smooth Increase	11.84217 (TO)	2.38747 (TO)	4.99097 (TO)
privEsc7_06	One-way Update	Smooth Increase	97.97670 (TO)	0.99913	0.99991
privEsc7_07	One-way Update	Smooth Increase	47.01716 (TO)	0.99892	2.47753 (TO)
privEsc7_08	One-way Update	Smooth Increase	2.70883	0.99798	0.99903
privEsc7_09	One-way Update	Smooth Increase	56.91420 (TO)	0.99923	4.63263 (TO)
privEsc7_10	One-way Update	Smooth Increase	98.44357 (TO)	0.99741	1.55566 (TO)

Table B.6

Solution quality (achieved undiscounted gaps) of individual algorithms on privilege escalation games domain under two-way update, rough increase setting. If the gap is not sufficiently tight (i.e., less or equal 1), we present the reason for not finishing in the bracket (TO - time out, OoM - out of memory).

Instance	Update Setting	Increase Setting	dHSVI	GoalHorizon	GoalDiscount
privEsc4_01	Two-way Update	Rough Increase	1.06734	0.94157	0.99234
privEsc4_02	Two-way Update	Rough Increase	1.13144	0.96295	0.98772
privEsc4_03	Two-way Update	Rough Increase	1.04754	0.99220	0.97657
privEsc4_04	Two-way Update	Rough Increase	1.04002	0.97762	0.99768
privEsc4_05	Two-way Update	Rough Increase	1.11704	0.92660	0.99120
privEsc4_06	Two-way Update	Rough Increase	1.33381	0.97993	0.99972
privEsc4_07	Two-way Update	Rough Increase	1.14546	0.98817	0.99586
privEsc4_08	Two-way Update	Rough Increase	0.81959	0.91805	0.99721
privEsc4_09	Two-way Update	Rough Increase	0.98770	0.96038	0.97616
privEsc4_10	Two-way Update	Rough Increase	1.04620	0.98272	0.99123
privEsc5_01	Two-way Update	Rough Increase	1.28543	0.99341	0.99937
privEsc5_02	Two-way Update	Rough Increase	1.08450	0.99822	0.99132
privEsc5_03	Two-way Update	Rough Increase	1.05022	0.99369	0.99896
privEsc5_04	Two-way Update	Rough Increase	1.07695	0.98065	0.98819
privEsc5_05	Two-way Update	Rough Increase	1.10460	0.99687	0.99737
privEsc5_06	Two-way Update	Rough Increase	1.10423	0.99168	0.99780
privEsc5_07	Two-way Update	Rough Increase	1.04691	0.99978	0.99872
privEsc5_08	Two-way Update	Rough Increase	1.23404	0.98654	0.99822
privEsc5_09	Two-way Update	Rough Increase	1.07641	0.98791	0.99876
privEsc5_10	Two-way Update	Rough Increase	1.03845	0.99706	0.99888
privEsc6_01	Two-way Update	Rough Increase	1.15448	0.99946	0.99974
privEsc6_02	Two-way Update	Rough Increase	25.12554 (TO)	0.99813	0.99966
privEsc6_03	Two-way Update	Rough Increase	1.12068	0.99587	0.99826
privEsc6_04	Two-way Update	Rough Increase	45.49291 (TO)	0.99454	0.99977
privEsc6_05	Two-way Update	Rough Increase	85.12973 (TO)	0.99795	1.37032 (TO)
privEsc6_06	Two-way Update	Rough Increase	1.25893	0.99991	0.99861
privEsc6_07	Two-way Update	Rough Increase	38.97663 (TO)	0.99871	0.99989
privEsc6_08	Two-way Update	Rough Increase	1.15164	0.99458	0.99993
privEsc6_09	Two-way Update	Rough Increase	67.76043 (TO)	0.98866	0.99931
privEsc6_10	Two-way Update	Rough Increase	12.67228 (TO)	0.99589	0.99986
privEsc7_01	Two-way Update	Rough Increase	8.79662 (TO)	0.99977	0.99986
privEsc7_02	Two-way Update	Rough Increase	192.65172 (TO)	4.81376 (TO)	2.87420 (TO)
privEsc7_03	Two-way Update	Rough Increase	68.98724 (TO)	0.99911	1.19575 (TO)
privEsc7_04	Two-way Update	Rough Increase	43.66524 (TO)	0.99980	0.99963
privEsc7_05	Two-way Update	Rough Increase	155.40851 (TO)	6.76632 (TO)	7.97876 (TO)
privEsc7_06	Two-way Update	Rough Increase	43.22615 (TO)	0.99745	2.63946 (TO)
privEsc7_07	Two-way Update	Rough Increase	31.82328 (TO)	0.99974	3.52153 (TO)
privEsc7_08	Two-way Update	Rough Increase	134.88304 (TO)	0.99934	0.99989
privEsc7_09	Two-way Update	Rough Increase	50.41234 (TO)	0.99989	9.53890 (TO)
privEsc7_10	Two-way Update	Rough Increase	91.02089 (TO)	0.99975	4.66746 (TO)

Table B.7

Solution quality (achieved undiscounted gaps) of individual algorithms on privilege escalation games domain under two-way update, smooth increase setting. If the gap is not sufficiently tight (i.e., less or equal 1), we present the reason for not finishing in the bracket (TO - time out, OoM - out of memory).

Instance	Update Setting	Increase Setting	dHSVI	GoalHorizon	GoalDiscount
privEsc4_01	Two-way Update	Smooth Increase	1.13450	0.94157	0.96161
privEsc4_02	Two-way Update	Smooth Increase	1.29175	0.96295	0.99281
privEsc4_03	Two-way Update	Smooth Increase	1.44045	0.99220	0.99582
privEsc4_04	Two-way Update	Smooth Increase	1.24388	0.97762	0.99469
privEsc4_05	Two-way Update	Smooth Increase	1.27329	0.92660	0.98876
privEsc4_06	Two-way Update	Smooth Increase	1.33381	0.97993	0.97849
privEsc4_07	Two-way Update	Smooth Increase	1.32960	0.98817	0.99860
privEsc4_08	Two-way Update	Smooth Increase	1.06335	0.91805	0.99157
privEsc4_09	Two-way Update	Smooth Increase	1.22152	0.96038	0.99598
privEsc4_10	Two-way Update	Smooth Increase	1.27986	0.98272	0.99895
privEsc5_01	Two-way Update	Smooth Increase	1.28543	0.99341	0.99664
privEsc5_02	Two-way Update	Smooth Increase	1.59888	0.99822	0.99776
privEsc5_03	Two-way Update	Smooth Increase	1.36029	0.99369	0.99763
privEsc5_04	Two-way Update	Smooth Increase	1.38083	0.98065	0.99857
privEsc5_05	Two-way Update	Smooth Increase	1.67977	0.99687	0.99888
privEsc5_06	Two-way Update	Smooth Increase	1.25513	0.99168	0.99539

Table B.7 (continued)

Instance	Update Setting	Increase Setting	dHSVI	GoalHorizon	GoalDiscount
privEsc5_07	Two-way Update	Smooth Increase	1.43426	0.99978	0.99658
privEsc5_08	Two-way Update	Smooth Increase	1.23404	0.98654	0.99962
privEsc5_09	Two-way Update	Smooth Increase	1.71242	0.98791	0.99797
privEsc5_10	Two-way Update	Smooth Increase	1.38202	0.99706	0.99988
privEsc6_01	Two-way Update	Smooth Increase	2.18307	0.99946	0.99896
privEsc6_02	Two-way Update	Smooth Increase	1.17447	0.99813	0.99991
privEsc6_03	Two-way Update	Smooth Increase	1.99353	0.99587	0.99995
privEsc6_04	Two-way Update	Smooth Increase	1.28131	0.99454	0.99874
privEsc6_05	Two-way Update	Smooth Increase	84.74807 (TO)	0.99795	0.99651
privEsc6_06	Two-way Update	Smooth Increase	1.68638	0.99991	0.99976
privEsc6_07	Two-way Update	Smooth Increase	3.47570 (TO)	0.99871	0.99935
privEsc6_08	Two-way Update	Smooth Increase	2.09043	0.99458	0.99963
privEsc6_09	Two-way Update	Smooth Increase	29.19936 (TO)	0.98866	0.99888
privEsc6_10	Two-way Update	Smooth Increase	2.17360	0.99589	0.99998
privEsc7_01	Two-way Update	Smooth Increase	2.49143	0.99977	0.99983
privEsc7_02	Two-way Update	Smooth Increase	67.28148 (TO)	4.81376 (TO)	0.99910
privEsc7_03	Two-way Update	Smooth Increase	56.84984 (TO)	0.99911	0.99970
privEsc7_04	Two-way Update	Smooth Increase	3.11023	0.99980	0.99995
privEsc7_05	Two-way Update	Smooth Increase	51.41502 (TO)	6.76632 (TO)	3.74153 (TO)
privEsc7_06	Two-way Update	Smooth Increase	68.76197 (TO)	0.99745	0.99980
privEsc7_07	Two-way Update	Smooth Increase	143.16508 (TO)	0.99974	1.15550 (TO)
privEsc7_08	Two-way Update	Smooth Increase	2.76371	0.99934	0.99993
privEsc7_09	Two-way Update	Smooth Increase	49.79715 (TO)	0.99989	4.14629 (TO)
privEsc7_10	Two-way Update	Smooth Increase	88.75713 (TO)	0.99975	1.33443 (TO)

B.2. Scalability with guaranteed solution quality

When comparing goal algorithms to the modified discounted HSVI (udHSVI) in terms of algorithm scalability, we can observe that just like under *two-way update*, the goal algorithms scale better with increased problem difficulty under *one-way update* as well. Furthermore, the goal algorithms come out of the comparison on more difficult instances better under *one-way update*.

B.2.1. GoalHorizon vs udHSVI (Figs. B.21 and B.22)

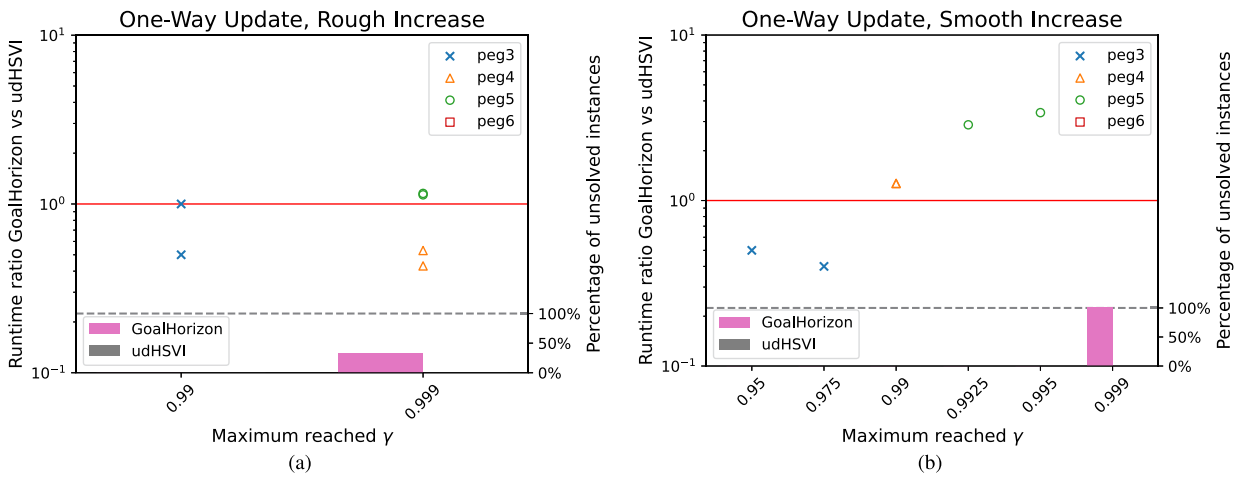


Fig. B.21. Scalability comparison between the GoalHorizon algorithm and the udHSVI algorithm on the domain of pursuit-evasion games.

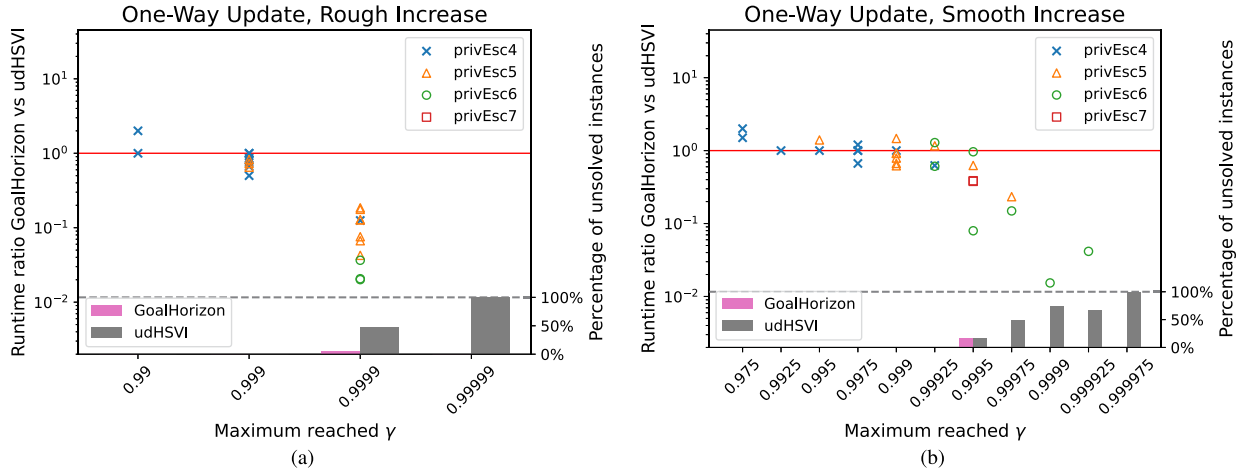


Fig. B.22. Scalability comparison between the GoalHorizon algorithm and the udHSVI algorithm on the domain of privilege escalation games.

B.2.2. GoalDiscount vs udHSVI (Figs. B.23 and B.24)

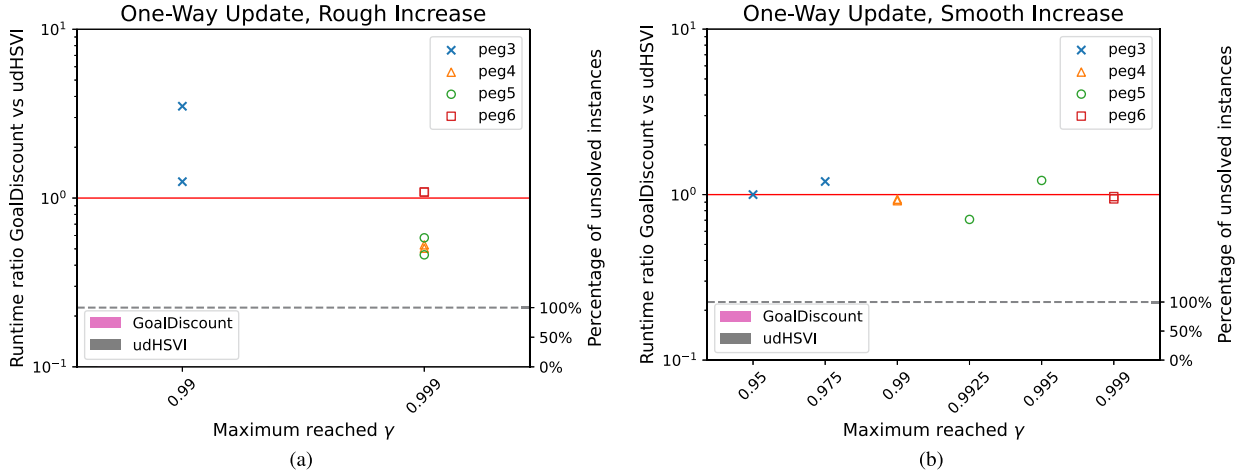


Fig. B.23. Scalability comparison between GoalDiscount algorithm and udHSVI algorithm on the domain of pursuit-evasion games.

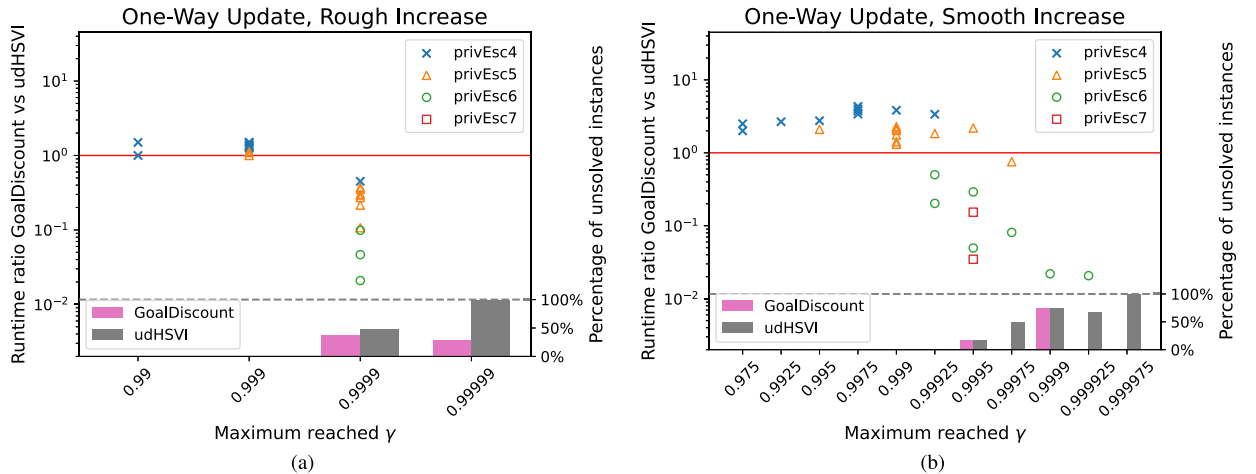


Fig. B.24. Scalability comparison between GoalDiscount algorithm and udHSVI algorithm on the domain of privilege escalation games.

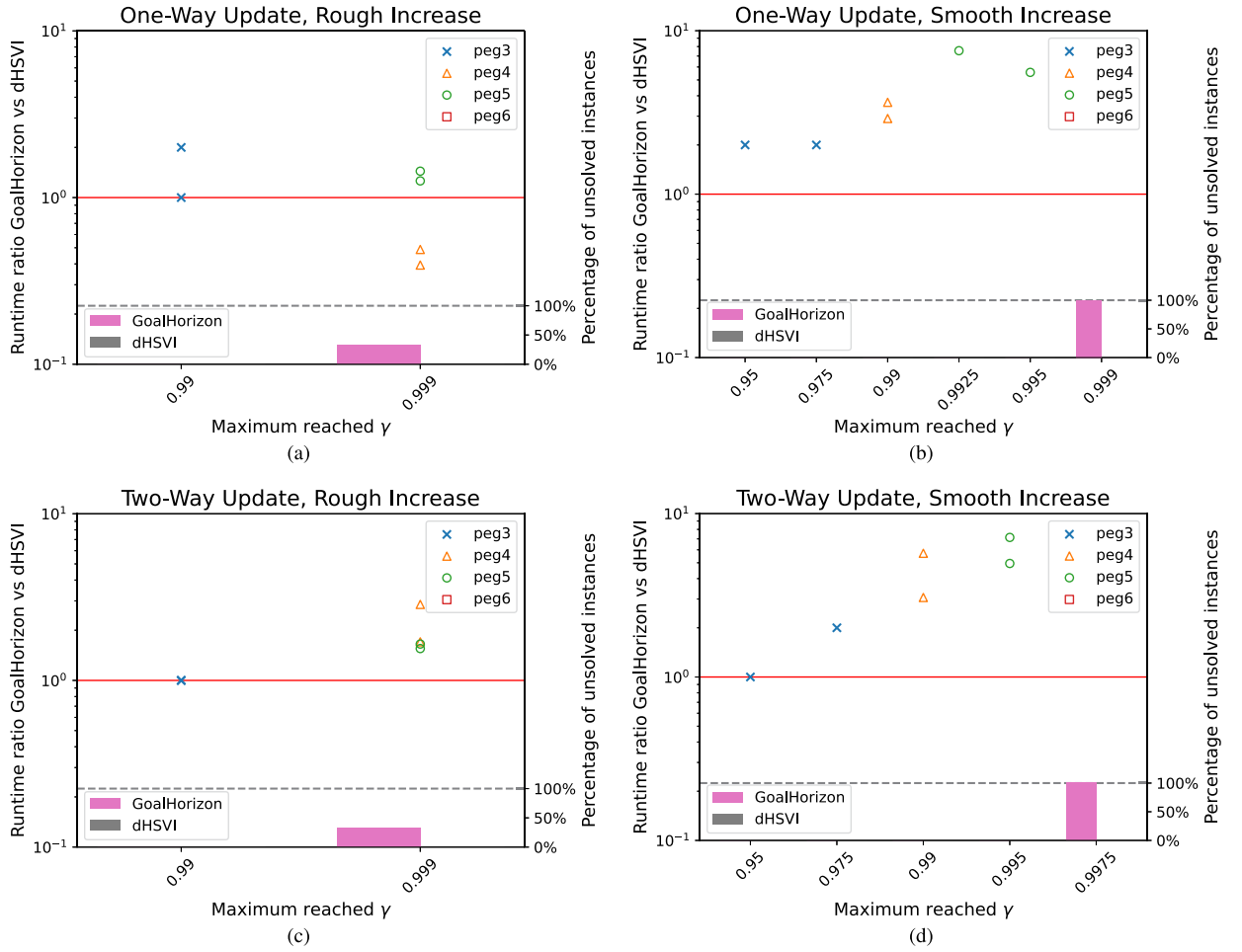


Fig. B.25. Scalability comparison between the GoalHorizon algorithm and the discounted HSVI (dHSVI) algorithm on the domain of pursuit-evasion games.

B.3. Algorithm scalability

Now we present results of experiments comparing GoalHorizon algorithm (Section 5.1) and GoalDiscount algorithm (Section 5.2) to the traditional discounted HSVI (dHSVI) algorithm based on the scalability. The experiments are done on the domains described in Section 6.

Similarly to the scalability comparison between presented POSSPGs algorithms and the modified version of the discounted HSVI algorithm (udHSVI), we visualize obtained scalability results using dual charts. On the primary y-axis (the left one) is the ratio $\frac{\text{goal_algorithm}}{\text{baseline}}$ of the times needed to find a solution. This ratio is visualized as a scatter plot in the upper part of the chart. Whenever a point ends up being below the red 1-line, it means that the particular goal algorithm is better than the baseline (in this case, the discounted HSVI algorithm). Situations where one of the compared algorithms was not able to find the solution in the given time limit are omitted from the scatter plot. On the secondary y-axis (the right one) is the respective percentage of games for which the individual algorithms were unable to find a solution in the given time limit, and this is visualized as a grouped bar plot in the bottom part of the charts.

B.3.1. GoalHorizon vs discounted HSVI

When comparing the GoalHorizon algorithm to the discounted HSVI algorithm based on the algorithm scalability on the domain of pursuit-evasion games (Fig. B.25), we can observe that the GoalHorizon algorithm scales better with one-way update setting (Fig. B.25a and Fig. B.25b). Despite that, we can clearly see that on this particular domain, the GoalHorizon algorithm loses to the discounted HSVI algorithm in terms of scalability since for all settings combinations (except the one-way update, rough increase one) the GoalHorizon algorithm is never faster than the discounted HSVI algorithm. The reason for the better performance for this particular setting (one-way update, rough increase) is that the discounted HSVI algorithm solves the same problem instances but with a higher discount factor γ (due to the fact that these values of γ were obtained by the GoalDiscount algorithm assuming rough update setting). This can be seen when comparing values of discount factor gamma for fixed update setting based on the increase setting.

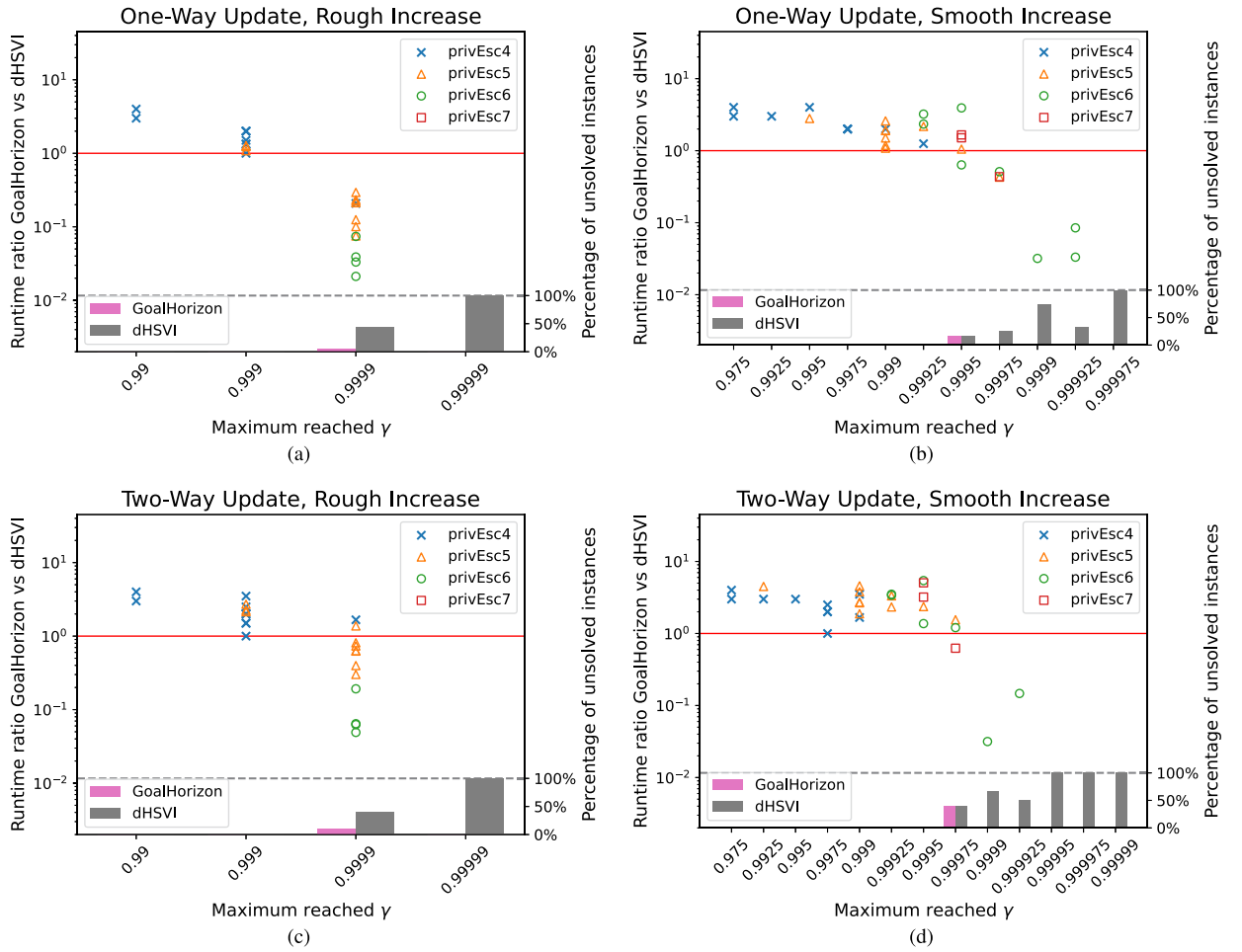


Fig. B.26. Scalability comparison between the GoalHorizon algorithm and the discounted HSVI (dHSVI) algorithm on the domain of privilege escalation games.

For example, assume *peg4* instances under one-way update setting. When comparing corresponding figures (Fig. B.25a and Fig. B.25b), we can observe that for the rough increase setting, we obtained value of discount factor $\gamma = 0.999$ for both instances. However, for the smooth increase setting, the obtained value of the discount factor was $\gamma = 0.99$ (again for both instances). Since the computational complexity of HSVI-based algorithms increases with the value of the discount factor γ and only the discounted HSVI algorithm assumes discount factor γ (i.e., run times of the GoalHorizon algorithm are identical for both rough increase and smooth increase setting) it is expected to see results to be more in favor of the discounted HSVI algorithm under smooth increase setting compared to the rough increase setting (with update setting being fixed). Better scalability results of the discounted HSVI algorithm under smooth increase setting compared to the rough increase setting are expected in general since smooth increase setting can lead only to smaller values of discount factor γ and thus decreased computational complexity (the same instances will be solved for a smaller value of discount factor γ under smooth increase setting).

Furthermore, even for the one-way update, rough increase setting combination, we can observe that the GoalHorizon algorithm is unable to solve any instance of *peg6*, just like under all remaining setting combinations. This only highlights that the GoalHorizon algorithm is not the best choice for pursuit-evasion games in terms of scalability. We think that this is mostly due to no additional information (besides whether the game ended or still continues) provided to the partially informed player during the course of pursuit-evasion games. Thus, there can be not enough shared information between individual k -cutoff games to have similarities among the corresponding computed strategies (i.e., the strategy computed for $k + 1$ -cutoff game highly differs from the one computed for k -cutoff game).

We can see that the situation gets completely different for the domain of privilege escalation games (Fig. B.26). Based on the individual charts, the GoalHorizon algorithm still better compares to the discounted HSVI algorithm under one-way update setting (Fig. B.26a and Fig. B.26b). This time, however, we can see that the GoalHorizon algorithm keeps with the discounted HSVI algorithm better for smaller discount factors and outperforms it with larger ones by a significant margin. This is further highlighted by the percentage of unsolved games for larger discount factors, where we can see that the discounted HSVI algorithm struggles to solve games within the time limit. On the other hand, the GoalHorizon algorithm has almost no issues finding a solution. This can be clearly seen for values of discount factors $\gamma \geq 0.9995$. For these values, the discounted HSVI algorithm does not solve any of the

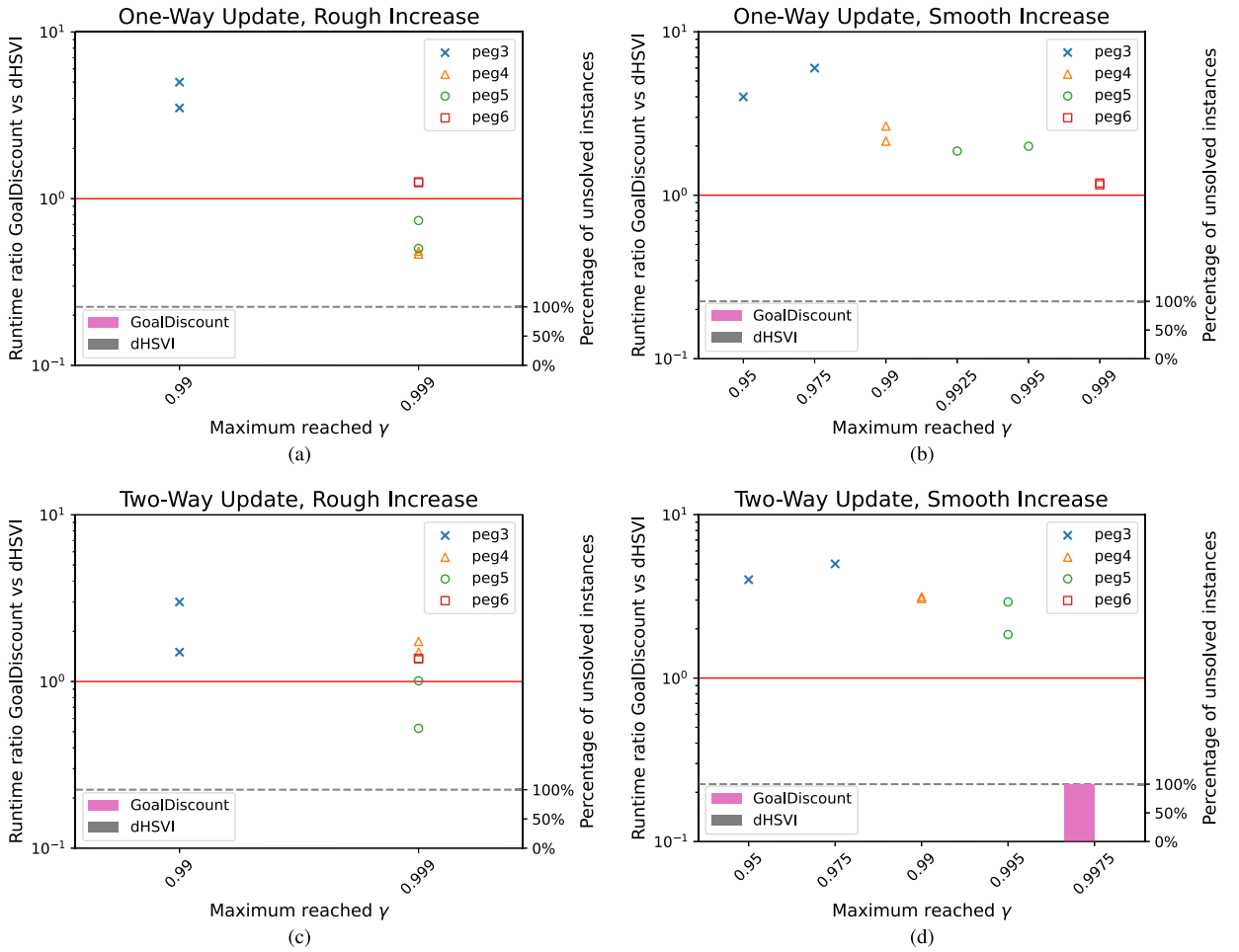


Fig. B.27. Scalability comparison between the GoalDiscount algorithm and the discounted HSVI (dHSVI) algorithm on the domain of pursuit-evasion games.

instances, while the GoalHorizon algorithm is capable of solving all of them. Furthermore, on the corresponding instance (namely *privEsc6* and *privEsc7* instances), the GoalHorizon algorithm does not get even close to the time limit. In terms of actual runtimes, the GoalHorizon algorithm is capable of finding solutions for *privEsc6* instances in 417 - 1663 seconds and for *privEsc7* instances in 3455 - 24477 seconds. If the GoalHorizon algorithm did not find the solution (instances of *privEsc7*) in the given time limit (Fig. B.26a — $\gamma = 0.9999$, Fig. B.26b — $\gamma = 0.9995$, Fig. B.26c — $\gamma = 0.9999$, and Fig. B.26d — $\gamma = 0.99975$), it is always the case that the discounted HSVI algorithm was not able to find a solution in the given time limit as well. We think that the better performance of the GoalHorizon algorithm on the privilege escalation domain compared to the pursuit-evasion domain is due to more information provided to the partially informed player (the defender) during the course of the game (after a successful attack on any layer, the defender knows the set of possible targets of that attack which helps with the estimation of possible attack targets on the following layer) and thus more structured belief space.

B.3.2. GoalDiscount vs discounted HSVI

For the GoalDiscount algorithm on the privilege escalation domain (Fig. B.27), we can again see that under one-way update setting it compares slightly better to the discounted HSVI algorithm. Furthermore, unlike in the case of the GoalHorizon algorithm, it scales significantly better with the increased discount factor to the point it is able to match or even outperform the discounted HSVI algorithm. This is mostly visible for situations where the maximum reached discount factor γ was obtained by the GoalDiscount algorithm under rough increase computational setting (Fig. B.27a and Fig. B.27c). This is mostly due to the fact that the discounted HSVI algorithm has to operate with higher discount factors compared to the smooth increase.¹⁸ Because of this, we can also spot

¹⁸ Note that using lower values of discount factor γ obtained under smooth increase computational setting lowers runtime of the discounted HSVI algorithm but in the same time it results in a worse performance of found solution under undiscounted setting (see Section 6.3).

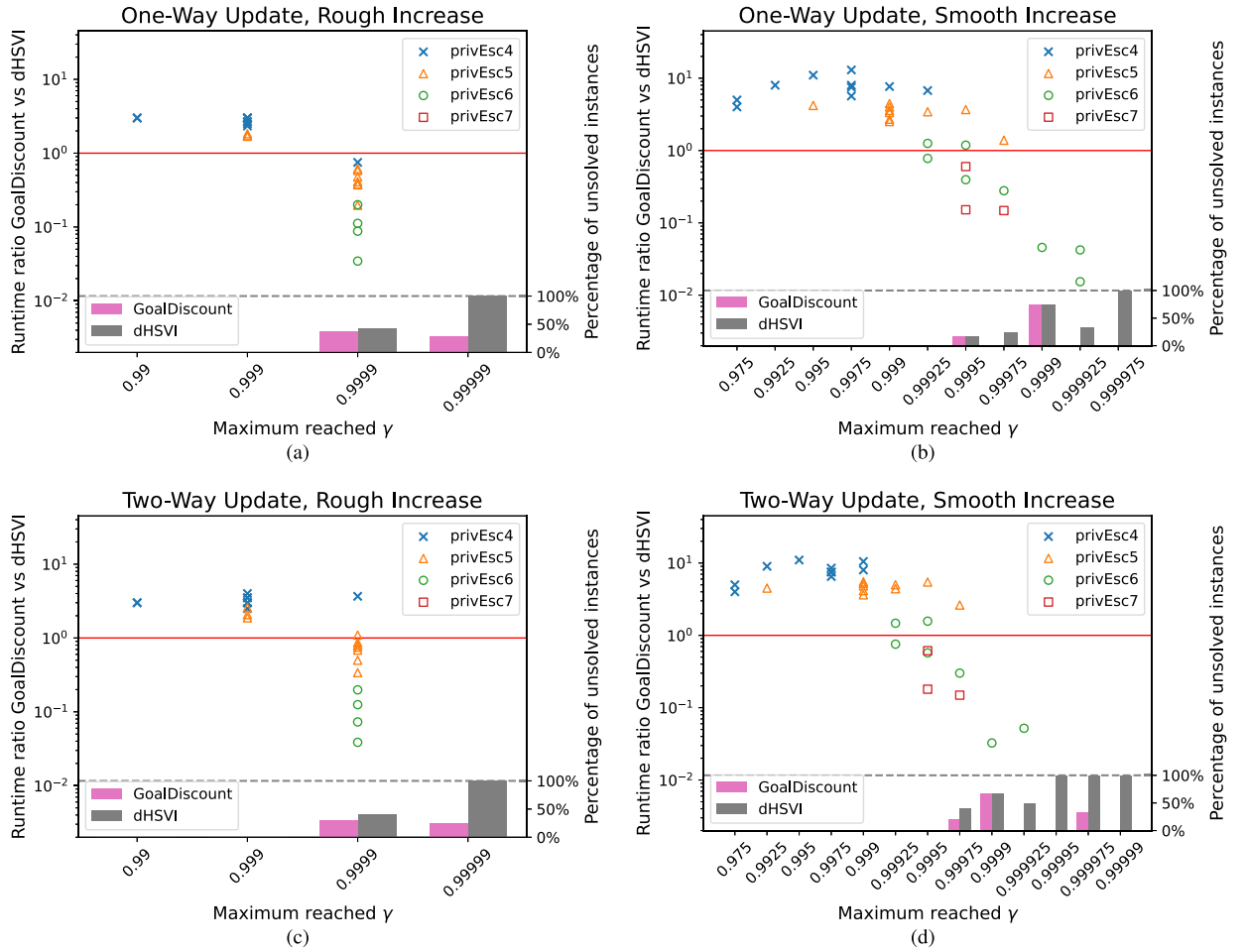


Fig. B.28. Scalability comparison between the GoalDiscount algorithm and the discounted HSVI (dHSVI) algorithm on the domain of privilege escalation games.

the same trend of better scalability with a higher discount factor for situations where the maximum reached discount factor γ was obtained under smooth increase setting (Fig. B.27c and Fig. B.27d) but it is not that pronounced.¹⁹

Better scalability with increased discount factor for the GoalDiscount algorithm compared to discounted HSVI algorithm is further highlighted on the privilege escalation domain (Fig. B.28).²⁰ Similarly to the comparison on pursuit-evasion games, we can observe that the GoalDiscount algorithm compares to the discounted HSVI algorithm better when considering rough increase computational setting. The results for smooth increase computational setting (Fig. B.28b and Fig. B.28d) then allow us to distribute the results to more values of discount factor γ and better observe how both algorithms perform. We can see that there are some instances that the GoalDiscount algorithm was not able to solve in the given time limit. However, these instances do not require the largest values of discount factor γ (assuming smooth increase computational setting), which was hidden for rough increase computational setting and the inability to solve them in the given time limit is caused rather by being larger (all of them are instances of privEsc7) and more complex rather than larger discount factor (the same instances were not solved by the discounted HSVI algorithm either). For the largest value of the discount factor γ , we can observe that the GoalDiscount algorithm solves all the instances while the discounted HSVI algorithm is unable to solve any of them. Furthermore, the GoalDiscount algorithm solves these instances with a large portion of the given time limit being unused. If assuming one-way update, smooth increase computational setting combination, the instances requiring the largest value of discount factor $\gamma = 0.999975$ were all solved by the GoalDiscount algorithm under 8600 seconds.

¹⁹ Unsolved instances for the GoalDiscount algorithm (Fig. B.27d) are not due to exceeding the time limit but running out of memory due to two-way update computational setting and thus using a larger number of alpha vectors and points to represent corresponding bounds during the computation. The algorithm was able to find the solution with a memory limit increased to 256 GB with corresponding runtime ratios $\frac{\text{goal_algorithm}}{\text{baseline}}$ being 2.62 and 1.76 respectively.

²⁰ However, compared to the results of the GoalHorizon algorithm (Fig. B.26) we can observe that the performance of the GoalDiscount algorithm is slightly worse in terms of unsolved instances.

Table C.8
Generally used symbols.

Symbol	Description
c/r	Cost/Reward (we use this interchangeably; costs of player 2 are modeled as rewards of player 1, and costs of player 1 are modeled as his negative rewards)
G	Game/Problem
G'	Game/Problem assuming discount factor γ
S	Set of states
A_i	Set of actions of player i
O	Set of observations of imperfectly informed player (player 1)
o_{reach}	Terminal observation (an observation indicating that goal was reached)
T	Transition function
R	Reward function
b_{init}	Initial belief
γ	Discount factor
γ^*	“Optimal” discount factor (sufficient for avoiding suboptimal decisions)
ε	Target gap
V^*	Optimal value function in terms of problem being solved
V^k	Value function of finite-horizon game with horizon k
V_h	Partition of value function with h stages remaining to the maximum horizon
V_h^*	Partition of optimal value function with h stages remaining to the maximum horizon
$\mathcal{H}$	Maximum horizon
H	Backup operator for undiscounted problems
H^γ	Backup operator for discounted problems
$\sigma_{\text{unif}} / \sigma_{\text{unif}}^k$	Uniform strategy (if the game has finite horizon k we use σ_{unif}^k)
$\sigma_{\text{perf}} / \sigma_{\text{perf}}^k$	Optimal strategy profile of player 1 assuming perfect information version of the game (if the game has finite horizon k we use σ_{perf}^k)
val^σ	Value of strategy σ
Γ	Set of alpha vectors representing lower bound
Υ	Set of points representing upper bound
$\overline{X}$	Upper bound on the value of X
$\underline{X}$	Lower bound on the value of X

Table C.9
Symbols specific for the GoalHorizon algorithm.

Symbol	Description
G_k	k -cutoff game (finite-horizon game with horizon k)
$\overline{V}^+$	Auxiliary upper bound
Γ_i	Partition of the set of alpha vectors representing lower bound at stage i
Υ_i	Partition of the set of points representing upper bound at stage i
$\varepsilon_{\text{inner}}$	Gap achieved by the inner loop of Algorithm 3 (lines 9-17)
F_k	Set of feasible updates when there are k stages remaining to reach the maximum horizon

Table C.10
Symbols specific for the GoalDiscount algorithm.

Symbol	Description
V_γ	Value function of the game assuming discount factor $\gamma < 1$
V_1	Value function of the game assuming discount factor $\gamma = 1$ (i.e., undiscounted problem)
γ_{init}	Initial value of the discount factor
val_γ^σ	Value of strategy σ in terms of V_γ
val_1^σ	Value of strategy σ in terms of V_1

Appendix C. Notation used in the paper

Here, we provide a list of symbols used in the paper. To make the list easier to follow, we divide it into three separate lists: generally used symbols (Table C.8), symbols specific for the GoalHorizon algorithm (Table C.9), and symbols specific for the GoalDiscount algorithm (Table C.10).

References

- [1] Francesco Amigoni, Nicola Basilico, A game theoretical approach to finding optimal strategies for pursuit evasion in grid environments, in: Proceedings of the 2012 IEEE International Conference on Robotics and Automation, IEEE, 2012, pp. 2155–2162.
- [2] Raghav Aras, Alain Dutech, François Charpillet, Mixed integer linear programming for exact finite-horizon planning in decentralized POMDPs, in: Proceedings of the International Conference on Automated Planning and Scheduling-ICAPS 2007, 2007, pp. 18–25.
- [3] Yu Bai, Chi Jin, Song Mei, Tiancheng Yu, Near-optimal learning of extensive-form games with imperfect information, in: Proceedings of the International Conference on Machine Learning, PMLR, 2022, pp. 1337–1382.

- [4] Nicola Basilico, Nicola Gatti, Francesco Amigoni, et al., Leader-follower strategies for robotic patrolling in environments with arbitrary topologies, in: *Proceedings of the International Joint Conference on Autonomous Agents and Multi Agent Systems (AAMAS)*, 2009, pp. 57–64.
- [5] Nicola Basilico, Andrea Celli, Giuseppe De Nittis, Nicola Gatti, Coordinating multiple defensive resources in patrolling games with alarm systems, in: *Proceedings of the 16th Conference on Autonomous Agents and MultiAgent Systems*, 2017, pp. 678–686.
- [6] Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław Debiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, et al., Dota 2 with large scale deep reinforcement learning. arXiv preprint, arXiv:1912.06680, 2019.
- [7] Dimitri P. Bertsekas, John N. Tsitsiklis, An analysis of stochastic shortest path problems, *Math. Oper. Res.* 16 (3) (1991) 580–595.
- [8] Dimitri P. Bertsekas, *Dynamic Programming and Optimal Control*, vol. 1, Athena Scientific, Belmont, MA, 1995.
- [9] Blai Bonet, Hector Geffner, Solving POMDPs: RTDP-Bel vs. point-based algorithms, in: *Proceedings of the IJCAI*, Pasadena CA, 2009, pp. 1641–1646.
- [10] Yunxia Chen, Qing Zhao, Vikram Krishnamurthy, Dejan Djonin, Transmission scheduling for optimizing sensor network lifetime: a stochastic shortest path approach, *IEEE Trans. Signal Process.* 55 (5) (2007) 2294–2309.
- [11] Liyu Chen, Haipeng Luo, Chen-Yu Wei, Minimax regret for stochastic shortest path with adversarial costs and known transition, arXiv preprint, arXiv:2012.04053, 2020.
- [12] Timothy H. Chung, Geoffrey A. Hollinger, Volkan Isler, Search and pursuit-evasion in mobile robotics, *Auton. Robots* 31 (4) (2011) 299–316.
- [13] Aurélien Delage, Olivier Buffet, Jilles S. Dibangoye, Abdallah Saffidine, HSVI can solve zero-sum partially observable stochastic games, *Dyn. Games Appl.* (2023) 1–55.
- [14] Jean Alexis Delamer, Yoko Watanabe, Caroline P. Carvalho Chane, Solving path planning problems in urban environments based on a priori sensor availability and execution error propagation, in: *AIAA Scitech 2019 Forum*, 2019.
- [15] Maxim Egorov, Mykel J. Kochenderfer, Jaak J. Uudmae, Target surveillance in adversarial environments using POMDPs, in: *30th AAAI Conference on Artificial Intelligence*, AAAI 2016, 2016, pp. 2473–2479.
- [16] Rosemary Emery-Montemerlo, Geoff Gordon, Jeff Schneider, Sebastian Thrun, Approximate solutions for partially observable stochastic games with common payoffs, in: *Proceedings of the Third International Joint Conference on Autonomous Agents and Multiagent Systems*, 2004, in: *AAMAS 2004*, IEEE, 2004, pp. 136–143.
- [17] Fei Fang, Thanh Nguyen, Rob Pickles, Wai Lam, Gopalasamy Clements, Bo An, Amandeep Singh, Milind Tambe, Andrew Lemieux, Deploying PAWS: field optimization of the protection assistant for wildlife security, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, 2016, pp. 3966–3973.
- [18] Sam Ganzfried, Tuomas Sandholm, Computing equilibria in multiplayer stochastic games of imperfect information, in: *Proceedings of the 21st International Joint Conference on Artificial Intelligence*, 2009, pp. 140–146.
- [19] Thomas C. Hales, Historical overview of the Kepler conjecture, in: *The Kepler Conjecture: The Hales-Ferguson Proof*, 2011, pp. 65–82.
- [20] Eric A. Hansen, Daniel S. Bernstein, Shlomo Zilberstein, Dynamic programming for partially observable stochastic games, in: *Proceedings of the AAAI*, vol. 4, 2004, pp. 709–715.
- [21] Kristoffer Arnsfelt Hansen, Michal Koucky, Peter Bro Miltersen, Winning concurrent reachability games requires doubly-exponential patience, in: *Proceedings of the 2009 24th Annual IEEE Symposium on Logic in Computer Science*, IEEE, 2009, pp. 332–341.
- [22] Karel Horák, Branislav Božanský, Michal Pěchouček, Heuristic search value iteration for one-sided partially observable stochastic games, in: *Proceedings of the 31st AAAI Conference on Artificial Intelligence*, 2017, pp. 558–564.
- [23] Karel Horák, Branislav Božanský, Krishnendu Chatterjee, Goal-HSVI: Heuristic search value iteration for goal-POMDPs, in: *IJCAI International Joint Conference on Artificial Intelligence*, 2018, pp. 4764–4770.
- [24] Karel Horák, Branislav Božanský, Petr Tomášek, Christopher Kiekintveld, Charles Kamhoua, Optimizing honeypot strategies against dynamic lateral movement using partially observable stochastic games, *Comput. Secur.* 87 (2019) 101579.
- [25] Karel Horák, Branislav Božanský, Vojtěch Kovářik, Christopher Kiekintveld, Solving zero-sum one-sided partially observable stochastic games, *Artif. Intell.* 316 (2023) 103838.
- [26] Volkan Isler, Nikhil Karnad, The role of information in the cop-robber game, *Theor. Comput. Sci.* 399 (3) (2008) 179–190.
- [27] Kerem Kaynar, A taxonomy for attack graph generation and usage in network security, *J. Inf. Secur. Appl.* 29 (2016) 27–56.
- [28] Y.F. Khalil, A novel probabilistically timed dynamic model for physical security attack scenarios on critical infrastructures, *Process Saf. Environ. Prot.* 102 (2016).
- [29] Daphne Koller, Nimrod Megiddo, Bernhard Von Stengel, Efficient computation of equilibria for extensive two-person games, *Games Econ. Behav.* 14 (2) (1996) 247–259.
- [30] Igor Kotenko, Mikhail Stepashkin, Attack graph based evaluation of network security, *Lect. Notes Comput. Sci.* 4237 (2006) 216–227.
- [31] Vojtěch Kovářik, Dominik Seitz, Viliam Lišý, Jan Rudolf, Shuo Sun, Karel Ha, Value functions for depth-limited solving in zero-sum imperfect-information games, *Artif. Intell.* 314 (2023) 103805.
- [32] Akshat Kumar, Shlomo Zilberstein, Dynamic programming approximations for partially observable stochastic games, in: *Proceedings of the Twenty-Second International FLAIRS Conference*, AAAI Press, 2009, pp. 547–552.
- [33] Sejoon Lim, Christian Sommer, Evdokia Nikolova, Daniela Rus, Practical route planning under delay uncertainty: stochastic shortest path queries, in: *Proceedings of the Robotics: Science and Systems*, vol. 8, 2013, pp. 249–256.
- [34] Omid Madani, Steve Hanks, Anne Condon, On the undecidability of probabilistic planning and infinite-horizon partially observable Markov decision problems, in: *Proceedings of the AAAI/IAAI*, 1999, pp. 541–548.
- [35] Is $[0, 1] \cdot A$ a convex set?, Online, <https://math.stackexchange.com/questions/4077059>, 2021. (Accessed 8 February 2024).
- [36] Matej Moravčík, Martin Schmid, Neil Burch, Viliam Lišý, Dustin Morrill, Nolan Bard, Trevor Davis, Kevin Waugh, Michael Johanson, Michael Bowling, Deepstack: expert-level artificial intelligence in heads-up no-limit poker, *Science* 356 (6337) (2017) 508–513.
- [37] Gergely Neu, Andras Gyorgy, Csaba Szepesvári, The adversarial stochastic shortest path problem with unknown transition probabilities, in: *Proceedings of the Artificial Intelligence and Statistics*, 2012, pp. 805–813.
- [38] C. Kien Nguyen, Tansu Alpcan, Tamer Basar, Security games with incomplete information, in: *Proceedings of the 2009 IEEE International Conference on Communications*, IEEE, 2009, pp. 1–6.
- [39] Hukukane Nikaidō, On a minimax theorem and its applications to functional analysis, *J. Math. Soc. Jpn.* 5 (1) (1953) 86–94.
- [40] Gethin Norman, David Parker, Marta Kwiatkowska, Sandeep Shukla, Rajesh Gupta, Using probabilistic model checking for dynamic power management, *Form. Asp. Comput.* 17 (2) (2005) 160–176.
- [41] Stephen D. Patek, Dimitri P. Bertsekas, Stochastic shortest path games, *SIAM J. Control Optim.* 37 (3) (1999) 804–824.
- [42] Stephen D. Patek, On partially observed stochastic shortest path problems, *Syst. Eng.* i (1999) 1–14.
- [43] James Pita, Manish Jain, Janusz Marecki, Fernando Ordóñez, Christopher Portway, Milind Tambe, Craig Western, Praveen Paruchuri, Sarit Kraus, Deployed ARMOR protection: the application of a game theoretic model for security at the Los Angeles international airport, in: *Proceedings of the 7th International Joint Conference on Autonomous Agents and Multiagent Systems: Industrial Track*, 2008, pp. 125–132.
- [44] Martin L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*, John Wiley & Sons, 2014.
- [45] Jérôme Renault, Xavier Venel, A distance for probability spaces, and long-term values in Markov decision processes and repeated games, 2012.
- [46] Aviv Rosenberg, Yishay Mansour, Stochastic shortest path with adversarially changing costs, arXiv preprint, arXiv:2006.11561, 2020.
- [47] Sankardas Roy, Charles Ellis, Sajjan Shiva, Dipankar Dasgupta, Vivek Shandilya, Qishi Wu, A survey of game theory as applied to network security, in: *Proceedings of the 2010 43rd Hawaii International Conference on System Sciences*, IEEE, 2010, pp. 1–10.

- [48] S. Saisubramanian, K.H. Wray, L. Pineda, S. Zilberstein, Proceedings of the planning in stochastic environments with goal uncertainty, in: 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2019, pp. 1649–1654.
- [49] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al., Mastering the game of Go with deep neural networks and tree search, *Nature* 529 (7587) (2016) 484–489.
- [50] Trey Smith, Reid Simmons, Heuristic search value iteration for POMDPs, in: Proceedings of the 20th Conference on Uncertainty in Artificial Intelligence (UAI), 2004, pp. 520–527.
- [51] Trey Smith, Reid Simmons, Point-based POMDP algorithms: improved analysis and implementation, in: Proceedings of the 21st Conference on Uncertainty in Artificial Intelligence (UAI), 2005, pp. 542–549.
- [52] Trey Smith, Probabilistic Planning for Robotic Exploration, PhD thesis, Carnegie Mellon University, Pittsburgh, PA, July 2007.
- [53] Sylvain Sorin, Stochastic games with incomplete information, in: Proceedings of the Stochastic Games and Applications, Springer, 2003, pp. 375–395.
- [54] Petr Tomášek, Karel Horák, Aditya Aradhye, Branislav Bošanský, Krishnendu Chatterjee, Solving partially observable stochastic shortest-path games, in: Zhi-Hua Zhou (Ed.), Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI-21, in: International Joint Conferences on Artificial Intelligence Organization, 2021, pp. 4182–4189, Main Track.
- [55] Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H. Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al., Grandmaster level in StarCraft II using multi-agent reinforcement learning, *Nature* 575 (7782) (2019) 350–354.
- [56] Yevgeniy Vorobeychik, Bo An, Milind Tambe, Satinder Singh, Computing solutions in infinite-horizon discounted adversarial patrolling games, in: Proceedings of the International Conference on Automated Planning and Scheduling, vol. 24, 2014, pp. 314–322.
- [57] Zijian Wang, Riccardo Spica, Mac Schwager, Game theoretic motion planning for multi-robot racing, in: Proceedings of the Distributed Autonomous Robotic Systems: The 14th International Symposium, Springer, 2019, pp. 225–238.
- [58] Auke J. Wiggers, Frans A. Oliehoek, Diederik M. Roijers, Structure in the Value Function of Two-Player Zero-Sum Games of Incomplete Information, Proceedings of the ECAI, vol. 285, IOS Press, 2016, pp. 1628–1629.
- [59] Auke Wiggers, Structure in the value function of two-player zero-sum games of incomplete information, Mém. de mast. University of Amsterdam, 2015.
- [60] Baowen Zhang, William Zhu, Zhi Xue, Mining Privilege Escalation Paths for Network Vulnerability Analysis, Proceedings of the Fourth International Conference on Fuzzy Systems and Knowledge Discovery (FSKD 2007), vol. 4, IEEE, 2007, pp. 56–60.